

ICPC/OI 算法模板集

ICPC/OI 算法模板集 (Algorithm Template Collection)1

 目录 (Table of Contents)2

 1. 快速参考2

 2. 基础数据结构 (Basic Data Structures)6

 3. 图论 (Graph Theory)17

 4. 数学与数论30

5. 字符串算法 (String Algorithms)41

 5.1 KMP 算法 (Knuth-Morris-Pratt)41

 5.2 Z 算法 (Z-Algorithm)42

 5.3 Manacher 算法 (马拉车 / 最长回文子串)43

 5.4 AC 自动机 (Aho-Corasick Automaton)45

 5.5 后缀数组 (Suffix Array)48

 5.6 字符串哈希 (Rolling Hash / String Hashing)50

 5.7 最小表示法 (Minimal Rotation / Booth's Algorithm)52

 5.8 字符串算法复杂度总结53

6. 动态规划 (Dynamic Programming)53

 6.1 背包问题 (Knapsack Problems)53

 6.2 混合背包55

 6.3 二维费用背包55

 6.4 最长上升子序列 (LIS) — $O(N \log N)$ 55

 6.5 最长公共子序列 (LCS)56

 6.6 编辑距离 (Edit Distance)57

 6.7 区间 DP (Interval DP)57

 6.8 树形 DP (Tree DP)59

 6.9 背包问题在树上61

 6.10 状压 DP (Bitmask DP)61

 6.11 数位 DP (Digit DP)63

 6.12 斜率优化 / 凸包优化 (Convex Hull Trick)64

 6.13 四边形不等式优化 (Divide and Conquer DP)65

 6.14 概率 DP / 期望 DP67

 6.15 博弈 DP67

 6.16 DP 技巧一览68

 6.17 综合例题68

ICPC/OI 算法模板集 (Algorithm Template Collection)

一本全面的竞赛编程参考手册，涵盖数据结构、图论、数学、字符串、动态规划、计算几何、网络流及高级技巧。所有实现使用标准 C++17/20。

6.18 总结68

7. 计算几何 (Computational Geometry)68

8 网络流81

 8.1 Dinic 最大流81

 8.2 MCMF 最小费用最大流84

 8.3 匈牙利算法 (指派问题)88

 8.4 Hopcroft-Karp 算法 (二分图最大匹配)91

 8.5 最小割应用93

 8.6 上下界网络流94

 8.7 复杂度速查99

 8.8 常见建图技巧100

 8.9 常见坑点100

 8.10 调试清单100

9. 高级技巧 (Advanced Techniques)100

 9.1 莫队算法 (Mo's Algorithm)100

 9.2 CDQ 分治 (CDQ Divide and Conquer)102

 9.3 整体二分 (Parallel Binary Search)102

 9.4 主席树 (Persistent Segment Tree)103

 9.5 Link-Cut Tree (LCT)104

 9.6 树链剖分 (Heavy-Light Decomposition, HLD)106

 9.7 平衡树 (Balanced Binary Search Tree)107

 9.8 Bitset 优化109

 9.9 分块 (Sqrt Decomposition)110

 9.10 虚树 (Virtual Tree)110

 9.11 三元环计数 (3-Cycle Counting)111

 9.12 Meet-in-the-Middle (折半搜索)112

 9.13 随机化技巧112

 9.14 综合例题113

附录114

 附录 A: 常见错误与注意事项114

 附录 B: 来源与致谢119

1. 快速参考

复杂度速查表 (Complexity Cheat Sheet)

算法	时间	空间	适用场景
排序			
<code>std::sort</code>	$O(N \log N)$	$O(\log N)$	通用排序
计数排序	$O(N + K)$	$O(K)$	小范围整数
基数排序	$O(N * W)$	$O(N + K)$	定长键值
查找			
二分查找	$O(\log N)$	$O(1)$	有序数组
三分查找	$O(\log N)$	$O(1)$	单峰函数
双指针	$O(N)$	$O(1)$	有序数组、滑动窗口
图论 (N 个点, M 条边)			
BFS / DFS	$O(N + M)$	$O(N)$	无权图遍历
Dijkstra (二叉堆)	$O(M \log N)$	$O(N)$	非负边权
Bellman-Ford	$O(N * M)$	$O(N)$	允许负边权
差分约束 (SPFA 判负环)	$O(N * M)$ 最坏	$O(N + M)$	不等式组求解
Floyd-Warshall	$O(N^3)$	$O(N^2)$	全源最短路, N 较小
Kruskal 最小生成树	$O(M \log M)$	$O(N)$	稀疏图
Prim 最小生成树	$O(M \log N)$	$O(N)$	稠密图
拓扑排序	$O(N + M)$	$O(N)$	DAG
SCC (Tarjan / Kosaraju)	$O(N + M)$	$O(N)$	有向图
LCA (倍增法)	$O(N \log N)$ 预处理 / $O(\log N)$ 查询	$O(N \log N)$	树上查询
字符串 (N 个字符)			
KMP	$O(N)$	$O(N)$	单模式匹配
Z 算法	$O(N)$	$O(N)$	前缀匹配
Manacher	$O(N)$	$O(N)$	最长回文子串
后缀数组	$O(N \log N)$	$O(N)$	子串查询
Aho-Corasick	$O(N + M + Z)$	$O(N * \Sigma)$	多模式匹配
数学			
埃氏筛	$O(N \log \log N)$	$O(N)$	筛出 N 以内的素数
Miller-Rabin	$O(K \log^3 N)$	$O(1)$	素性检测 (K 轮)
GCD (欧几里得算法)	$O(\log \min(a,b))$	$O(1)$	整除/约分
快速幂取模	$O(\log E)$	$O(1)$	$a^b \bmod m$
数据结构			
树状数组 (BIT/Fenwick)	$O(\log N)$	$O(N)$	前缀和、单点更新
线段树	$O(\log N)$	$O(4N)$ 或 $O(2N)$	区间查询、区间更新
ST 表 (Sparse Table)	$O(N \log N)$ 预处理 / $O(1)$ 查询	$O(N \log N)$	静态 RMQ
DSU (并查集)	$O(\alpha(N))$ 均摊	$O(N)$	连通分量
Treap / Splay	$O(\log N)$ 均摊	$O(N)$	有序集合、分裂合并
网络流			
Dinic	$O(V^2 * E)$ 一般图, $O(E\sqrt{V})$ 二分图	$O(E + V)$	最大流
MCMF (SPFA)	$O(F * V * E)$ 最坏情况	$O(V + E)$	最小费用最大流
匈牙利算法	$O(V^3)$	$O(V^2)$	分配/指派问题
计算几何			
凸包 (Graham Scan)	$O(N \log N)$	$O(N)$	最小凸多边形
最近点对	$O(N \log N)$	$O(N)$	平面最近两点
点在多边形内	$O(N)$	$O(1)$	射线法

常用类型定义与工具 (Common Typedefs and Utilities)

```
#include <bits/stdc++.h>
using namespace std;

// ---- 类型别名 ----
using ll = long long;
using ull = unsigned long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vll = vector<ll>;
using vvi = vector<vi>;
using vvll = vector<vll>;

// ---- 宏 ----
#define endl "\n"
#define spc " "
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#define per(i, a, b) for (int i = (b) - 1; i >= (a); --i)
#define eb emplace_back
#define pb push_back
#define fast ios::sync_with_stdio(false); cin.tie(nullptr);

// ---- 常量 ----
const int INF = 1e9; // 用于非加权/小权图 (如 BFS 距离)
const ll LINF = 1e18; // 用于加权图最短路 (Dijkstra 等), 确保 > 最大可能路径和
const int MOD = 1e9 + 7; // 常用质数模数, 也可用 998244353 (NTT 友好)
```

精选核心模板 (Verified from KACTL / jiangly)

以下模板来自竞赛标杆实现 (KACTL、jiangly), 简洁且久经考验, 建议优先选用。

并查集 DSU — KACTL 压缩版 (推荐)

此版本用负数大小表示根节点, 代码最精炼, 路径压缩与按大小合并一步到位。

```
```cpp
// KACTL 风格 DSU (负数 size 编码)
struct UF {
 vi e; // 正数 = 父节点, 负数绝对值 = 集合大小
 UF(int n) : e(n, -1) {}
 bool sameSet(int a, int b) { return find(a) == find(b); }
 int size(int x) { return -e[find(x)]; }
 int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
 bool join(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return false;
 if (e[a] > e[b]) swap(a, b); // a 的集合更大 (负数更小)
 e[a] += e[b]; // 合并大小
 e[b] = a; // b 指向 a
 return true;
 }
};
```

## 并查集 DSU — jiangly 风格 (显式 siz 数组)

```
// jiangly 风格 DSU (迭代 find, 显式 siz 数组)
struct DSU {
 vector<int> f, siz;
 DSU(int n) { f.resize(n); iota(f.begin(), f.end(), 0); siz.assign(n, 1); }
 int find(int x) {
 while (x != f[x]) x = f[x] = f[f[x]]; // 路径压缩 + 迭代避免递归
 return x;
 }
 bool merge(int x, int y) {
 x = find(x), y = find(y);
 if (x == y) return false;
 siz[x] += siz[y];
 f[y] = x;
 return true;
 }
 int size(int x) { return siz[find(x)]; }
};
```

## 树状数组 (Fenwick Tree) — KACTL 压缩版 (含 lower\_bound)

KACTL 风格: `pos |= pos + 1` 替代 `pos += pos & -pos`, 前缀和用 `pos &= pos - 1`。附带 `lower_bound` (在 BIT 上二分前缀和)。

```
// KACTL 风格 Fenwick Tree (1-indexed, 含 lower_bound)
struct FT {
 vector<ll> s;
 FT(int n) : s(n) {} // s 下标范围 [0, n-1]

 // 单点增加 dif
 void update(int pos, ll dif) {
 for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
 }

 // 查询前缀和 [0, pos) —— 即前 pos 个元素的和
 ll query(int pos) {
 ll res = 0;
 for (; pos > 0; pos &= pos - 1) res += s[pos - 1];
 return res;
 }

 // 区间和 [l, r) 半开区间
 ll rangeSum(int l, int r) { return query(r) - query(l); }

 // lower_bound: 找到最小的 pos 使得前缀和 >= sum (BIT 上二分)
 // 若 sum <= 0 返回 -1; 若 sum 大于总和和要求调用方保证不越界
 int lower_bound(ll sum) {
 if (sum <= 0) return -1;
 int pos = 0;
 for (int pw = 1 << 25; pw; pw >= 1) {
 if (pos + pw <= sz(s) && s[pos + pw - 1] < sum) {
 pos += pw;
 sum -= s[pos - 1];
 }
 }
 return pos;
 }
};
```

### 树状数组（Fenwick Tree）—— jiangly 模板化版（含 select）

jiangly 风格：i += i & -i 写法，支持泛型模板，含 select（等价于 lower\_bound）。

```
// jiangly 风格 Fenwick Tree (模板化, 1-indexed)
template<typename T>
struct Fenwick {
 int n;
 vector<T> a;
 Fenwick(int n_) { init(n_); }
 void init(int n_) { n = n_; a.assign(n, T{}); }

 // 单点增加值 v (0-indexed 位置 x)
 void add(int x, const T &v) {
 for (int i = x + 1; i <= n; i += i & -i) a[i - 1] = a[i - 1] + v;
 }

 // 查询前缀和 [0, x)
 T sum(int x) {
 T ans{};
 for (int i = x; i > 0; i -= i & -i) ans = ans + a[i - 1];
 return ans;
 }

 // 区间和 [l, r)
 T rangeSum(int l, int r) { return sum(r) - sum(l); }

 // select: 找到最小的 pos 使得前缀和 >= k (所有元素非负时有效)
 int select(const T &k) {
 int x = 0;
 T cur{};
 for (int i = 1 << __lg(n); i; i /= 2) {
 if (x + i <= n && cur + a[x + i - 1] <= k) {
 x += i;
 cur = cur + a[x - 1];
 }
 }
 return x;
 }
};
```

### 线段树 —— KACTL 迭代版（2N 空间，推荐）

迭代线段树：2N 空间（非 4N），常数远小于递归版。半开区间 [b, e)。支持单点修改、区间查询。本模板以区间最大值为例，换 f 和 unit 即可支持 sum/min/gcd 等。

```
// KACTL 风格迭代线段树 (2N 空间, 半开区间 [b, e))
struct Tree {
 typedef int T;
 static constexpr T unit = INT_MIN; // 单位元: max->INT_MIN, sum->0, min->INT_MAX
 T f(T a, T b) { return max(a, b); } // 结合函数

 vector<T> s;
 int n;

 // n = 元素数量, def = 初始值
 Tree(int n = 0, T def = unit) : s(2 * n, def), n(n) {}

 // 单点赋值 (非增量), 0-indexed
 void update(int pos, T val) {
 for (s[pos += n] = val; pos /= 2;)
 s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
 }

 // 区间查询 [b, e), 0-indexed, 半开
 T query(int b, int e) {
 T ra = unit, rb = unit;
 for (b += n, e += n; b < e; b /= 2, e /= 2) {
 if (b % 2) ra = f(ra, s[b++]);
 if (e % 2) rb = f(s[--e], rb);
 }
 return f(ra, rb);
 }
};
```

## 强连通分量 SCC —— jiangly Tarjan (压缩版)

jiangly 风格的 Tarjan 实现, 关键点: 回边用 `dfn[y]` 更新 `low[x]` (非 `low[y]`), 避免跨分量错误。 `bel` 数组是逆拓扑序的, 缩点后 DAG 上的 DP 可直接按 `bel` 顺序遍历。

```
// jiangly 风格 SCC (Tarjan, 压缩版)
// 关键: 回边/横叉边用 dfn[y] 更新 low[x] (已访问但未出栈的点)
struct SCC {
 int n;
 vector<vector<int>> adj;
 vector<int> stk, dfn, low, bel; // bel 值 0..cnt-1, 逆拓扑序
 int cur, cnt;

 SCC(int n) {
 this->n = n;
 adj.assign(n, {});
 dfn.assign(n, -1);
 low.resize(n);
 bel.assign(n, -1);
 cur = cnt = 0;
 }

 void addEdge(int u, int v) { adj[u].push_back(v); }

 void dfs(int x) {
 dfn[x] = low[x] = cur++;
 stk.push_back(x);
 for (auto y : adj[x]) {
 if (dfn[y] == -1) { // 树边: y 未被访问
 dfs(y);
 low[x] = min(low[x], low[y]);
 } else if (bel[y] == -1) { // 回边/横叉边: y 已访问但未出栈
 low[x] = min(low[x], dfn[y]); // 关键: 用 dfn[y] 而非 low[y]
 }
 }
 if (dfn[x] == low[x]) { // x 是 SCC 的根
 int y;
 do {
 y = stk.back();
 bel[y] = cnt;
 stk.pop_back();
 } while (y != x);
 cnt++;
 }
 }

 // 返回 bel 数组, 0..cnt-1 为逆拓扑序
 vector<int> work() {
 for (int i = 0; i < n; i++)
 if (dfn[i] == -1) dfs(i);
 return bel;
 }
};
```

## 重链剖分 HLD —— jiangly 综合版

包含 `isAncestor`、`jump`、`rootedParent` (换根后的父节点)、`rootedSize`、`rootedLca` 等高级操作。节点编号 0-indexed。

```

// jiangly 风格 HLD (重链剖分, 含换根操作)
struct HLD {
 int n;
 vector<vector<int>> adj;
 vector<int> parent, depth, top, in, out, sz, heavy;
 int cur;

 HLD(int n) {
 this->n = n;
 adj.resize(n);
 parent.resize(n, -1);
 depth.resize(n);
 top.resize(n);
 in.resize(n);
 out.resize(n);
 sz.resize(n);
 heavy.assign(n, -1);
 cur = 0;
 }

 void addEdge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }

 // 以 root 为根建树 (默认 0)
 void build(int root = 0) {
 dfs1(root);
 dfs2(root, root);
 }

 void dfs1(int u) {
 sz[u] = 1;
 int max_sz = 0;
 for (auto v : adj[u]) {
 if (v == parent[u]) continue;
 parent[v] = u;
 depth[v] = depth[u] + 1;
 dfs1(v);
 sz[u] += sz[v];
 if (sz[v] > max_sz) { max_sz = sz[v]; heavy[u] = v; }
 }
 }

 void dfs2(int u, int tp) {
 top[u] = tp;
 in[u] = cur++;
 if (heavy[u] != -1) dfs2(heavy[u], tp);
 for (auto v : adj[u]) {
 if (v != parent[u] && v != heavy[u]) dfs2(v, v);
 }
 out[u] = cur; // 半开区间 [in[u], out[u]) 为子树
 }

 // u 是否是 v 的祖先 (基于 in/out 区间)
 bool isAncestor(int u, int v) { return in[u] <= in[v] && in[v] < out[u]; }

 // LCA: O(log N)
 int lca(int u, int v) {
 while (top[u] != top[v]) {
 if (depth[top[u]] < depth[top[v]]) swap(u, v);
 u = parent[top[u]];
 }
 return depth[u] < depth[v] ? u : v;
 }

 // u 向上跳 k 步, O(log N); 若 k 超过深度返回根
 int jump(int u, int k) {
 while (u != -1) {
 if (depth[u] - depth[top[u]] >= k) return in[top[u]] + (depth[u] - depth[top[u]] - k);
 k -= depth[u] - depth[top[u]] + 1;
 u = parent[top[u]];
 }
 return -1;
 }

 // 以 root 为根时 u 的父节点
 int rootedParent(int root, int u) {
 if (root == u) return -1;
 if (!isAncestor(u, root)) return parent[u];
 return jump(root, depth[root] - depth[u] - 1);
 }

 // 以 root 为根时 u 子树大小
 int rootedSize(int root, int u) {
 if (root == u) return n;
 if (!isAncestor(u, root)) return sz[u];
 int v = jump(root, depth[root] - depth[u] - 1);
 return n - sz[v];
 }

 // 以 root 为根时的 LCA: lca(a) ^ lca(b) ^ lca(c) 技巧
 int rootedLca(int root, int a, int b) {
 int x = lca(a, b), y = lca(b, root), z = lca(root, a);
 return x ^ y ^ z;
 }
};

```

## 比赛技巧 (Contest Tips / 比赛技巧)

## 2. 基础数据结构 (Basic Data Structures)

### 2.1 前缀和与差分 (Prefix Sum & Difference Array)

用于快速区间求和（一维/二维）与区间加法操作（差分数组）。建表  $O(N)$ ，查询  $O(1)$ 。

```

// ----- 一维前缀和 -----
// 建表 $O(N)$, 区间求和 $O(1)$
vector<ll> build_prefix(const vector<ll>& a) {
 int n = sz(a);
 vector<ll> pre(n + 1);
 rep(i, 0, n) pre[i + 1] = pre[i] + a[i];
 return pre;
}
// 查询 $a[l..r]$ 的和 (θ -indexed, 闭区间)
ll range_sum(const vector<ll>& pre, int l, int r) {
 return pre[r + 1] - pre[l];
}

// ----- 二维前缀和 -----
// 建表 $O(N*M)$, 矩形求和 $O(1)$
struct Prefix2D {
 vlll p;
 Prefix2D(const vlll& a) {
 int n = sz(a), m = sz(a[0]);
 p.assign(n + 1, vll(m + 1));
 rep(i, 0, n) rep(j, 0, m)
 p[i + 1][j + 1] = a[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j];
 }
 // 查询矩形 $(r1, c1) .. (r2, c2)$ 闭区间的和
 ll query(int r1, int c1, int r2, int c2) {
 return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
 }
};

// ----- 差分数组 -----
// 区间加 $O(1)$, 还原数组 $O(N)$
// 所有加法操作完成后, 求前缀和即可得到最终数组
void range_add(vi& diff, int l, int r, int val) {
 diff[l] += val;
 if (r + 1 < sz(diff)) diff[r + 1] -= val;
}

```

### 异或差分 (XOR Difference)

区间异或修改与加法差分类似：对  $[l, r]$  整体  $\oplus x$ , 只需  $\text{diff}[l] \text{ ^= } x, \text{diff}[r+1] \text{ ^= } x$ , 前缀 XOR 还原。原理： $x \oplus x = 0x \oplus x = 0$  使得右端点后自动抵消。

```

// ----- 异或差分：区间异或 $O(1)$, 还原 $O(N)$ -----
void range_xor(vi& diff, int l, int r, int val) {
 diff[l] ^= val;
 if (r + 1 < sz(diff)) diff[r + 1] ^= val;
}
void restore_xor(const vi& diff, vi& a) {
 int cur = 0;
 rep(i, 0, sz(a)) { cur ^= diff[i]; a[i] = cur; }
}

// ----- 树上路径异或标记 (路径打 tag) -----
// $u \text{ ^= } x, v \text{ ^= } x, \text{lca} \text{ ^= } x, \text{fa}[\text{lca}] \text{ ^= } x$, 然后 DFS 合并
// ----- 边区间：边 (u, v) 影响 $[u, v-1]$ -----
// $\text{diff}[u] \text{ ^= } w; \text{diff}[v] \text{ ^= } w;$ // 右端点 $(v-1)+1 = v$

```

常见场景：区间异或修改/状态切换 (toggle)、树上路径异或标记、边集奇偶性判断。

## 2.2 树状数组 (Fenwick Tree / Binary Indexed Tree)

单点修改与前缀和查询均为  $O(\log N)$ 。无法直接处理区间修改（需要两个 BIT 实现）。空间  $O(N)$ 。

```

template <typename T>
struct Fenwick {
 int n;
 vector<T> bit;

 Fenwick(int n_) : n(n_), bit(n + 1) {}
 Fenwick(const vector<T>& a) : n(sz(a)), bit(n + 1) {
 rep(i, 0, n) add(i, a[i]); // $O(N \log N)$ 建树; $O(N)$ 版本见下
 }

 // $O(N)$ 建树 — 从数组初始化时推荐使用
 void build_linear(const vector<T>& a) {
 bit.assign(n + 1, 0);
 rep(i, 0, n) {
 bit[1 + i] += a[i];
 int j = i + 1 + ((i + 1) & -(i + 1));
 if (j <= n) bit[j] += bit[i + 1];
 }
 }

 // 在位置 i 增加 δ (0 -indexed)
 void add(int i, T delta) {
 for (++i; i <= n; i += i & -i) bit[i] += delta;
 }

 // 前缀和 $a[0..i]$ (0 -indexed 闭区间)
 T sum(int i) {
 T s = 0;
 for (++i; i > 0; i -= i & -i) s += bit[i];
 return s;
 }

 // 区间和 $a[l..r]$ (0 -indexed 闭区间)
 T range_sum(int l, int r) { return sum(r) - sum(l - 1); }

 // 查找使前缀和 $\geq k$ 的最小下标 (要求所有 $a[i] \geq 0$)
 // 也称为 find_kth — 用于顺序统计
 int lower_bound(T k) {
 int idx = 0;
 // 注意: 循环变量用 pw 而非 bit , 避免与成员变量 bit 重名
 for (int pw = 1 << (31 - __builtin_clz(n)); pw; pw >= 1) {
 int nxt = idx + pw;
 if (nxt <= n && bit[nxt] < k) {
 k -= bit[nxt];
 idx = nxt;
 }
 }
 return idx; // 返回 0 -indexed 结果; 需检查 $idx < n$
 }
};

// ---- jiangly 风格 Fenwick (精简版, 含 select) ----
// 更紧凑的写法, select(k) 返回第 k 小的下标 (1 -indexed)
template <typename T>
struct FenwickJ {
 int n;
 vector<T> a;

 FenwickJ(int n_ = 0) : n(n_), a(n + 1) {}

 // 单点加
 void add(int x, const T& v) {
 for (int i = x; i <= n; i += i & -i) a[i] += v;
 }

 // 前缀和 $[1..x]$
 T sum(int x) {
 T ans = 0;
 for (int i = x; i > 0; i -= i & -i) ans += a[i];
 return ans;
 }

 // 区间和 $[l..r]$ (1 -indexed 闭区间)
 T range_sum(int l, int r) { return sum(r) - sum(l - 1); }

 // 查找第 k 小的元素下标 (1 -indexed), 要求所有权重非负
 int select(const T& k) {
 int x = 0;
 T cur = 0;
 for (int i = 1 << (31 - __builtin_clz(n)); i; i >= 1) {
 if (x + i <= n && cur + a[x + i] < k) {
 x += i;
 cur += a[x];
 }
 }
 return x + 1;
 }
};

// ---- 树状数组实现区间加 + 单点查 ----
// 直接在 BIT 中维护差分数组即可
// 区间 $[l, r]$ 加 v : add(l, v), add(r+1, -v)
// 查询点 i 的值: sum(i)

// ---- 树状数组实现区间加 + 区间求和 ----
// 维护两个 BIT: bit1 维护差分, bit2 维护 $i * diff[i]$
template <typename T>
struct FenwickRange {
 int n;
 Fenwick<T> bit1, bit2;
 FenwickRange(int n_) : n(n_), bit1(n), bit2(n) {}

 void _add(Fenwick<T>& b, int i, T v) {
 for (++i; i <= n; i += i & -i) b.bit[i] += v; // 直接访问 bit 成员以提速
 }

 T _sum(Fenwick<T>& b, int i) {
 T s = 0;
 for (++i; i > 0; i -= i & -i) s += b.bit[i];
 return s;
 }

 void range_add(int l, int r, T v) {
 _add(bit1, l, v);

```



```

 _add(bit1, r + 1, -v);
 _add(bit2, l, v * l);
 _add(bit2, r + 1, -v * (r + 1));
}

T prefix_sum(int i) {
 return _sum(bit1, i) * (i + 1) - _sum(bit2, i);
}

T range_sum(int l, int r) { return prefix_sum(r) - prefix_sum(l - 1); }
};

```

### 2.3 线段树 (Segment Tree)

区间查询与区间修改均为  $O(\log N)$ 。支持任意满足结合律的运算。空间  $O(4N)$ 。

```

template <typename T>
struct SegTree {
 int n;
 vector<T> tree, lazy;
 vector<bool> has_lazy;
 T id;

 SegTree(int n_, T id_ = T{}) : n(n_), id(id_) {
 tree.assign(4 * n, id);
 lazy.assign(4 * n, id);
 has_lazy.assign(4 * n, false);
 }

 // 合并两个节点：按需覆盖为自定义运算
 // 求和：return a + b; 求最小值：return min(a, b);
 T merge(T a, T b) { return a + b; }

 // 对代表区间 [l, r] 的节点施加懒标记
 void apply(int idx, int l, int r, T val) {
 tree[idx] += val * (r - l + 1); // 区间求和
 // tree[idx] += val; // 区间最值
 lazy[idx] += val;
 has_lazy[idx] = true;
 }

 void push(int idx, int l, int r) {
 if (has_lazy[idx]) {
 int mid = (l + r) / 2;
 apply(idx * 2, l, mid, lazy[idx]);
 apply(idx * 2 + 1, mid + 1, r, lazy[idx]);
 lazy[idx] = id;
 has_lazy[idx] = false;
 }
 }

 // 从数组建树：O(N)
 void build(int idx, int l, int r, const vector<T>& a) {
 if (l == r) {
 tree[idx] = a[l];
 return;
 }
 int mid = (l + r) / 2;
 build(idx * 2, l, mid, a);
 build(idx * 2 + 1, mid + 1, r, a);
 tree[idx] = merge(tree[idx * 2], tree[idx * 2 + 1]);
 }

 // 单点修改：a[pos] = val
 void point_update(int idx, int l, int r, int pos, T val) {
 if (l == r) {
 tree[idx] = val;
 return;
 }
 push(idx, l, r);
 int mid = (l + r) / 2;
 if (pos <= mid) point_update(idx * 2, l, mid, pos, val);
 else point_update(idx * 2 + 1, mid + 1, r, pos, val);
 tree[idx] = merge(tree[idx * 2], tree[idx * 2 + 1]);
 }

 // 区间修改：将 [ql, qr] 内所有元素加 val
 void range_update(int idx, int l, int r, int ql, int qr, T val) {
 if (ql > r || qr < l) return;
 if (ql <= l && r <= qr) {
 apply(idx, l, r, val);
 return;
 }
 push(idx, l, r);
 int mid = (l + r) / 2;
 range_update(idx * 2, l, mid, ql, qr, val);
 range_update(idx * 2 + 1, mid + 1, r, ql, qr, val);
 tree[idx] = merge(tree[idx * 2], tree[idx * 2 + 1]);
 }

 // 区间查询：聚合 [ql, qr]
 T query(int idx, int l, int r, int ql, int qr) {
 if (ql > r || qr < l) return id;
 if (ql <= l && r <= qr) return tree[idx];
 push(idx, l, r);
 int mid = (l + r) / 2;
 return merge(query(idx * 2, l, mid, ql, qr),
 query(idx * 2 + 1, mid + 1, r, ql, qr));
 }

 // ----- 包装方法 (0-indexed) -----
 void build(const vector<T>& a) { build(1, 0, n - 1, a); }
 void point_update(int pos, T val) { point_update(1, 0, n - 1, pos, val); }
 void range_update(int l, int r, T val) { range_update(1, 0, n - 1, l, r, val); }
 T query(int l, int r) { return query(1, 0, n - 1, l, r); }
};

// ----- 线段树求最小值/最大值 (无懒标记) -----
// 只需将 merge 改为 return min(a, b), 并用 INF 作为单位元
template <typename T>
struct SegTreeMin {
 int n; vector<T> tree; T id;
 SegTreeMin(int n_, T id_) : n(n_), id(id_) { tree.assign(4 * n, id); }
 T merge(T a, T b) { return min(a, b); }
 // ... 与上面相同的建树/修改/查询结构, 省略懒标记
};

// ----- KACTL 风格迭代线段树 (2N 内存, 半开区间 [b, e)) -----
// 特点：数组大小 2*n, 自底向上, 无需递归, 常数小
// 单位元 id 必须满足 merge(x, id) == merge(id, x) == x
template <typename T>
struct SegTreeIter {
 int n;
 vector<T> tree;
 T id;

 SegTreeIter(int n_, T id_ = T{}) : n(n_), id(id_) {
 tree.assign(2 * n, id);
 }
};

```

```

// 从数组建树 $O(N)$
void build(const vector<T>& a) {
 for (int i = 0; i < n; ++i) tree[n + i] = a[i];
 for (int i = n - 1; i > 0; --i)
 tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
}

T merge(T a, T b) { return a + b; } // 按需覆盖

// 单点修改: $a[pos] = val$
void point_update(int pos, T val) {
 for (tree[pos += n] = val; pos > 1; pos >>= 1)
 tree[pos >> 1] = merge(tree[pos], tree[pos ^ 1]);
}

// 区间查询 $[l, r)$ — 半开区间
T query(int l, int r) {
 T resl = id, resr = id;
 for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
 if (l & 1) resl = merge(resl, tree[l++]);
 if (r & 1) resr = merge(tree[--r], resr);
 }
 return merge(resl, resr);
}

// 区间加 (不带懒标记的简单版本仅支持单点修改与查询)
// 如需区间加, 需维护额外懒标记数组, 写法类似递归版本
};

```

## 2.4 ST 表 (Sparse Table)

静态区间最小值/最大值/gcd 查询  $O(1)$ 。建表  $O(N \log N)$ 。不支持修改。

```

template <typename T>
struct SparseTable {
 int n;
 vector<vector<T>>> st;
 vi log2;

 SparseTable(const vector<T>& a) : n(sz(a)) {
 log2.resize(n + 1);
 log2[1] = 0;
 rep(i, 2, n + 1) log2[i] = log2[i / 2] + 1;

 int K = log2[n] + 1;
 st.assign(K, vector<T>(n));
 rep(i, 0, n) st[0][i] = a[i];

 rep(k, 1, K) {
 for (int i = 0; i + (1 << k) <= n; ++i)
 st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
 }
 }

 // 区间最小值查询 $[l, r]$ 闭区间, 0 -indexed
 T query(int l, int r) {
 int k = log2[r - l + 1];
 return min(st[k][l], st[k][r - (1 << k) + 1]);
 }
};

// ---- 二维 ST 表 ----
// 静态二维 RMQ: 建表 $O(N * M * \log N * \log M)$, 查询 $O(1)$
template <typename T>
struct SparseTable2D {
 int n, m;
 vector<vector<vector<vector<T>>>>> st;
 vi log2;

 SparseTable2D(const vll& a) : n(sz(a)), m(sz(a[0])) {
 int maxSz = max(n, m);
 log2.resize(maxSz + 1);
 log2[1] = 0;
 rep(i, 2, maxSz + 1) log2[i] = log2[i / 2] + 1;

 int Kn = log2[n] + 1, Km = log2[m] + 1;
 st.assign(Kn, vector<vector<vector<T>>>>(Km, vll(n, vll(m))));
 rep(i, 0, n) rep(j, 0, m) st[0][0][i][j] = a[i][j];
 // ... (类似地展开 $k1, k2$)
 // 内存开销较大: 仅在网格较小 ($n, m \leq 500$) 时使用
 }
};

```

## 2.5 并查集 (Disjoint Set Union / Union-Find)

路径压缩 + 按大小/按秩合并, 实现近似常数时间的合并与查找。均摊  $O(\alpha(N))$ 。

```

struct DSU {
 vi par, sz;
 int comps; // 连通分量个数

 DSU(int n) : par(n), sz(n, 1), comps(n) {
 iota(all(par), 0);
 }

 int find(int x) {
 return par[x] == x ? x : par[x] = find(par[x]);
 }

 // 合并成功返回 true, 已在同一集合中返回 false
 bool unite(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return false;
 if (sz[a] < sz[b]) swap(a, b);
 par[b] = a;
 sz[a] += sz[b];
 comps--;
 return true;
 }

 bool same(int a, int b) { return find(a) == find(b); }
 int size(int x) { return sz[find(x)]; }
};

// ---- KACTL 风格紧凑 DSU (单个 vector e, 负数表示集合大小) ----
// 更简洁且内存友好的写法: e[i] >= 0 表示父节点, e[i] < 0 表示根且 -e[i] 为集合大小
struct DSUCompact {
 vi e;

 DSUCompact(int n) : e(n, -1) {}

 // 查找 (带路径压缩)
 int find(int x) {
 return e[x] < 0 ? x : e[x] = find(e[x]);
 }

 // 返回合并后的集合大小; 如果已在同一集合则返回当前大小
 int unite(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return -e[a];
 if (e[a] > e[b]) swap(a, b); // e[a] 更负 (集合更大)
 e[a] += e[b];
 e[b] = a;
 return -e[a];
 }

 bool same(int a, int b) { return find(a) == find(b); }

 // 获取集合大小 (仅当 x 为根时调用)
 int size(int x) { return -e[find(x)]; }
};

// ---- 可撤销并查集 (DSU with Rollback) ----
// 支持撤销最近一次合并操作。不使用路径压缩 (仅使用按秩合并)。
struct DSU_Rollback {
 vi par, rnk;
 vector<pii> history; // (节点, 旧父节点)
 int comps;

 DSU_Rollback(int n) : par(n), rnk(n, 0), comps(n) {
 iota(all(par), 0);
 }

 int find(int x) {
 while (par[x] != x) x = par[x];
 return x;
 }

 bool unite(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return false;
 if (rnk[a] < rnk[b]) swap(a, b);
 history.eb(b, par[b]);
 par[b] = a;
 comps--;
 if (rnk[a] == rnk[b]) {
 history.eb(~a, rnk[a]); // 用负数编码秩的变化
 rnk[a]++;
 }
 return true;
 }

 void rollback(int checkpoint) { // 回滚到 history 大小为 checkpoint 的状态
 while (sz(history) > checkpoint) {
 auto [u, old] = history.back(); history.pop_back();
 if (u < 0) {
 rnk[~u] = old;
 continue;
 }
 par[u] = old;
 comps++;
 }
 }

 int snapshot() { return sz(history); }
};

```

## 2.6 单调栈与单调队列 (Monotonic Stack & Monotonic Queue)

用于解决下一个/上一个更大/更小元素问题以及滑动窗口最值。时间复杂度  $O(N)$ 。

```

// ---- 下一个更大元素 ----
// 返回下标数组；若不存在更大元素则为 -1
vi next_greater(const vi& a) {
 int n = sz(a);
 vi res(n, -1);
 stack<int> st;
 per(i, 0, n) {
 while (!st.empty() && a[st.top()] <= a[i]) st.pop();
 if (!st.empty()) res[i] = st.top();
 st.push(i);
 }
 return res;
}

// ---- 滑动窗口最大值 ----
// 双端队列存储下标，队首为当前窗口最大值
vi sliding_max(const vi& a, int k) {
 int n = sz(a);
 deque<int> dq;
 vi res;
 rep(i, 0, n) {
 while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
 while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
 dq.push_back(i);
 if (i >= k - 1) res.pb(a[dq.front()]);
 }
 return res;
}

// ---- 柱状图中最大矩形 ----
// 利用单调栈 $O(N)$
ll max_rectangle_histogram(const vi& heights) {
 int n = sz(heights);
 vi left(n), right(n, n);
 stack<int> st;

 rep(i, 0, n) {
 while (!st.empty() && heights[st.top()] >= heights[i]) st.pop();
 left[i] = st.empty() ? -1 : st.top();
 st.push(i);
 }
 while (!st.empty()) st.pop();

 per(i, 0, n) {
 while (!st.empty() && heights[st.top()] >= heights[i]) st.pop();
 right[i] = st.empty() ? n : st.top();
 st.push(i);
 }

 ll ans = 0;
 rep(i, 0, n) ans = max(ans, (ll)heights[i] * (right[i] - left[i] - 1));
 return ans;
}

```

## 2.7 分块与莫队算法 (Sqrt Decomposition & Mo's Algorithm)

通用技巧：将数组分成  $\sqrt{n}$  块，块操作复杂度  $O(\sqrt{n})$ 。

```

// ---- 分块实现带单点修改的区间求和 ----
struct SqrtDecomp {
 int n, blk;
 vi a;
 vll block_sum;

 SqrtDecomp(const vi& arr) : n(sz(arr)), a(arr) {
 blk = max(1, (int)sqrt(n));
 block_sum.assign((n + blk - 1) / blk, 0);
 rep(i, 0, n) block_sum[i / blk] += a[i];
 }

 void point_update(int idx, int val) {
 block_sum[idx / blk] += val - a[idx];
 a[idx] = val;
 }

 ll range_query(int l, int r) {
 ll ans = 0;
 int bl = l / blk, br = r / blk;
 if (bl == br) {
 rep(i, l, r + 1) ans += a[i];
 } else {
 rep(i, l, (bl + 1) * blk) ans += a[i];
 rep(b, bl + 1, br) ans += block_sum[b];
 rep(i, br * blk, r + 1) ans += a[i];
 }
 return ans;
 }
};

// ---- 莫队算法 (普通莫队) ----
// 离线处理数组区间查询, $O((N+Q) * \sqrt{N})$, 适用于 $N, Q \sim 1e5$
struct MoQuery {
 int l, r, idx;
};

void mo_algorithm(vi& a, vector<MoQuery>& queries) {
 int n = sz(a), q = sz(queries);
 int blk = max(1, (int)(n / sqrt(q))); // 希尔伯特序更优, 见下方

 sort(all(queries), [&](const MoQuery& x, const MoQuery& y) {
 int bx = x.l / blk, by = y.l / blk;
 if (bx != by) return bx < by;
 return bx & 1 ? x.r > y.r : x.r < y.r; // 奇偶优化
 // 奇数块 r 降序, 偶数块 r 升序
 });

 int cur_l = 0, cur_r = -1;
 // 在此维护全局状态 (如计数数组 freq、当前答案 ans)
 // 对于每个查询:
 // while cur_l > q.l: add(--cur_l)
 // while cur_r < q.r: add(++cur_r)
 // while cur_l < q.l: remove(cur_l++)
 // while cur_r > q.r: remove(cur_r--)
}

// ---- 莫队算法 (希尔伯特序优化) ----
// 希尔伯特曲线排序, 常数优于普通分块排序, 实测快 30%~50%
inline int64_t hilbert_order(int x, int y, int pow, int rotate) {
 if (pow == 0) return 0;
 int hpow = 1 << (pow - 1);
 int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
 seg = (seg + rotate) & 3;
 int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
 int nrot = (rotate + ((seg == 0 || seg == 3) ? 1 : 0)) & 3;
 int64_t sub_square_size = 1LL << (2 * (pow - 1));
 int64_t ans = seg * sub_square_size;
 int64_t add = hilbert_order(nx, ny, pow - 1, nrot);
 ans += (seg == 1 || seg == 2) ? add : (sub_square_size - add - 1);
 return ans;
}

// ---- 具体 add / remove 示例: 统计区间内不同元素个数 ----
// 全局状态
int distinct_cnt = 0;
vi freq; // 值域足够大时可用 map / 离散化

void add(int pos, const vi& a) {
 int val = a[pos];
 if (freq[val] == 0) distinct_cnt++;
 freq[val]++;
}

void remove(int pos, const vi& a) {
 int val = a[pos];
 freq[val]--;
 if (freq[val] == 0) distinct_cnt--;
}

// 莫队主循环示例
void solve_mo(vi& a, vector<MoQuery>& queries) {
 int n = sz(a), q = sz(queries);
 int K = max(1, (int)ceil(log2(n))); // 希尔伯特序参数
 int blk = max(1, (int)(n / sqrt(q)));

 // 计算希尔伯特序
 vector<int64_t> h_order(q);
 rep(i, 0, q) h_order[i] = hilbert_order(queries[i].l, queries[i].r, K, 0);

 vi order(q);
 iota(all(order), 0);
 sort(all(order), [&](int i, int j) { return h_order[i] < h_order[j]; });

 // 初始化全局状态
 int max_val = *max_element(all(a));
 freq.assign(max_val + 1, 0);
 distinct_cnt = 0;

 int cur_l = 0, cur_r = -1;
 vi ans(q);

 for (int qi : order) {
 auto [ql, qr, idx] = queries[qi];

```

```

 while (cur_l > ql) add(--cur_l, a);
 while (cur_r < qr) add(++cur_r, a);
 while (cur_l < ql) remove(cur_l++, a);
 while (cur_r > qr) remove(cur_r--, a);
 ans[idx] = distinct_cnt;
 }

 // ans 中存储每个查询的结果
}

// ---- 带修改莫队 ----
// 引入时间维度： $(l/block, r/block, time)$ 。块大小取 $N^{2/3}$
// 复杂度 $O(N^{5/3})$ 。查询和修改交替出现。

```

## 2.8 字典树 (Trie / Prefix Tree)

插入和查找字符串均为  $O(L)$ ，其中  $L$  为字符串长度。支持前缀查询。

```

struct Trie {
 static const int ALPHA = 26;
 struct Node {
 array<int, ALPHA> nxt;
 int cnt; // 以此节点结尾的字符串数量
 int visited; // 经过此节点的字符串数量
 Node() : cnt(0), visited(0) { nxt.fill(-1); }
 };

 vector<Node> t;
 Trie() { t.eb(); } // 根节点在索引 0

 void insert(const string& s) {
 int v = 0;
 t[v].visited++;
 for (char ch : s) {
 int c = ch - 'a';
 if (t[v].nxt[c] == -1) {
 t[v].nxt[c] = sz(t);
 t.eb();
 }
 v = t[v].nxt[c];
 t[v].visited++;
 }
 t[v].cnt++;
 }

 bool search(const string& s) {
 int v = 0;
 for (char ch : s) {
 int c = ch - 'a';
 if (t[v].nxt[c] == -1) return false;
 v = t[v].nxt[c];
 }
 return t[v].cnt > 0;
 }

 int prefix_count(const string& s) { // 统计以 s 为前缀的字符串数量
 int v = 0;
 for (char ch : s) {
 int c = ch - 'a';
 if (t[v].nxt[c] == -1) return 0;
 v = t[v].nxt[c];
 }
 return t[v].visited;
 }
};

// ---- 01 字典树 (Binary Trie) 用于异或问题 ----
// 最大异或对、前缀异或查询等
struct BinaryTrie {
 static const int BITS = 30; // 适用于值 up to 1e9
 struct Node {
 array<int, 2> nxt;
 int cnt;
 Node() : cnt(0) { nxt.fill(-1); }
 };
 vector<Node> t;
 BinaryTrie() { t.eb(); }

 void insert(int x) {
 int v = 0;
 per(b, BITS, 0) {
 int bit = (x >> b) & 1;
 if (t[v].nxt[bit] == -1) {
 t[v].nxt[bit] = sz(t);
 t.eb();
 }
 v = t[v].nxt[bit];
 t[v].cnt++;
 }
 }

 // 查询与 x 异或值最大的结果
 int max_xor(int x) {
 // 修正: 检查 cnt > 0 而非 nxt == -1, 否则删除后可能走到空节点
 if (t[0].cnt == 0) return 0;
 int v = 0, ans = 0;
 per(b, BITS, 0) {
 int bit = (x >> b) & 1;
 int want = bit ^ 1;
 if (t[v].nxt[want] != -1 && t[t[v].nxt[want]].cnt > 0) {
 ans |= (1 << b);
 v = t[v].nxt[want];
 } else {
 v = t[v].nxt[bit];
 }
 }
 return ans;
 }

 void erase(int x) { // 仅当 x 已插入时才调用
 int v = 0;
 per(b, BITS, 0) {
 int bit = (x >> b) & 1;
 int nxt = t[v].nxt[bit];
 v = nxt;
 t[v].cnt--;
 }
 // 修正: 同步更新根节点的 cnt, 确保空树判断正确
 t[0].cnt--;
 }
};

```



## 2.9 有序/无序集合的自定义策略 (Ordered / Hash Set with Custom Policy)

```
// ---- 自定义哈希 (用于 unordered 容器) ----
// 防止在 Codeforces 等平台上被反哈希攻击导致 TLE
struct custom_hash {
 static uint64_t splitmix64(uint64_t x) {
 x += 0x9e3779b97f4a7c15;
 x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
 x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
 return x ^ (x >> 31);
 }
 size_t operator()(uint64_t x) const {
 static const uint64_t FIXED_RANDOM =
 chrono::steady_clock::now().time_since_epoch().count();
 return splitmix64(x + FIXED_RANDOM);
 }
};
// 用法: unordered_map<int, int, custom_hash>
// 用法: unordered_set<ll, custom_hash>

// ---- 有序集合 (通过 PBDS, GNU 策略基础数据结构) ----
// 需要: #include <ext/pb_ds/assoc_container.hpp>
// #include <ext/pb_ds/tree_policy.hpp>
// using namespace __gnu_pbds;
// typedef tree<int, null_type, less<int>, rb_tree_tag,
// tree_order_statistics_node_update> ordered_set;
// ordered_set.find_by_order(k) — 第 k 小元素 (0-indexed)
// ordered_set.order_of_key(x) — 严格小于 x 的元素个数
```

## 3. 图论 (Graph Theory)

### 3.1 Graph Representation (图的存储)

```
// ---- 邻接表 (Adjacency List) — 最常用 ----
struct Edge {
 int to, weight;
};
vector<vector<Edge>> adj; // adj[u] = {v, w} 的列表

// 无权图:
vvi adj_unweighted;

// ---- 边集数组 (Edge List) — 用于 Kruskal, Bellman-Ford ----
struct Edgelist {
 int u, v, w;
 bool operator<(const Edgelist& o) const { return w < o.w; }
};

// ---- 邻接矩阵 (Adjacency Matrix) — 用于 Floyd 稠密图 ----
vvl mat; // mat[i][j] = 边权; 无边则为 INF; i==j 时为 0
```

### 3.2 DFS & BFS (深度优先搜索 / 广度优先搜索)

**English:** DFS / BFS | **Chinese:** 深度优先搜索 / 广度优先搜索

图的基本遍历。DFS 用于连通性、环检测、拓扑序（在 DAG 中通过后序遍历得到）。BFS 用于无权图的最短路径。

```
// ---- DFS (递归实现) ----
vvi adj;
vector<bool> vis;
vi order; // 用于拓扑排序：递归结束后压入

void dfs(int u) {
 vis[u] = true;
 for (int v : adj[u]) {
 if (!vis[v]) dfs(v);
 }
 order.pb(u); // 后序遍历，用于拓扑排序
}

// ---- BFS (迭代实现，无权图最短路径) ----
vi bfs(int start, int n) {
 vi dist(n, INF);
 dist[start] = 0;
 queue<int> q;
 q.push(start);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v : adj[u]) {
 if (dist[v] == INF) {
 dist[v] = dist[u] + 1;
 q.push(v);
 }
 }
 }
 return dist;
}

// ---- 0-1 BFS (边权为 0 或 1 的最短路径) ----
// 使用双端队列：权值为 0 的边从队首插入，权值为 1 的边从队尾插入
// 在 0-1 权图中求最短路径，复杂度 $O(V+E)$
vi bfs_01(int start, int n, const vector<vector<pii>>& adj) {
 vi dist(n, INF);
 dist[start] = 0;
 deque<int> dq;
 dq.push_front(start);
 while (!dq.empty()) {
 int u = dq.front(); dq.pop_front();
 for (auto [v, w] : adj[u]) {
 if (dist[u] + w < dist[v]) {
 dist[v] = dist[u] + w;
 if (w == 0) dq.push_front(v);
 else dq.push_back(v);
 }
 }
 }
 return dist;
}
}
```

### 3.3 Dijkstra（最短路径——非负权）

**English:** Dijkstra's Algorithm | **Chinese:** 迪杰斯特拉算法 / 单源最短路径

非负边权图的单源最短路径。二叉堆实现复杂度  $O(M \log N)$ 。

```
// 返回 {dist, parent}, dist[v] = INF 表示不可达, parent[v] = -1 表示源点
// 注意：INF 必须足够大（推荐 1e18），防止路径权重累加溢出
pair<vll, vi> dijkstra(int src, int n, const vector<vector<pii>>& adj) {
 vll dist(n, LINF); // LINF = 1e18, 远大于典型权值的最大可能和
 vi par(n, -1);
 dist[src] = 0;

 // 小根堆：{distance, node}
 priority_queue<pll, vector<pll>, greater<pll>> pq;
 pq.push({0, src});

 while (!pq.empty()) {
 auto [d, u] = pq.top(); pq.pop();
 if (d != dist[u]) continue; // 过期记录，跳过
 for (auto [v, w] : adj[u]) {
 if (dist[u] + w < dist[v]) {
 dist[v] = dist[u] + w;
 par[v] = u;
 pq.push({dist[v], v});
 }
 }
 }
 return {dist, par};
}

// ---- 路径重建：从源点到目标点 ----
vi get_path(int src, int dst, const vi& par) {
 vi path;
 for (int v = dst; v != -1; v = par[v]) path.pb(v);
 reverse(all(path));
 if (path[0] != src) return {}; // 不可达
 return path;
}

// ---- 稠密图 Dijkstra ($O(V^2)$ ，当 $E \sim V^2$ 时优于堆优化版本) ----
vll dijkstra_dense(int src, int n, const vll& mat) {
 vll dist(n, LINF);
 vector<bool> vis(n, false);
 dist[src] = 0;
 rep(iter, 0, n) {
 int u = -1;
 rep(i, 0, n) if (!vis[i] && (u == -1 || dist[i] < dist[u])) u = i;
 if (u == -1 || dist[u] == LINF) break;
 vis[u] = true;
 rep(v, 0, n) {
 if (mat[u][v] != LINF && dist[u] + mat[u][v] < dist[v])
 dist[v] = dist[u] + mat[u][v];
 }
 }
 return dist;
}
}
```

### 3.4 Bellman-Ford & SPFA (贝尔曼-福特算法 / SPFA)

**English:** Bellman-Ford Algorithm | **Chinese:** 贝尔曼-福特算法

支持负权边的单源最短路径，能检测从源点可达的负环。复杂度  $O(VE)$ 。

```
// 返回 {dist, has_negative_cycle}, has_negative_cycle 表示存在从源点可达的负环
// 进行 V-1 轮松弛，然后检查是否还能继续松弛 (有负环)
pair<vll, bool> bellman_ford(int src, int n, const vector<EdgeList>& edges) {
 vll dist(n, LINF);
 dist[src] = 0;

 rep(i, 0, n - 1) {
 bool relaxed = false;
 for (auto& e : edges) {
 if (dist[e.u] != LINF && dist[e.u] + e.w < dist[e.v]) {
 dist[e.v] = dist[e.u] + e.w;
 relaxed = true;
 }
 }
 if (!relaxed) break; // 提前退出优化：本轮无松弛则后续也不会变化
 }

 // 第 V 轮检测：若还能松弛说明存在负环
 bool neg_cycle = false;
 for (auto& e : edges) {
 if (dist[e.u] != LINF && dist[e.u] + e.w < dist[e.v]) {
 neg_cycle = true;
 break;
 }
 }
 return {dist, neg_cycle};
}

// ----- SPFA (Shortest Path Faster Algorithm, 队列优化的 Bellman-Ford) -----
// 最坏 $O(VE)$ ，实践中通常远快于标准 Bellman-Ford
// 可以检测负环：记录每个节点入队次数， $\geq N$ 则存在负环
vll spfa(int src, int n, const vector<vector<pii>>& adj) {
 vll dist(n, LINF);
 vi inq(n, 0), cnt(n, 0); // cnt：记录入队次数，用于负环检测
 dist[src] = 0;
 queue<int> q;
 q.push(src);
 inq[src] = true;

 while (!q.empty()) {
 int u = q.front(); q.pop();
 inq[u] = false;
 for (auto [v, w] : adj[u]) {
 if (dist[u] + w < dist[v]) {
 dist[v] = dist[u] + w;
 if (!inq[v]) {
 q.push(v);
 inq[v] = true;
 cnt[v]++;
 if (cnt[v] > n) {
 // 从源点可达的负环已检测到
 return {}; // 或返回特殊错误标记
 }
 }
 }
 }
 }
 return dist;
}
```

#### 3.4.1 差分约束 (Difference Constraints)

**English:** Difference Constraints (SPFA 判负环) | **Chinese:** 差分约束系统

将不等式组  $x_a - x_b \leq y_x a - x_b \leq y$  转化为图论最短路：建边  $b \rightarrow a$  权值  $y$ 。因为最短路满足三角不等式  $\text{dist}[a] \leq \text{dist}[b] + y$ 。加入超级源点 0 连所有点 (权值 0)，以 0 为源跑 SPFA，若存在负环则无解，否则  $\text{dist}[1..n]$  为一组合法解。

常见转化： $x_a - x_b \geq y \Rightarrow x_b - x_a \leq -y \Rightarrow x_b - x_a \leq -y$  (变号转  $\leq$ )； $x_a = x_b \Rightarrow x_a - x_b \leq 0 \wedge x_b - x_a \leq 0 \Rightarrow x_a - x_b \leq 0 \wedge x_b - x_a \leq 0$ 。

```

// ----- 差分约束 (SPFA 判负环, P5960 模板) -----
// 给出 m 条形如 $x_c - x_{c'} \leq y$ 的不等式, 求任意一组解, 无解输出 "NO"
// 复杂度 $O(NM)$ 最坏, 加入超级源点 0
// 返回 {hasSolution, dist}; dist[i] = x_i 的值 (i 从 1 开始)
pair<bool, vi> diff_constraints(int n, const vector<tuple<int,int,int>>& constraints) {
 // 建图: 对于每条 $x_c - x_{c'} \leq y$, 加边 $c' \rightarrow c$ 权 y
 vector<vector<pii>> adj(n + 1);
 for (auto& [c, c_prime, y] : constraints) {
 adj[c_prime].pb({c, y}); // $x_c \leq x_{c'} + y \rightarrow dist[c] \leq dist[c'] + y$
 }
 // 超级源点 0 连所有点, 权值 0 (避免图不连通)
 rep(i, 1, n + 1) adj[0].pb({i, 0});

 vi dist(n + 1, 0); // dist[0] = 0, 其他初始 0 (任意值均可, 不影响解的存在性)
 vi cnt(n + 1, 0); // cnt[i] = i 的入队次数, $> n$ 表示有负环
 vector<bool> inq(n + 1, false);
 deque<int> q;
 q.push_back(0);
 inq[0] = true;

 while (!q.empty()) {
 int u = q.front(); q.pop_front();
 inq[u] = false;
 for (auto& [v, w] : adj[u]) {
 if (dist[v] > dist[u] + w) {
 dist[v] = dist[u] + w;
 if (!inq[v]) {
 q.push_back(v);
 inq[v] = true;
 if (++cnt[v] > n) return {false, {}}; // 负环 $\rightarrow$ 无解
 }
 }
 }
 }
 return {true, dist}; // 有解, dist[1..n] 为答案
}

```

### 3.5 Floyd-Warshall (弗洛伊德算法——全源最短路径)

**English:** Floyd-Warshall Algorithm | **Chinese:** 弗洛伊德算法 / 全源最短路径

求所有点对之间的最短路径。复杂度  $O(V^3)$ 。支持负权边 (但不能有负环)。**关键:  $k$  循环必须在最外层, 保证路径的中间节点编号单调递增。**

```

// dist[i][j] = i 到 j 的最短距离; 对角线为 0 ; 无边则为 LINF
// 传入直接边权矩阵, 原地修改
// 重要: k 循环在最外层 (三重循环顺序为 $k \rightarrow i \rightarrow j$)
void floyd_warshall(vvll& dist, int n) {
 rep(k, 0, n) {
 rep(i, 0, n) {
 rep(j, 0, n) {
 if (dist[i][k] == LINF) continue;
 if (dist[k][j] == LINF) continue;
 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
 }
 }
 }
 // 可选: 检测负环——若 dist[i][i] < 0 则存在负环
}

// ----- Floyd 含路径重建 -----
void floyd_with_path(vvll& dist, vvi& nxt, int n) {
 rep(i, 0, n) rep(j, 0, n) nxt[i][j] = j;
 rep(k, 0, n) rep(i, 0, n) rep(j, 0, n) {
 if (dist[i][k] + dist[k][j] < dist[i][j]) {
 dist[i][j] = dist[i][k] + dist[k][j];
 nxt[i][j] = nxt[i][k];
 }
 }
}

```

### 3.6 并查集 (Disjoint Set Union / Union-Find)

**English:** DSU / Union-Find | **Chinese:** 并查集

本节提供三种实现, 按 compactness 从低到高排列。MST 等算法推荐使用版本 B (KACTL) 或版本 A。

```

// ----- A. 标准 DSU (路径压缩 + 按大小合并) -----
// 含组件计数, 直观易懂
struct DSU {
 vi par, sz;
 int comps; // 连通分量数

 DSU(int n) : par(n), sz(n, 1), comps(n) {
 iota(all(par), 0);
 }

 int find(int x) {
 return par[x] == x ? x : par[x] = find(par[x]);
 }

 // 合并成功返回 true, 已在同一集合则返回 false
 bool unite(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return false;
 if (sz[a] < sz[b]) swap(a, b);
 par[b] = a;
 sz[a] += sz[b];
 comps--;
 return true;
 }

 bool same(int a, int b) { return find(a) == find(b); }
 int size(int x) { return sz[find(x)]; }
};

// ----- B. KACTL 紧凑版 DSU (负数大小, 极简代码) -----
// 竞赛推荐: 代码最短, 内存最优 (仅一个 vector<int>)
// e[i] < 0 表示 i 是根, -e[i] 为集合大小
struct UF {
 vi e; // e[i] < 0: i 是根, -e[i] = 集合大小; e[i] >= 0: 指向父节点
 UF(int n) : e(n, -1) {}

 bool sameSet(int a, int b) { return find(a) == find(b); }
 int size(int x) { return -e[find(x)]; }

 int find(int x) {
 return e[x] < 0 ? x : e[x] = find(e[x]);
 }

 bool join(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return false;
 if (e[a] > e[b]) swap(a, b); // e 为负数, 更负表示更大的集合
 e[a] += e[b]; // 合并大小 (负数相加)
 e[b] = a;
 return true;
 }
};

// ----- C. jiangly 风格 DSU (迭代 find, 避免递归爆栈) -----
// find 使用 while 循环 + 路径压缩, 不会因递归过深而爆栈
struct DSU_Iterative {
 vector<int> f, siz;
 DSU_Iterative(int n) {
 f.resize(n);
 iota(f.begin(), f.end(), 0);
 siz.assign(n, 1);
 }
 int find(int x) {
 while (x != f[x]) x = f[x] = f[f[x]];
 return x;
 }
 bool merge(int x, int y) {
 x = find(x), y = find(y);
 if (x == y) return false;
 siz[x] += siz[y];
 f[y] = x;
 return true;
 }
 int size(int x) { return siz[find(x)]; }
};

// ----- DSU 可回滚版 (可撤销并查集) -----
// 支持撤销最近一次合并, 不能使用路径压缩 (改用按秩合并)。
struct DSU_Rollback {
 vi par, rnk;
 vector<pii> history; // (节点, 旧父节点); 秩变更用负数编码
 int comps;

 DSU_Rollback(int n) : par(n), rnk(n, 0), comps(n) {
 iota(all(par), 0);
 }

 int find(int x) {
 while (par[x] != x) x = par[x];
 return x;
 }

 bool unite(int a, int b) {
 a = find(a), b = find(b);
 if (a == b) return false;
 if (rnk[a] < rnk[b]) swap(a, b);
 history.eb(b, par[b]);
 par[b] = a;
 comps--;
 if (rnk[a] == rnk[b]) {
 history.eb(~a, rnk[a]); // 用负数标记秩变更
 rnk[a]++;
 }
 return true;
 }
};

```

### 3.7 Minimum Spanning Tree (最小生成树)

**English:** Minimum Spanning Tree (Kruskal / Prim) | **Chinese:** 最小生成树 (克鲁斯卡尔算法 / 普里姆算法)

求连接所有节点的最小边权树。

```
// ----- Kruskal ($O(M \log M)$) --- 适合稀疏图 -----
ll kruskal(int n, vector<EdgeList>& edges) {
 sort(all(edges));
 DSU dsu(n);
 ll total = 0;
 int taken = 0;
 for (auto& e : edges) {
 if (dsu.unite(e.u, e.v)) {
 total += e.w;
 taken++;
 if (taken == n - 1) break;
 }
 }
 return taken == n - 1 ? total : LINF; // 不连通则返回 LINF
}

// ----- Prim ($O(M \log N)$) --- 适合稠密图 -----
// 从任意节点出发, 重复添加连接"已访问"与"未访问"节点的最小权边
ll prim(int n, const vector<vector<pii>& adj) {
 vll min_w(n, LINF);
 vector<bool> vis(n, false);
 min_w[0] = 0;
 ll total = 0;

 // {weight, node}; 使用 set 支持 decrease-key 操作
 set<pll> pq;
 pq.insert({0, 0});

 while (!pq.empty()) {
 auto [w, u] = *pq.begin(); pq.erase(pq.begin());
 if (vis[u]) continue;
 vis[u] = true;
 total += w;

 for (auto [v, edge_w] : adj[u]) {
 if (!vis[v] && edge_w < min_w[v]) {
 pq.erase({min_w[v], v});
 min_w[v] = edge_w;
 pq.insert({min_w[v], v});
 }
 }
 }
 return total;
}

// ----- 最小瓶颈生成树 (MBST) -----
// MST 本身一定是 MBST (最小化路径上最大边权)。直接求 MST, 取最大边权即可。

// ----- 次小生成树 (Second Best MST) -----
// 先求 MST, 然后对每条非树边, 尝试替换树上路径中的最大边。
// 需要在 MST 上做 LCA 与路径最大边权查询。
```

### 3.8 Topological Sort & Kahn's Algorithm (拓扑排序)

**English:** Topological Sort | **Chinese:** 拓扑排序

DAG 顶点的线性排序。复杂度  $O(V+E)$ 。

```
// ----- 基于 DFS 的拓扑排序 (后序遍历逆序) -----
// 对每个未访问节点调用: 结果 order 是后序遍历的逆序
// 仅对 DAG 有效: 需要检测环时加上状态标记
void dfs_topo(int u, const vvi& adj, vector<bool>& vis, vi& order) {
 vis[u] = true;
 for (int v : adj[u])
 if (!vis[v]) dfs_topo(v, adj, vis, order);
 order.pb(u);
}
// 所有 DFS 完成后执行 reverse(all(order)) 即得拓扑序

// ----- Kahn 算法 (基于 BFS + 入度) -----
// 推荐使用: 能自然检测环 (结果大小 < N 则存在环)
vi kahn(int n, const vvi& adj) {
 vi indeg(n);
 rep(u, 0, n) for (int v : adj[u]) indeg[v]++;

 queue<int> q;
 rep(i, 0, n) if (indeg[i] == 0) q.push(i);

 vi order;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 order.pb(u);
 for (int v : adj[u]) {
 if (--indeg[v] == 0) q.push(v);
 }
 }
 return order; // sz(order) < n 说明存在环
}

// ----- DAG 上的动态规划 (DP on DAG) -----
// 拓扑序保证处理某节点时其所有前驱都已处理完毕
void dp_on_dag(int n, const vvi& adj) {
 vi order = kahn(n, adj);
 vi dp(n); // 初始化 dp 值
 for (int u : order) {
 for (int v : adj[u]) {
 // dp[v] = max(dp[v], dp[u] + ...); 具体取决于问题
 }
 }
}
```

### 3.9 Strongly Connected Components (强连通分量)

**English:** SCC (Tarjan / Kosaraju) | **Chinese:** 强连通分量

将有向图分解为 SCC, 复杂度  $O(V+E)$ 。缩点后的图 (condensation DAG) 是无环的。提供两种版本: A 为标准实现, B 为 jiangly 紧凑版本。

```

// ---- A. Tarjan SCC (单次DFS, 标准实现) ----
struct TarjanSCC {
 int n, timer, scc_cnt;
 vi dfn, low, scc_id;
 vector<bool> in_stk;
 stack<int> stk; // 也可用 vector 代替 stack 以便调试
 vvi adj;

 TarjanSCC(int n_, const vvi& adj_) : n(n_), adj(adj_) {
 dfn.assign(n, -1);
 low.resize(n);
 scc_id.resize(n);
 in_stk.assign(n, false);
 timer = scc_cnt = 0;

 rep(i, 0, n) if (dfn[i] == -1) dfs(i);
 }

 void dfs(int u) {
 dfn[u] = low[u] = ++timer;
 stk.push(u);
 in_stk[u] = true;

 for (int v : adj[u]) {
 if (dfn[v] == -1) { // 树边
 dfs(v);
 low[u] = min(low[u], low[v]);
 } else if (in_stk[v]) { // 回边: 必须用 dfn[v], 不能用 low[v]
 low[u] = min(low[u], dfn[v]);
 }
 }

 if (low[u] == dfn[u]) { // SCC 的根
 int v;
 do {
 v = stk.top(); stk.pop();
 in_stk[v] = false;
 scc_id[v] = scc_cnt;
 } while (v != u);
 scc_cnt++;
 }
 }

 // 构建缩点 DAG
 vvi build_condensation() {
 vvi cond(scc_cnt);
 rep(u, 0, n) {
 for (int v : adj[u]) {
 if (scc_id[u] != scc_id[v])
 cond[scc_id[u]].pb(scc_id[v]);
 }
 }
 // 去重: 排序后 unique
 rep(i, 0, scc_cnt) {
 sort(all(cond[i]));
 cond[i].erase(unique(all(cond[i])), cond[i].end());
 }
 return cond;
 }
};

// ---- B. jiangly 紧凑版 Tarjan SCC ----
// 适合竞赛: 代码极短, 使用全局数组而非成员变量
struct SCC {
 int n;
 vector<vector<int>> adj;
 vector<int> stk, dfn, low, bel;
 int cur, cnt;

 SCC(int n) {
 this->n = n;
 adj.assign(n, {});
 dfn.assign(n, -1);
 low.resize(n);
 bel.assign(n, -1);
 cur = cnt = 0;
 }

 void addEdge(int u, int v) { adj[u].push_back(v); }

 void dfs(int x) {
 dfn[x] = low[x] = cur++;
 stk.push_back(x);
 for (auto y : adj[x]) {
 if (dfn[y] == -1) { // 树边
 dfs(y);
 low[x] = min(low[x], low[y]);
 } else if (bel[y] == -1) { // 回边: 使用 dfn[y] (正确做法)
 low[x] = min(low[x], dfn[y]);
 }
 }
 if (dfn[x] == low[x]) { // SCC 根节点
 int y;
 do {
 y = stk.back();
 bel[y] = cnt;
 stk.pop_back();
 } while (y != x);
 cnt++;
 }
 }

 vector<int> work() {
 for (int i = 0; i < n; i++)
 if (dfn[i] == -1)
 dfs(i);
 return bel;
 }
};

```

**关键细节：**回边更新 `low[u]` 时必须用 `dfn[v]`（不是 `low[v]`）。原因：`low[v]` 可能来自 `v` 所在 SCC 的某个更早祖先，若错误地用 `low[v]` 更新，会导致跨 SCC 边也被纳入同一个 SCC，使 SCC 划分错误。

### 3.10 Bridges & Articulation Points（桥与割点）

**English:** Bridges & Articulation Points | **Chinese:** 桥 / 割点 / 无向图双连通分量

在无向图中找出所有桥和割点。使用 DFS low-link 值，复杂度  $O(V+E)$ 。

```
// 找出无向图中的所有桥
// 桥的定义：删除该边后连通分量数增加
struct FindBridges {
 int n, timer;
 vi dfn, low;
 vector<pii> bridges; // 存储所有桥的端点
 vvi adj;

 FindBridges(int n_, const vvi& adj_) : n(n_), adj(adj_) {
 dfn.assign(n, -1);
 low.resize(n);
 timer = 0;
 rep(i, 0, n) if (dfn[i] == -1) dfs(i, -1);
 }

 void dfs(int u, int p) {
 dfn[u] = low[u] = ++timer;
 for (int v : adj[u]) {
 if (v == p) continue; // 跳过父边 (单边)
 if (dfn[v] == -1) {
 dfs(v, u);
 low[u] = min(low[u], low[v]);
 if (low[v] > dfn[u]) // 桥的条件：子节点无法从其他路径回到 u 及以上
 bridges.eb(u, v);
 } else {
 low[u] = min(low[u], dfn[v]);
 }
 }
 }
};

// ----- 割点 (Articulation Points) -----
// 割点的定义：删除该节点后连通分量数增加
struct FindArtPoints {
 int n, timer;
 vi dfn, low;
 vector<bool> is_art;
 vvi adj;

 FindArtPoints(int n_, const vvi& adj_) : n(n_), adj(adj_) {
 dfn.assign(n, -1); low.resize(n);
 is_art.assign(n, false);
 timer = 0;
 rep(i, 0, n) if (dfn[i] == -1) dfs(i, -1);
 }

 void dfs(int u, int p) {
 dfn[u] = low[u] = ++timer;
 int children = 0;
 for (int v : adj[u]) {
 if (v == p) continue;
 if (dfn[v] == -1) {
 dfs(v, u);
 low[u] = min(low[u], low[v]);
 if (p != -1 && low[v] >= dfn[u]) // 非根节点的割点条件
 is_art[u] = true;
 children++;
 } else {
 low[u] = min(low[u], dfn[v]);
 }
 }
 if (p == -1 && children > 1) is_art[u] = true; // 根节点：需至少两个子节点
 }
};
```

### 3.11 Lowest Common Ancestor（最近公共祖先）

**English:** LCA (Binary Lifting) | **Chinese:** 最近公共祖先（倍增法）

预处理  $O(N \log N)$ ，每次 LCA 查询  $O(\log N)$ 。同时支持  $k$  级祖先查询和路径最值查询。



```

struct LCA {
 int n, LOG;
 vvi up; // up[k][v] = v 的第 2^k 级祖先
 vi depth;
 const vvi& adj; // 引用外部邻接表, 避免拷贝

 LCA(int n_, int root, const vvi& adj_) : n(n_), adj(adj_) {
 LOG = 32 - __builtin_clz(n); // floor(log2(n)) + 1
 up.assign(LOG, vi(n, -1));
 depth.resize(n);
 dfs(root, -1);
 build();
 }

 // 递归 DFS——当 N 较大 (≥ 5e5) 且树退化为链时可能栈溢出。
 // 严格场合请使用下方的非递归 DFS 版本 (见 struct 后的注释)。
 void dfs(int u, int p) {
 up[0][u] = p;
 for (int v : adj[u]) {
 if (v == p) continue;
 depth[v] = depth[u] + 1;
 dfs(v, u);
 }
 }

 void build() {
 rep(k, 1, LOG) {
 rep(v, 0, n) {
 if (up[k - 1][v] != -1)
 up[k][v] = up[k - 1][up[k - 1][v]];
 }
 }
 }

 int lca(int a, int b) {
 if (depth[a] < depth[b]) swap(a, b);
 // 将 a 提升到与 b 同一深度
 int diff = depth[a] - depth[b];
 rep(k, 0, LOG) if (diff & (1 << k)) a = up[k][a];
 if (a == b) return a;
 // 同时向上跳 (从最大的步长开始)
 per(k, LOG, 0) {
 if (up[k][a] != up[k][b]) {
 a = up[k][a];
 b = up[k][b];
 }
 }
 return up[0][a];
 }

 // 树上两点距离 (边权为 1)
 int dist(int a, int b) {
 return depth[a] + depth[b] - 2 * depth[lca(a, b)];
 }

 // 求 v 的第 k 级祖先; 不存在则返回 -1
 int kth_ancestor(int v, int k) {
 rep(bit, 0, LOG) if (k & (1 << bit)) {
 v = up[bit][v];
 if (v == -1) break;
 }
 return v;
 }
};

// ---- 非递归 DFS (链式退化的树需要, 避免栈溢出) ----
// 当 N ≥ 5e5 且树可能退化为链 (如 P3379) 时, 递归 DFS 的 5e5 层调用会爆栈。
// 替换方法: 将 LCA 构造函数中的 dfs(root, -1) 改为 dfs_iter(root), 并加入下方代码:
//
// void dfs_iter(int root) {
// vi stk = {root};
// vi par(n, -1);
// par[root] = -1;
// while (!stk.empty()) {
// int u = stk.back(); stk.pop_back();
// up[0][u] = par[u];
// for (int v : adj[u]) {
// if (v == par[u]) continue;
// depth[v] = depth[u] + 1;
// par[v] = u;
// stk.push_back(v);
// }
// }
// }
//
// 注意: 非递归 DFS 得到的 depth 和 up[0] 与递归版本完全等价,
// 但遍历子节点的顺序相反 (不影响 LCA 正确性)。

// ---- Euler Tour + RMQ LCA (O(N) 预处理, O(1) 查询) ----
// 更复杂, 但查询更快。对欧拉序列建 Sparse Table。
// 欧拉序列: 进入节点时记录, 从每个子节点返回时也记录 (最后一个除外)。
// LCA(a,b) = 欧拉序列中 first[a] 到 first[b] 之间深度最小的节点

```

### 3.12 Heavy-Light Decomposition (树链剖分 / 轻重链剖分)

**English:** Heavy-Light Decomposition (HLD) | **Chinese:** 树链剖分 / 轻重链剖分

将树分解为重路径, 支持路径查询与更新, 复杂度  $O(\log^2 N)$ 。通常配合线段树使用。 本节提供两个版本: A 为标准版 (含路径查询模板), B 为 jiangly 综合版 (含 jump、换根等高级功能)。

```

// ----- A. 标准 HLD (基础版) -----
struct HLD {
 int n, timer;
 vi sz, depth, par, heavy, head, pos; // pos = DFS 位置 (0-indexed)
 vvi adj;

 HLD(int n_, const vvi& adj_) : n(n_), adj(adj_) {
 sz.resize(n); depth.resize(n); par.resize(n);
 heavy.assign(n, -1); head.resize(n); pos.resize(n);
 timer = 0;
 dfs_sz(0, -1);
 dfs_hld(0, -1, 0);
 }

 void dfs_sz(int u, int p) {
 sz[u] = 1;
 par[u] = p;
 int max_sz = 0;
 for (int v : adj[u]) {
 if (v == p) continue;
 depth[v] = depth[u] + 1;
 dfs_sz(v, u);
 sz[u] += sz[v];
 if (sz[v] > max_sz) {
 max_sz = sz[v];
 heavy[u] = v;
 }
 }
 }

 void dfs_hld(int u, int p, int h) {
 head[u] = h;
 pos[u] = timer++;
 if (heavy[u] != -1)
 dfs_hld(heavy[u], u, h); // 继续当前重链
 for (int v : adj[u]) {
 if (v == p || v == heavy[u]) continue;
 dfs_hld(v, u, v); // 开始新的轻链
 }
 }

 // 通用路径查询：从 u 到 v，在每条重链片段上查询
 // 调用 seg.query(pos[head[a]], pos[a]) 处理每段
 template <typename Func>
 void path_query(int a, int b, Func&& query_seg) {
 while (head[a] != head[b]) {
 if (depth[head[a]] < depth[head[b]]) swap(a, b);
 query_seg(pos[head[a]], pos[a]); // 线段树查询区间 [l, r]
 a = par[head[a]];
 }
 if (depth[a] > depth[b]) swap(a, b);
 query_seg(pos[a], pos[b]); // 最后一段 (LCA → 较深节点)
 }
};

// 使用示例 (配合线段树) :
// HLD hld(n, adj);
// SegTree<ll> seg(n, 0);
// hld.path_query(u, v, [&](int l, int r) {
// ans += seg.query(l, r);
// });

// ----- 迭代线段树 (KACTL 风格, 2N 存储, 半开区间) -----
// 配合 HLD 使用：比递归线段树更快，无递归开销
// 查询使用半开区间 [l, r)，与 HLD 的 path_query 兼容
struct SegTreeIter {
 typedef int T;
 static constexpr T unit = INT_MIN; // 单位元 (求和用 0, min 用 INF)
 T f(T a, T b) { return max(a, b); } // 结合运算

 vector<T> s;
 int n;

 SegTreeIter(int n = 0, T def = unit) : s(2 * n, def), n(n) {}

 void update(int pos, T val) {
 for (s[pos += n] = val; pos /= 2;)
 s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
 }

 // 查询半开区间 [b, e)
 T query(int b, int e) {
 T ra = unit, rb = unit;
 for (b += n, e += n; b < e; b /= 2, e /= 2) {
 if (b % 2) ra = f(ra, s[b++]);
 if (e % 2) rb = f(s[--e], rb);
 }
 return f(ra, rb);
 }
};

// ----- B. jiangly 综合版 HLD (含高级功能) -----
// 特性：in/out 欧拉序、isAncestor、jump (沿链向上跳 k 步)、
// rootedParent (换根后求父节点)、rootedSize (换根后求子树大小)、
// rootedLca (换根后求 LCA)
struct HLD_Comprehensive {
 int n;
 vector<vector<int>> adj;
 vector<int> sz, top, dep, parent, in, out;
 int cur;

 HLD_Comprehensive() {}
 HLD_Comprehensive(int n_) : n(n_) {
 adj.assign(n, {});
 sz.resize(n); top.resize(n); dep.resize(n);
 parent.resize(n); in.resize(n); out.resize(n);
 }

 void addEdge(int u, int v) {
 adj[u].push_back(v);
 adj[v].push_back(u);
 }
};

```

```

void init(int root = 0) {
 top[root] = root;
 dep[root] = 0;
 parent[root] = -1;
 cur = 0;
 dfs1(root);
 dfs2(root);
}

void dfs1(int u) {
 sz[u] = 1;
 if (parent[u] != -1)
 adj[u].erase(find(adj[u].begin(), adj[u].end(), parent[u]));
 for (auto& v : adj[u]) {
 parent[v] = u;
 dep[v] = dep[u] + 1;
 dfs1(v);
 sz[u] += sz[v];
 if (sz[v] > sz[adj[u][0]])
 swap(v, adj[u][0]); // 重儿子放在 adj[u][0]
 }
}

void dfs2(int u) {
 in[u] = cur++;
 for (int v : adj[u]) {
 top[v] = (v == adj[u][0] ? top[u] : v); // 重儿子继承 top, 轻儿子新开链
 dfs2(v);
 }
 out[u] = cur;
}

int lca(int u, int v) {
 while (top[u] != top[v]) {
 if (dep[top[u]] < dep[top[v]]) swap(u, v);
 u = parent[top[u]];
 }
 return dep[u] < dep[v] ? u : v;
}

int dist(int u, int v) {
 return dep[u] + dep[v] - 2 * dep[lca(u, v)];
}

// 判断 u 是否为 v 的祖先 (基于欧拉序 in/out)
bool isAncestor(int u, int v) {
 return in[u] <= in[v] && in[v] < out[u];
}

// 从 u 沿树向上跳 k 步 (0-indexed, k=0 返回 u)
// 注意: k 不能超过深度, 否则行为未定义
int jump(int u, int k) {
 if (k == 0) return u;
 while (true) {
 int h = top[u];
 int d = dep[u] - dep[h]; // u 到所在重链顶端的距离
 if (k <= d) {
 // 目标在当前重链上: 重链的 DFS 序连续, 直接用 in 数组定位
 // 需要维护一个反向映射数组 rev (in[i] = 第几个节点)
 // 简单方法: 由于 adj[u][0] 是重儿子, 沿 parent 回溯 d-k 步
 // 但在 jiangly 代码中通常用 rev[in[u] - k]
 // 此处展示概念: 实际使用时需额外维护 rev
 while (k--) u = parent[u]; // 简化写法, 实际可用 rev[in[u]-k]
 return u;
 }
 k -= d + 1;
 u = parent[h];
 }
}

// 换根后求 u 的父节点 (root 为新的根)
int rootedParent(int root, int u) {
 if (root == u) return -1;
 if (!isAncestor(u, root)) return parent[u];
 return jump(root, dep[root] - dep[u] - 1);
}

// 换根后求以 root 为根时 u 的子树大小
int rootedSize(int root, int u) {
 if (root == u) return n;
 if (!isAncestor(u, root)) return sz[u];
 return n - sz[rootedParent(root, u)];
}

// 换根后三点 LCA: 三个点以 root 为根时的 LCA
// 公式: rootedLca(a,b,c) = lca(a,b) ⊕ lca(b,c) ⊕ lca(c,a) (异或)
int rootedLca(int root, int a, int b, int c) {
 return lca(a, b) ^ lca(b, c) ^ lca(c, a);
}

// 换根后两点 LCA (root 为新的根)
int rootedLca(int root, int a, int b) {
 return rootedLca(root, a, b, root);
}
};

```

HLD 使用说明:

### 3.13 Euler Tour & Subtree Queries (欧拉序 / DFS 序)

**English:** Euler Tour (Flattening Tree to Array) | **Chinese:** 欧拉序 / DFS 序

把树映射到数组上, 使得每个子树对应一个连续区间。O(N) 预处理。

```

struct EulerTour {
 int n, timer;
 vi tin, tout, euler; // euler[tin[u]] = u
 vvi adj;

 EulerTour(int n_, const vvi& adj_) : n(n_), adj(adj_) {
 tin.resize(n); tout.resize(n);
 timer = 0;
 dfs(0, -1);
 }

 void dfs(int u, int p) {
 tin[u] = timer++;
 euler.pb(u);
 for (int v : adj[u]) {
 if (v != p) dfs(v, u);
 }
 tout[u] = timer - 1; // 闭区间: 子树 = [tin[u], tout[u]]
 }

 // 判断 u 是否为 v 的祖先
 bool is_ancestor(int u, int v) {
 return tin[u] <= tin[v] && tout[v] <= tout[u];
 }
};
// 子树更新/查询: 在区间 [tin[u], tout[u]] 上用树状数组或线段树操作

```

### 3.14 Bipartite Checking & Coloring (二分图检测与染色)

**English:** Bipartite Graph | **Chinese:** 二分图检测 / 二染色

判断图是否为二分图 (可二染色)。复杂度  $O(V+E)$ 。

```

// 检查图是否为二分图 (可二染色), 返回染色 {0, 1}, 不可染色则返回空
// 使用 BFS 染色, 同时支持非连通图
vi bipartite_color(int n, const vvi& adj) {
 vi col(n, -1);
 rep(i, 0, n) {
 if (col[i] != -1) continue;
 col[i] = 0;
 queue<int> q; q.push(i);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v : adj[u]) {
 if (col[v] == -1) {
 col[v] = col[u] ^ 1; // 交替染色
 q.push(v);
 } else if (col[v] == col[u]) {
 return {}; // 同色相邻 → 不是二分图
 }
 }
 }
 }
 return col;
}

```

### 3.15 Cycle Detection (环检测)

**English:** Cycle Detection | **Chinese:** 环检测

```

// ----- 无向图环检测 -----
// DFS 过程中, 若遇到已访问且非父节点的邻居, 则存在环
// 复杂度 $O(V+E)$
bool has_cycle_undirected(int u, int p, const vvi& adj, vector<bool>& vis) {
 vis[u] = true;
 for (int v : adj[u]) {
 if (v == p) continue;
 if (vis[v]) return true;
 if (has_cycle_undirected(v, u, adj, vis)) return true;
 }
 return false;
}

// ----- 有向图环检测 (三色 DFS) -----
// 0 = 未访问, 1 = 当前栈中 (正在处理), 2 = 已处理完毕
bool has_cycle_directed(int u, const vvi& adj, vi& state) {
 state[u] = 1;
 for (int v : adj[u]) {
 if (state[v] == 1) return true; // 回边 → 存在环
 if (state[v] == 0 && has_cycle_directed(v, adj, state)) return true;
 }
 state[u] = 2;
 return false;
}

```

### 3.16 Tree Diameter (树的直径)

**English:** Tree Diameter | **Chinese:** 树的直径

```
// 方法一：两次 BFS/DFS (仅适用于非负边权树)
// 任选一点，找到最远点 A；从 A 出发找最远点 B，A-B 即为直径
// 复杂度 $O(V+E)$
int tree_diameter(const vvi& adj) {
 auto bfs = [&](int src) {
 vi dist(sz(adj), INF);
 dist[src] = 0;
 queue<int> q; q.push(src);
 int far = src;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 if (dist[u] > dist[far]) far = u;
 for (int v : adj[u]) {
 if (dist[v] == INF) {
 dist[v] = dist[u] + 1;
 q.push(v);
 }
 }
 }
 return make_pair(far, dist[far]);
 };
 auto [a, _] = bfs(0);
 auto [b, diameter] = bfs(a);
 return diameter; // 也可以记录 a, b 以获取路径本身
}

// 方法二：树形 DP (支持负权边, 通用方法)
// height[u] = u 到其子树中叶节点的最长距离
// 经过 u 的直径候选 = 最优两个子树的 height 之和 + 边权
// 复杂度 $O(N)$
```

### 3.17 Topological K-th Path / K 短路

**English:** K Shortest Paths | **Chinese:** K 短路

对于非负权图，Dijkstra 变体为每个节点维护 K 条最优距离。

```
// K 短简单路：极难 (Yen 算法, $O(K * V * (E + V \log V))$)
// K 短游走 (可重复经过节点/边)：下面为每个节点维护 K 条最优距离
// 若只需求前 K 条长度不同的简单路，建议用 Eppstein 算法 (更高效)

vll k_shortest_walks(int src, int dst, int k, int n, const vector<vector<pii>&& adj) {
 vector<priority_queue<ll>> best(n); // 每节点维护大小为 K 的最大堆
 priority_queue<pll, vector<pll>, greater<pll>> pq;
 pq.push({0, src});

 while (!pq.empty()) {
 auto [d, u] = pq.top(); pq.pop();
 if (sz(best[dst]) >= k && d > best[dst].top()) continue;
 if (sz(best[u]) >= k && d >= best[u].top()) continue;

 best[u].push(d);
 if (sz(best[u]) > k) best[u].pop();

 for (auto [v, w] : adj[u]) {
 if (sz(best[v]) < k || d + w < best[v].top())
 pq.push({d + w, v});
 }
 }

 vll res;
 while (!best[dst].empty()) { res.pb(best[dst].top()); best[dst].pop(); }
 reverse(all(res));
 return res; // 前 K 短距离 (可能不足 K 条)
}
```

### 3.18 Virtual Tree (虚树)

**English:** Virtual Tree | **Chinese:** 虚树

从 K 个关键节点的子集构建辅助树 (保留 LCA 关系)。虚树总节点数  $O(K)$ 。

```
// 从一组节点构建虚树
// 需要：预处理的 LCA, EulerTour 的 tin/tout
// 返回虚树的边 (虚树中的父子关系)
// 输入 nodes 需要提前按 tin 排序
vector<pii> build_virtual_tree(vi nodes, LCA& lca, EulerTour& et) {
 sort(all(nodes), [&](int a, int b) { return et.tin[a] < et.tin[b]; });
 int m = sz(nodes);
 // 将相邻关键节点的 LCA 也加入节点集
 rep(i, 0, m - 1) nodes.pb(lca.lca(nodes[i], nodes[i + 1]));
 sort(all(nodes), [&](int a, int b) { return et.tin[a] < et.tin[b]; });
 nodes.erase(unique(all(nodes)), nodes.end());

 vector<pii> edges; // {parent, child}
 vi stk; // 维护从根到当前节点的路径
 for (int u : nodes) {
 while (!stk.empty() && !et.is_ancestor(stk.back(), u)) stk.pop_back();
 if (!stk.empty()) edges.pb(stk.back(), u);
 stk.pb(u);
 }
 return edges;
}
```

### 3.19 2-SAT (2-SAT 问题)

**English:** 2-SAT | **Chinese:** 2-SAT 问题 / 二元可满足性问题

判断布尔变量能否满足一个析取范式 (CNF)，其中每个子句恰含两个文字。复杂度  $O(V+E)$ ，通过 SCC 求解。

```
// 变量 i (0-indexed) 对应节点 2*i (真) 和 2*i+1 (假)
// 子句 (a or b) : 添加蕴含边 ~a -> b 和 ~b -> a
struct TwoSAT {
 int n;
 vvi adj;
 TwoSAT(int n_) : n(n_), adj(2 * n_) {}

 // 添加子句 (a_val or b_val)
 // a_val, b_val: true 表示正文字, false 表示否定文字
 void add_clause(int a, bool a_val, int b, bool b_val) {
 int u = 2 * a + !a_val; // ~a
 int v = 2 * b + b_val; // b
 adj[u].pb(v); // ~a -> b
 u = 2 * b + !b_val; // ~b
 v = 2 * a + a_val; // a
 adj[u].pb(v); // ~b -> a
 }

 // 添加 XOR 约束 : (a XOR b)
 void add_xor(int a, int b) {
 add_clause(a, true, b, true); // 不能同时为假
 add_clause(a, false, b, false); // 不能同时为真
 }

 // 添加强制赋值 : a = val
 void set_val(int a, bool val) {
 add_clause(a, val, a, val); // (a_val or a_val) = a_val
 }

 // 返回布尔赋值, 不可满足则返回空
 vector<bool> solve() {
 TarjanSCC scc(2 * n, adj);
 vector<bool> ans(n);
 rep(i, 0, n) {
 if (scc.scc_id[2 * i] == scc.scc_id[2 * i + 1])
 return {}; // 不可满足 : i 的真假节点在同一个 SCC 中
 // 选择拓扑序靠后的 (SCC 编号较小的), 即优先取"较晚"的决策
 ans[i] = scc.scc_id[2 * i] < scc.scc_id[2 * i + 1];
 }
 return ans;
 }
};
```

常见易错点总结

问题	易错点	正确做法
INF 取值	用 1e9 可能小于路径权重累加和	用 const ll INF = 1e18;
Floyd 循环顺序	k 不在最外层	k 必须最外层 (k->i->j), 否则错误
SCC 回边更新	用 low[v] 更新 low[u]	必须用 dfn[v], 否则跨 SCC 合并错误
Dijkstra 稠密图	盲目用堆优化版	E ~ V^2 时用 O(V^2) 朴素版更快
SPFA 负环检测	只用 inq 数组	还要用 cnt[v] 记录入队次数, >= N 则有负环
桥的判定	用 low[v] >= dfn[u]	桥的条件是 low[v] > dfn[u] (严格大于)
割点-根节点	忘记特殊处理根节点	根节点需 children > 1 才是割点
Kruskal 边排序	忘记排序	必须按边权升序排序, 否则不保证最小生成树
Prim 的 set	用 priority_queue 代替 set	PQ 不支持 decrease-key, 改用 set 或在 push 时忽略旧值

说明：

4. 数学与数论

4.1 模运算

```
const int MOD = 1e9 + 7; // 通用模数 ; NTT 场景请使用 998244353
// 处理负数取模 : safe_mod = (x % MOD + MOD) % MOD
// 频繁调用时可内联 : int add(int a, int b) { return (a+b) % MOD; }

inline int add_mod(int a, int b) { return (a + b) % MOD; }
inline int sub_mod(int a, int b) { return (a - b + MOD) % MOD; }
inline int mul_mod(int a, int b) { return (ll)a * b % MOD; }

// 快速幂 : 计算 a^b mod m, 时间复杂度 O(log b)
ll mod_pow(ll a, ll b, ll m = MOD) {
 ll res = 1;
 a %= m;
 while (b) {
 if (b & 1) res = res * a % m;
 a = a * a % m;
 b >>= 1;
 }
 return res;
}

// 费马小定理求逆元 : a^(MOD-2) ≡ a^{-1} (mod MOD)
// 要求 MOD 为素数
ll mod_inv(ll a, ll m = MOD) {
 return mod_pow(a, m - 2, m);
}

// 扩展欧几里得算法
// 返回 {g, x, y} 满足 a*x + b*y = g = gcd(a,b)
// 若 a 与 m 互质, 则 x 为 a 在模 m 意义下的逆元
tuple<ll, ll, ll> ext_gcd(ll a, ll b) {
 if (b == 0) return {a, 1, 0};
 auto [g, x, y] = ext_gcd(b, a % b);
 return {g, y, x - (a / b) * y};
}

ll mod_inv_ext(ll a, ll m) {
 auto [g, x, y] = ext_gcd(a, m);
 if (g != 1) return -1; // 逆元不存在
 return (x % m + m) % m;
}
```

## 4.2 组合数学

```
// ----- 阶乘与组合数 -----
struct Combinatorics {
 int n;
 vll fact, inv_fact;

 Combinatorics(int n_) : n(n_) {
 fact.resize(n + 1);
 inv_fact.resize(n + 1);
 fact[0] = 1;
 rep(i, 1, n + 1) fact[i] = fact[i - 1] * i % MOD;
 inv_fact[n] = mod_pow(fact[n], MOD - 2);
 per(i, n, 0) inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD;
 }

 ll C(int n, int k) { // 组合数 n 选 k
 if (k < 0 || k > n) return 0;
 return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
 }

 ll P(int n, int k) { // 排列数 n 排 k
 if (k < 0 || k > n) return 0;
 return fact[n] * inv_fact[n - k] % MOD;
 }

 // 卡特兰数: $C(2n, n) / (n+1) = C(2n, n) - C(2n, n+1)$
 ll catalan(int n) {
 return (C(2 * n, n) - C(2 * n, n + 1) + MOD) % MOD;
 }

 // 可重组合 (隔板法 / 星条旗问题) :
 // 从 n 类中允许重复地选 k 个: $C(n+k-1, k)$
};

// ----- 卢卡斯定理: n, k 很大但模数 p 为小素数时使用 -----
// $C(n, k) \bmod p$, 利用 p 进制展开
ll lucas(ll n, ll k, int p, Combinatorics& comb) {
 if (k == 0) return 1;
 return lucas(n / p, k / p, p, comb) * comb.C(n % p, k % p) % p;
}
```

### 4.3 素数筛法

```
// ----- 线性筛（欧拉筛） -----
// $O(N)$: 求所有素数、最小质因子 spf , 以及积性函数
struct LinearSieve {
 int n;
 vi primes, spf;
 // 可在筛的过程中同步计算 ϕ , μ , τ , σ 等积性函数

 LinearSieve(int n_) : n(n_), spf(n + 1) {
 rep(i, 2, n + 1) {
 if (spf[i] == 0) {
 spf[i] = i;
 primes.pb(i);
 }
 for (int p : primes) {
 if (p > spf[i] || (ll)i * p > n) break;
 spf[i * p] = p;
 }
 }
 }

 bool is_prime(int x) { return x >= 2 && spf[x] == x; }
};

// ----- Miller-Rabin 素性测试 -----
// 概率型算法: 对于 64 位整数使用确定性基底集合
// 时间复杂度: $O(K \log^3 N)$, $K=12$ 覆盖 2^{64} 范围
bool miller_rabin(ll n) {
 if (n < 2) return false;
 // 小素数试除
 for (ll p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
 if (n == p) return true;
 if (n % p == 0) return false;
 }

 ll d = n - 1;
 int s = 0;
 while (d % 2 == 0) { d /= 2; s++; }

 // 64 位确定性基底 (覆盖所有 $n < 2^{64}$)
 for (ll a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
 if (a % n == 0) continue;
 ll x = mod_pow(a, d, n);
 if (x == 1 || x == n - 1) continue;
 bool composite = true;
 rep(r, 0, s - 1) {
 x = ((__int128)x * x % n);
 if (x == n - 1) { composite = false; break; }
 }
 if (composite) return false;
 }
 return true;
}

// ----- Pollard-Rho 质因数分解 -----
// 期望 $O(N^{1/4})$, 内部调用 Miller-Rabin 进行素性检测。
ll pollard_rho(ll n) {
 if (n % 2 == 0) return 2;
 if (n % 3 == 0) return 3;
 while (true) {
 ll c = (ll)rand() % (n - 1) + 1;
 auto f = [&](ll x) { return ((__int128)x * x + c) % n; };
 ll x = 2, y = 2, d = 1;
 while (d == 1) {
 x = f(x); y = f(f(y));
 d = gcd(abs(x - y), n);
 }
 if (d != n) return d;
 }
}

// 获取全部质因子及其指数
vector<pll> factorize(ll n) {
 vector<pll> factors;
 auto factor = [&](auto&& self, ll x) {
 if (x == 1) return;
 if (miller_rabin(x)) {
 factors.pb({x, 1}); // 调用侧负责合并相同质因子
 return;
 }
 ll d = pollard_rho(x);
 self(self, d);
 self(self, x / d);
 };
 factor(factor, n);
 sort(all(factors));
 vector<pll> result;
 for (auto& [p, c] : factors) {
 if (result.empty() || result.back().first != p) result.pb({p, 1});
 else result.back().second++;
 }
 return result;
}
```



## 4.4 数论函数

```
// ---- 欧拉函数 $\varphi(n)$ ----
// 1..n 中与 n 互质的数的个数
// 单点计算: $O(\sqrt{n})$
ll phi(ll n) {
 ll result = n;
 for (ll p = 2; p * p <= n; p++) {
 if (n % p == 0) {
 while (n % p == 0) n /= p;
 result -= result / p;
 }
 }
 if (n > 1) result -= result / n;
 return result;
}

// ---- 欧拉函数线性筛 $O(N)$ ----
// 在线性筛中同步计算:
// $\phi[1] = 1$
// 对于素数 p : $\phi[p] = p-1$
// 当 $i \% p == 0$ 时: $\phi[i * p] = \phi[i] * p$
// 当 $i \% p \neq 0$ 时: $\phi[i * p] = \phi[i] * (p-1)$

// ---- 莫比乌斯函数 $\mu(n)$ ----
// $\mu(1)=1$; 若 n 含平方因子则 $\mu(n)=0$; 若 n 为 k 个不同质数之积则 $\mu(n)=(-1)^k$
// 单点计算 $O(\sqrt{n})$, 线性筛 $O(N)$

// ---- 约数个数 $\tau(n)$ 与约数和 $\sigma(n)$ ----
// 若 $n = \prod p_i^{e_i}$, 则:
// $\tau(n) = \prod (e_i + 1)$
// $\sigma(n) = \prod (p_i^{e_i+1} - 1) / (p_i - 1)$
```

## 4.5 中国剩余定理 (CRT)

```
// ---- CRT (模数两两互质) ----
// 解同余方程组 $x \equiv a_i \pmod{m_i}$, 其中 m_i 两两互质
// $M = \prod m_i$: 求 $x \pmod{M}$
ll crt(const vll& a, const vll& m) {
 ll M = 1;
 for (ll mi : m) M *= mi;
 ll x = 0;
 rep(i, 0, sz(a)) {
 ll Mi = M / m[i];
 ll inv = mod_inv_ext(Mi % m[i], m[i]);
 x = (x + a[i] * Mi % M * inv % M) % M;
 }
 return x;
}

// ---- Garner 算法 (模数未必互质) ----
// 合并同余式 $x \equiv r_i \pmod{m_i}$. 无解时返回 $\{-1, -1\}$.
// 利用扩展欧几里得逐对合并. $O(N \log M)$.
pll crt_general(vll r, vll m) { // 返回 $\{x, lcm\}$ 或 $\{-1, -1\}$
 ll x = 0, M = 1;
 rep(i, 0, sz(r)) {
 ll ri = (r[i] % m[i] + m[i]) % m[i];
 auto [g, a, b] = ext_gcd(M, m[i]);
 if ((ri - x) % g != 0) return {-1, -1};
 x = x + (ri - x) / g * a % (m[i] / g) * M;
 M = M / g * m[i];
 x = (x % M + M) % M;
 }
 return {x, M};
}
```

## 4.6 线性递推与 Berlekamp-Massey 算法

中文名称: BM 算法求最短线性递推式

求生成给定序列的最短线性递推式。时间复杂度  $O(N^2)$ 。

```
// Berlekamp-Massey 算法:
// 给定 $s[0..n-1]$, 求最小的 $\{c_i\}$ 满足
// $s_k = -(c_1 * s_{k-1} + \dots + c_L * s_{k-L})$ 对所有 $k \geq L$ 成立
// 返回向量 C (长度 $L+1$), 其中 $C[0] = 1$, $s[n]$ 可用递推式预测
vll berlekamp_massey(const vll& s) {
 vll C(1, 1), B(1, 1); // C: 当前递推式; B: 上一次最优递推式
 int L = 0, m = 1; // L: 当前递推式的阶数; m: 上次更新至今的步数
 ll b = 1; // 上次更新时的误差
 rep(n, 0, sz(s)) {
 ll d = 0; // 递推式在第 n 项的误差
 rep(i, 0, L + 1) d = (d + C[i] * s[n - i]) % MOD;
 if (d == 0) { m++; continue; } // 递推式仍有效
 vll T = C; // 备份当前递推式
 ll coef = d * mod_inv(b) % MOD; // 修正系数
 // 将 C 扩展到足够长度, 并叠加修正项
 C.resize(max(sz(C), sz(B) + m));
 rep(i, 0, sz(B)) C[i + m] = (C[i + m] - coef * B[i] % MOD + MOD) % MOD;
 if (2 * L <= n) {
 // 当翻倍条件满足时, 更新最优递推式
 B = T;
 b = d;
 m = 1;
 L = n + 1 - L;
 } else {
 m++;
 }
 }
 return C;
}

// ---- 线性递推求第 N 项 (Kitamasa / Bostan-Mori) ----
// 给定 K 阶递推式和前 K 项, 求第 N 项
// 朴素 $O(K^2 \log N)$; 配合 NTT 可做到 $O(K \log K \log N)$
```

## 4.7 快速傅里叶变换与数论变换 (FFT / NTT)

中文名称: 快速傅里叶变换 / 数论变换

多项式乘法,  $O(N \log N)$ 。推荐使用 NTT (模 998244353, 原根 3) , 得到精确整数结果。

```
// ---- 数论变换 (Number Theoretic Transform) ----
// MOD = 998244353, 原根 = 3
// 支持最高 2^{23} 次的多项式 ($MOD-1 = 2^{23} * 7 * 17$)
const int NTT_MOD = 998244353;
const int NTT_ROOT = 3;

void ntt(vll& a, bool invert) {
 int n = sz(a);
 // 位逆序置换 (蝴蝶变换)
 for (int i = 1, j = 0; i < n; i++) {
 int bit = n >> 1;
 for (; j & bit; bit >>= 1) j ^= bit;
 j ^= bit;
 if (i < j) swap(a[i], a[j]);
 }

 for (int len = 2; len <= n; len <= 1) {
 ll wlen = mod_pow(NTT_ROOT, (NTT_MOD - 1) / len, NTT_MOD);
 if (invert) wlen = mod_inv(wlen, NTT_MOD);
 for (int i = 0; i < n; i += len) {
 ll w = 1;
 rep(j, 0, len / 2) {
 ll u = a[i + j];
 ll v = a[i + j + len / 2] * w % NTT_MOD;
 a[i + j] = (u + v) % NTT_MOD;
 a[i + j + len / 2] = (u - v + NTT_MOD) % NTT_MOD;
 w = w * wlen % NTT_MOD;
 }
 }
 }

 if (invert) {
 ll inv_n = mod_inv(n, NTT_MOD);
 for (ll& x : a) x = x * inv_n % NTT_MOD;
 }
}

// 多项式乘法: res = a * b
vll poly_mul(vll a, vll b) {
 int n = 1;
 while (n < sz(a) + sz(b) - 1) n <= 1;
 a.resize(n); b.resize(n);
 ntt(a, false); ntt(b, false);
 rep(i, 0, n) a[i] = a[i] * b[i] % NTT_MOD;
 ntt(a, true);
 a.resize(sz(a) + sz(b) - 1); // 截取实际长度
 return a;
}
```

## 4.8 高斯消元

中文名称: 高斯消元

解线性方程组  $Ax = b$ , 求行列式、秩、逆矩阵。 $O(N^3)$ 。

```

// ----- 实数域高斯消元 -----
// 返回值: 0 = 唯一解, 1 = 无穷多解, -1 = 无解
// 若有解, ans 将包含一组可行解
int gauss(vector<vector<double>>& a, vector<double>& b, vector<double>& ans) {
 int n = sz(a), m = sz(a[0]);
 vvi where(m, -1);
 ans.assign(m, 0);

 for (int col = 0, row = 0; col < m && row < n; ++col) {
 // 选主元 (部分主元法)
 int sel = row;
 rep(i, row, n) if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
 if (abs(a[sel][col]) < 1e-9) continue; // 该列已消
 swap(a[sel], a[row]);
 swap(b[sel], b[row]);
 where[col] = row;

 // 消去其他行
 rep(i, 0, n) {
 if (i == row) continue;
 double c = a[i][col] / a[row][col];
 rep(j, col, m) a[i][j] -= a[row][j] * c;
 b[i] -= b[row] * c;
 }
 row++;
 }

 // 判断解的情况
 rep(i, 0, n) {
 bool all_zero = true;
 rep(j, 0, m) if (abs(a[i][j]) > 1e-9) all_zero = false;
 if (all_zero && abs(b[i]) > 1e-9) return -1; // 无解
 }
 rep(j, 0, m) {
 if (where[j] != -1) ans[j] = b[where[j]] / a[where[j]][j];
 }
 rep(j, 0, m) if (where[j] == -1) return 1; // 无穷多解
 return 0; // 唯一解
}

// ----- 模意义下高斯消元 -----
// 适用于有限域 F_MOD: 使用模逆元进行除法
int gauss_mod(vvll& a, vll& b, vll& ans, ll mod = MOD) {
 int n = sz(a), m = sz(a[0]);
 vi where(m, -1);
 ans.assign(m, 0);

 for (int col = 0, row = 0; col < m && row < n; ++col) {
 // 选主元: 选 col 列绝对值最大 (即非零) 的行
 int sel = row;
 rep(i, row, n) if (a[i][col] > a[sel][col]) sel = i;
 if (a[sel][col] == 0) continue;
 swap(a[sel], a[row]);
 swap(b[sel], b[row]);
 where[col] = row;

 // 将主元归一化
 ll inv = mod_inv(a[row][col], mod);
 rep(j, col, m) a[row][j] = a[row][j] * inv % mod;
 b[row] = b[row] * inv % mod;

 // 消去其他行
 rep(i, 0, n) {
 if (i == row) continue;
 ll c = a[i][col];
 rep(j, col, m) a[i][j] = (a[i][j] - c * a[row][j] % mod + mod) % mod;
 b[i] = (b[i] - c * b[row] % mod + mod) % mod;
 }
 row++;
 }

 // 判断解的情况 (与实数域类似)
 rep(i, 0, n) {
 bool all_zero = true;
 rep(j, 0, m) if (a[i][j] != 0) all_zero = false;
 if (all_zero && b[i] != 0) return -1; // 无解
 }
 rep(j, 0, m) {
 if (where[j] != -1) ans[j] = b[where[j]]; // a[where[j]][j] 已归一化为 1
 }
 rep(j, 0, m) if (where[j] == -1) return 1; // 无穷多解
 return 0; // 唯一解
}

// ----- 模意义下行列式 (辗转相除法, 不要求模数为素数) -----
// O(N^3), 适用于任意模数
ll det_mod(vvll a, ll mod) {
 int n = sz(a);
 ll det = 1;
 rep(i, 0, n) {
 rep(j, i + 1, n) {
 while (a[j][i] != 0) {
 ll t = a[i][i] / a[j][i];
 rep(k, i, n) {
 a[i][k] = (a[i][k] - t * a[j][k] % mod + mod) % mod;
 swap(a[i][k], a[j][k]);
 }
 det = (-det + mod) % mod;
 }
 }
 det = det * a[i][i] % mod;
 if (det == 0) return 0;
 }
 return det;
}

```

## 4.9 矩阵快速幂

中文名称: 矩阵快速幂

求解线性递推或  $N$  很大的动态规划。 $K \times K$  矩阵,  $O(K^3 \log N)$ 。

```

template <typename T>
struct Matrix {
 int n, m;
 vector<vector<T>>> a;

 Matrix(int n_, int m_) : n(n_), m(m_), a(n, vector<T>(m)) {}
 Matrix(const vector<vector<T>>& a_) : n(sz(a_)), m(sz(a_[0])), a(a_) {}

 Matrix operator*(const Matrix& o) const {
 // assert(m == o.n);
 Matrix res(n, o.m);
 rep(i, 0, n) rep(k, 0, m) {
 if (a[i][k] == 0) continue; // 稀疏矩阵优化
 rep(j, 0, o.m) {
 res.a[i][j] = (res.a[i][j] + a[i][k] * o.a[k][j]) % MOD;
 }
 }
 return res;
 }

 Matrix pow(ll exp) const {
 // assert(n == m); // 必须是方阵
 Matrix res(n, n);
 rep(i, 0, n) res.a[i][i] = 1; // 单位矩阵
 Matrix base = *this;
 while (exp) {
 if (exp & 1) res = res * base;
 base = base * base;
 exp >>= 1;
 }
 return res;
 }

 T& operator()(int i, int j) { return a[i][j]; }
};

// 用法示例：斐波那契数列
// Matrix<ll> M({{1,1},{1,0}}); M = M.pow(N); cout << M(0,1);

```

#### 4.10 离散对数 / 大步小步算法 (BSGS)

中文名称：BSGS 算法 / 大步小步算法

求解  $a^x \equiv b \pmod{m}$ ，其中  $\gcd(a, m) = 1$ 。 $O(\sqrt{m})$ 。

```

// BSGS：求最小的 $x \geq 0$ 满足 $a^x \equiv b \pmod{m}$
// 若无解返回 -1。要求 $\gcd(a, m) = 1$ 。
ll bsgs(ll a, ll b, ll m) {
 a %= m; b %= m;
 if (b == 1) return 0;
 ll n = (ll)sqrt(m) + 1;

 // 小步：存储 $a^j * b \pmod{m} \rightarrow j$ ，其中 $j \in [0, n)$
 unordered_map<ll, ll, custom_hash> baby;
 ll cur = b;
 rep(j, 0, n) {
 baby[cur] = j;
 cur = cur * a % m;
 }

 // 大步：计算 a^n ，然后枚举 $(a^n)^i$
 ll giant = mod_pow(a, n, m);
 cur = 1;
 rep(i, 1, n + 1) {
 cur = cur * giant % m;
 if (baby.count(cur)) {
 ll ans = i * n - baby[cur];
 if (ans >= 0) return ans;
 }
 }
 return -1;
}

// ---- 扩展 BSGS ----
// 适用于 $\gcd(a, m) \neq 1$ 的情况。处理所有情况。
ll ex_bsgs(ll a, ll b, ll m) {
 a %= m; b %= m;
 if (b == 1 || m == 1) return 0;
 ll g, k = 0, ad = 1;
 while ((g = __gcd(a, m)) > 1) {
 if (b % g != 0) return -1;
 k++; m /= g; b /= g;
 ad = ad * (a / g) % m;
 if (ad == b) return k;
 }
 // 至此 $\gcd(a, m) = 1$ ，调用标准 BSGS
 ll n = (ll)sqrt(m) + 1;
 unordered_map<ll, ll, custom_hash> baby;
 ll cur = b;
 rep(j, 0, n) { baby[cur] = j; cur = cur * a % m; }
 ll giant = mod_pow(a, n, m);
 cur = ad;
 rep(i, 1, n + 1) {
 cur = cur * giant % m;
 if (baby.count(cur)) return i * n - baby[cur] + k;
 }
 return -1;
}

```

## 4.11 博弈论

```
// ---- Nim 游戏 ----
// N 堆石子，每轮可从任意一堆取至少 1 个。取到最后者为胜。
// 先手必胜当且仅当所有堆石子数的异或和 != 0
bool nim_winner(const vi& piles) {
 int x = 0;
 for (int p : piles) x ^= p;
 return x != 0;
}

// ---- SG 函数 / Sprague-Grundy 定理 ----
// 公平组合游戏：状态 → 可到达状态集合
// SG(state) = mex{SG(可到达状态)}
// 组合游戏的 SG = 各子游戏 SG 的异或和；SG != 0 则先手必胜
// 使用 DP / 记忆化搜索计算 SG 值
vi compute_grundy(int max_n) {
 vi sg(max_n + 1, 0);
 rep(i, 1, max_n + 1) {
 set<int> reachable;
 // 对每个可选操作：
 // reachable.insert(sg[i - move]); // 具体取决于游戏规则
 // 然后：while (reachable.count(sg[i])) sg[i]++;
 }
 return sg;
}
```

## 4.12 数学杂项工具

```
// ---- 位运算技巧 ----
int popcnt(ll x) { return __builtin_popcountll(x); } // 统计 1 的个数
int clz(ll x) { return __builtin_clzll(x); } // 统计前导零个数
int ctz(ll x) { return __builtin_ctzll(x); } // 统计末尾零个数
int lg2(ll x) { return 63 - __builtin_clzll(x); } // floor(log2(x))
bool is_pow2(ll x) { return x > 0 && (x & (x - 1)) == 0; } // 判断是否为 2 的幂

// Gosper's hack：生成下一个等 1 位数量的掩码，O(1) 均摊
ll next_perm_mask(ll mask) {
 ll c = mask & -mask;
 ll r = mask + c;
 return r | (((r ^ mask) >> 2) / c);
}

// ---- 分数类 ----
struct Frac {
 ll num, den;
 Frac(ll n = 0, ll d = 1) {
 ll g = __gcd(abs(n), abs(d));
 num = n / g; den = d / g;
 if (den < 0) { num = -num; den = -den; }
 }
 Frac operator+(const Frac& o) const { return Frac(num*o.den + o.num*den, den*o.den); }
 Frac operator-(const Frac& o) const { return Frac(num*o.den - o.num*den, den*o.den); }
 Frac operator*(const Frac& o) const { return Frac(num*o.num, den*o.den); }
 Frac operator/(const Frac& o) const { return Frac(num*o.den, den*o.num); }
 bool operator<(const Frac& o) const { return (__int128)num * o.den < (__int128)o.num * den; }
};

// ---- 整数平方根 ----
ll floor_sqrt(ll n) {
 if (n <= 0) return 0;
 ll x = (ll)sqrtl(n);
 while ((x + 1) * (x + 1) <= n) x++;
 while (x * x > n) x--;
 return x;
}

// ---- 整除分块 / 调和引理 ----
// sum_{i=1..n} floor(n/i) 的计算
// O(√n) 遍历：对于每个 distinct value v，对应区间 [l, r] = [n/(v+1)+1, n/v]
void harmonic_lemma(ll n) {
 for (ll l = 1, r; l <= n; l = r + 1) {
 ll v = n / l;
 r = n / v;
 // v 是区间 [l, r] 内所有 i 对应的 floor(n/i) 值
 // 处理：(r - l + 1) 个元素共享相同的 v 值
 }
}
```

## 4.13 常用数据结构补充

以下数据结构在组合计数、区间查询、动态连通性判断等数学问题中频繁出现，收录 KACTL 和 jiangly 两个经典实现作为参考。

### 4.13.1 并查集 (DSU / Disjoint Set Union)

**KACTL 版本**：利用负数存储大小，代码极短。find 递归路径压缩；join 按大小合并。每组大小 = -e[find(x)]。

```
// KACTL 并查集 — 负数组缩写法
struct UF {
 vi e;
 UF(int n) : e(n, -1) {}
 bool sameSet(int a, int b) { return find(a) == find(b); }
 int size(int x) { return -e[find(x)]; }
 int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
 bool join(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false;
 if (e[a] > e[b]) swap(a, b); // 负值：更小的 e[x] 表示更大的集合
 e[a] += e[b]; e[b] = a;
 return true;
 }
};
```

**jiangly 版本**：迭代式 find（安全避免递归爆栈），独立 siz 数组。merge 无需按大小合并即可直接使用。

```
// jiangly 并查集 — 迭代路径压缩
struct DSU {
 vector<int> f, siz;
 DSU(int n) {
 f.resize(n);
 iota(f.begin(), f.end(), 0);
 siz.assign(n, 1);
 }
 int find(int x) {
 while (x != f[x])
 x = f[x] = f[f[x]]; // 路径压缩
 return x;
 }
 bool merge(int x, int y) {
 x = find(x); y = find(y);
 if (x == y) return false;
 siz[x] += siz[y];
 f[y] = x;
 return true;
 }
 int size(int x) { return siz[find(x)]; }
};
```

#### 4.13.2 树状数组 (Fenwick Tree)

**KACTL 版本：**使用 `pos |= pos + 1` 的更新方式与 `pos &= pos - 1` 的前缀查询。内置 `lower_bound` 查询第一个前缀和  $\geq \text{sum}$  的位置。

```
// KACTL 树状数组 — 带 lower_bound
struct Fenwick {
 vector<ll> s;
 Fenwick(int n) : s(n) {}
 // 单点增加: pos 从 0 开始
 void add(int pos, ll dif) {
 for (; pos < sz(s); pos |= pos + 1)
 s[pos] += dif;
 }
 // 前缀和 [0, pos)
 ll sum(int pos) {
 ll res = 0;
 for (; pos > 0; pos &= pos - 1)
 res += s[pos - 1];
 return res;
 }
 // 第一个前缀和 $\geq \text{sum}$ 的下标 (0-indexed) ; 不存在时返回 -1
 int lower_bound(ll sum) {
 if (sum <= 0) return -1;
 int pos = 0;
 for (int pw = 1 << 25; pw; pw >= 1)
 if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
 pos += pw, sum -= s[pos - 1];
 return pos;
 }
};
```

**jiangly 版本：**模板化，使用标准 `i += i & -i` 索引。内置 `select(k)` 查找第  $k$  个 (0-indexed) 1 的位置。

```
// jiangly 树状数组 — 模板化, 带 select/kth
template<typename T>
struct Fenwick {
 int n;
 vector<T> a;
 Fenwick(int n_) { init(n_); }
 void init(int n_) { n = n_; a.assign(n, T{}); }
 // 下标从 0 开始
 void add(int x, const T& v) {
 for (int i = x + 1; i <= n; i += i & -i)
 a[i - 1] = a[i - 1] + v;
 }
 T sum(int x) { // 前缀和 [0, x)
 T ans{};
 for (int i = x; i > 0; i -= i & -i)
 ans = ans + a[i - 1];
 return ans;
 }
 T rangeSum(int l, int r) { return sum(r) - sum(l); }
 // 第 k 个 (0-indexed) 1 的位置, 即最小的 x 使得 sum(x) > k
 int select(const T& k) {
 int x = 0;
 T cur{};
 for (int i = 1 << __lg(n); i; i /= 2)
 if (x + i <= n && cur + a[x + i - 1] <= k) {
 x += i;
 cur = cur + a[x - 1];
 }
 return x;
 }
};
```

#### 4.13.3 线段树 — KACTL 迭代式

相比递归线段树，迭代版本常数更小、代码更短、无递归调用开销。2N 存储，半开区间  $[l, r)$ 。

```
// KACTL 迭代式线段树 (2N 存储, 半开区间 [l, r))
struct SegTree {
 typedef int T;
 static constexpr T unit = INT_MIN; // 单元: max 用 -INF; sum 用 0; min 用 INF
 T f(T a, T b) { return max(a, b); } // 结合函数, 按需修改

 vector<T> s;
 int n;

 SegTree(int n = 0, T def = unit) : s(2 * n, def), n(n) {}

 // 单点赋值 (非增量), 索引从 0 开始
 void update(int pos, T val) {
 for (s[pos += n] = val; pos /= 2;)
 s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
 }

 // 区间查询 [l, r), 半开区间
 T query(int l, int r) {
 T ra = unit, rb = unit;
 for (l += n, r += n; l < r; l /= 2, r /= 2) {
 if (l & 1) ra = f(ra, s[l++]);
 if (r & 1) rb = f(s[--r], rb);
 }
 return f(ra, rb);
 }
};
```

#### 4.13.4 强连通分量 — Tarjan 算法 (jiangly 版本)

关键修正: 回边 (已访问但未出栈的节点) 使用 `dfn[y]` 而非 `low[y]` 更新 `low[x]`, 这是正确的 Tarjan 写法。

```
// jiangly 强连通分量 — Tarjan 算法 (dfn[y] 用于回边更新)
struct SCC {
 int n;
 vector<vector<int>> adj;
 vector<int> stk, dfn, low, bel;
 int cur, cnt;

 SCC(int n) {
 this->n = n;
 adj.assign(n, {});
 dfn.assign(n, -1);
 low.resize(n);
 bel.assign(n, -1);
 cur = cnt = 0;
 }

 void addEdge(int u, int v) { adj[u].push_back(v); }

 void dfs(int x) {
 dfn[x] = low[x] = cur++;
 stk.push_back(x);
 for (auto y : adj[x]) {
 if (dfn[y] == -1) {
 dfs(y);
 low[x] = min(low[x], low[y]);
 } else if (bel[y] == -1) {
 low[x] = min(low[x], dfn[y]); // 注意: 用 dfn[y] 而非 low[y]
 }
 }
 if (dfn[x] == low[x]) {
 int y;
 do {
 y = stk.back();
 bel[y] = cnt;
 stk.pop_back();
 } while (y != x);
 cnt++;
 }
 }

 // 返回每个节点的 SCC 编号 (从 0 开始, 按拓扑逆序编号)
 vector<int> work() {
 for (int i = 0; i < n; i++)
 if (dfn[i] == -1) dfs(i);
 return bel;
 }
};
```

#### 4.13.5 重链剖分 (HLD) — jiangly 版本

全面版 HLD, 包含: `jump` (向上跳  $k$  步)、`isAncestor` (祖先判定)、`rootedParent` (换根父节点)、`rootedSize` (换根子树大小)、`rootedLca` (换根 LCA)、`path` (路径查询)。

```
// jiangly 重链剖分 — 全面版 (含 jump, isAncestor, rootedParent, rootedLca)
```

```
struct HLD {
 int n;
 vector<vector<int>> adj;
 vector<int> parent, depth, sz, in, out, head, rev;
 int cur;

 HLD(int n) {
 this->n = n;
 adj.resize(n);
 parent.resize(n);
 depth.resize(n);
 sz.resize(n);
 in.resize(n);
 out.resize(n);
 head.resize(n);
 rev.resize(n);
 cur = 0;
 }

 void addEdge(int u, int v) {
 adj[u].push_back(v);
 adj[v].push_back(u);
 }

 // 第一遍 DFS: 计算 parent、depth、子树大小, 并将重儿子换到 adj[u][0]
 void dfs1(int u, int p) {
 parent[u] = p;
 depth[u] = (p == -1 ? 0 : depth[p] + 1);
 sz[u] = 1;
 for (auto& v : adj[u]) {
 if (v == p) continue;
 dfs1(v, u);
 sz[u] += sz[v];
 if (adj[u][0] == p || sz[v] > sz[adj[u][0]])
 swap(v, adj[u][0]); // 重儿子放到首位
 }
 }

 // 第二遍 DFS: 分配 in/out、head、rev
 void dfs2(int u, int p) {
 in[u] = cur++;
 rev[in[u]] = u;
 for (auto v : adj[u]) {
 if (v == p) continue;
 head[v] = (v == adj[u][0] ? head[u] : v);
 dfs2(v, u);
 }
 out[u] = cur;
 }

 void work(int root = 0) {
 head[root] = root;
 dfs1(root, -1);
 dfs2(root, -1);
 }

 // 判断 u 是否是 v 的祖先 (包含相等情况)
 bool isAncestor(int u, int v) {
 return in[u] <= in[v] && in[v] < out[u];
 }

 // 从 u 沿树向上跳 k 步; k 太大时返回 -1
 int jump(int u, int k) {
 while (u != -1) {
 int toHead = depth[u] - depth[head[u]];
 if (k <= toHead) {
 return rev[in[u] - k];
 }
 k -= toHead + 1;
 u = parent[head[u]];
 }
 return -1;
 }

 // 在以 root 为根时 u 的父节点
 int rootedParent(int root, int u) {
 if (root == u) return -1;
 if (!isAncestor(u, root))
 return parent[u];
 return jump(root, depth[root] - depth[u] - 1);
 }

 // 在以 root 为根时 u 的子树大小
 int rootedSize(int root, int u) {
 if (root == u) return n;
 if (!isAncestor(u, root))
 return sz[u];
 int v = jump(root, depth[root] - depth[u] - 1);
 return n - sz[v];
 }

 // 遍历 u-v 路径上的重链区间, 调用 f(l, r) 处理 [l, r) (保证 l <= r)
 template<typename F>
 void path(int u, int v, F&& f) {
 while (head[u] != head[v]) {
 if (depth[head[u]] < depth[head[v]]) swap(u, v);
 f(in[head[u]], in[u] + 1);
 u = parent[head[u]];
 }
 if (depth[u] < depth[v]) swap(u, v);
 f(in[v], in[u] + 1);
 }

 // LCA (最近公共祖先)
 int lca(int u, int v) {
 while (head[u] != head[v]) {
 if (depth[head[u]] < depth[head[v]]) swap(u, v);
 u = parent[head[u]];
 }
 return depth[u] < depth[v] ? u : v;
 }
}
```



```
// 换根 LCA: rootedLca(root, a, b) = lca(a,b) ^ lca(b,root) ^ lca(root,a)
int rootedLca(int root, int a, int b) {
 return lca(a, b) ^ lca(b, root) ^ lca(root, a);
}

// 两点间距离 (按需求自行展开)
int dist(int u, int v) {
 return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}
};
```

#### 4.14 图论核心注意事项

本节列出图论算法实现中的关键易错点，包含 Dijkstra、Floyd 常见陷阱。

##### 4.14.1 Dijkstra INF 设置

**重要：**对于带权图，INF 必须设置为 **1e18**（而非 1e9），因为边权可以累加到很大：

```
const ll INF = 1e18; // 足以容纳 10^5 条边各 10^9 权重的路径和
```

错误使用 1e9 会导致 dist 数组溢出为“假更新值”，使得松弛判断 `dist[v] > dist[u] + w` 产生误判。

##### 4.14.2 Floyd 循环顺序

Floyd-Warshall 算法中 k 循环必须是最外层：

```
// 正确写法
for (int k = 0; k < n; k++) // 最外层 k
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

// 注意：若将 k 放在内层，算法退化为错误的松弛顺序
```

原因：k 在最外层保证“使用前 k 个顶点的最短路径”这一阶段定义成立。将 k 放在内层会丢失部分中间顶点的路径更新。

## 5. 字符串算法 (String Algorithms)

### 5.1 KMP 算法 (Knuth-Morris-Pratt)

#### 核心思想

KMP 算法利用模式串自身的对称性构造前缀函数（prefix function / pi 数组），实现线性时间的单模式匹配。前缀函数 `pi[i]` 表示子串 `s[0..i]` 的最长真前缀（true prefix）同时是真后缀（true suffix）的长度。

匹配失败时，利用 `pi` 数组将模式串向右滑动更多位置，避免朴素算法中主串指针的回退。

##### 5.1.1 前缀函数

构造 `pi` 数组本身也是一个 KMP 匹配过程——用模式串的前缀匹配自己。

**时间复杂度：** $O(n)$ ，其中  $n$  为模式串长度。每个字符最多被回退一次，均摊  $O(1)$ 。

**空间复杂度：** $O(n)$ 。

```
// 计算前缀函数 pi 数组
// pi[i] : 子串 s[0..i] 的最长真前缀且是真后缀的长度
// OI-wiki 参考实现
vector<int> getPi(const string &s) {
 int n = (int)s.size();
 vector<int> pi(n);
 // pi[0] = 0 (单个字符没有真前缀/真后缀)
 for (int i = 1, j = 0; i < n; i++) {
 // 回退：当前字符不匹配时，尝试更短的前缀
 while (j > 0 && s[i] != s[j]) {
 j = pi[j - 1];
 }
 // 匹配成功：扩展前缀长度
 if (s[i] == s[j]) {
 j++;
 }
 pi[i] = j;
 }
 return pi;
}
```

##### 5.1.2 KMP 匹配

用预处理好的 `pi` 数组在文本串中匹配模式串。

**时间复杂度：** $O(n+m)$ ，其中  $n$  为文本串长度， $m$  为模式串长度。

```
// KMP 字符串匹配
// 返回所有匹配位置 (0-indexed 起始下标)
// 文本串 text, 模式串 pat
vector<int> kmp(const string &text, const string &pat) {
 vector<int> pi = getPi(pat);
 vector<int> matches;
 int n = (int)text.size(), m = (int)pat.size();
 // j 表示当前已匹配的模式字符串数
 for (int i = 0, j = 0; i < n; i++) {
 // 不匹配时按 pi 回退
 while (j > 0 && text[i] != pat[j]) {
 j = pi[j - 1];
 }
 if (text[i] == pat[j]) {
 j++;
 }
 // 完整匹配
 if (j == m) {
 matches.push_back(i - m + 1);
 j = pi[j - 1]; // 继续匹配后续位置
 }
 }
 return matches;
}
```

### 5.1.3 最小循环节

利用 `pi` 数组可以  $O(n)O(n)$  求出串的最小循环节长度：

```
// 求字符串 s 的最小循环节长度
// 若 len % (len - pi[len-1]) == 0, 则最小循环节为 len - pi[len-1]
// 否则整个串自身是唯一的循环节
int minCycleLen(const string &s) {
 int n = (int)s.size();
 vector<int> pi = getPi(s);
 int cycle = n - pi[n - 1];
 if (n % cycle == 0) return cycle;
 return n;
}
```

## 5.2 Z 算法 (Z-Algorithm)

### 核心理想

Z 函数 `z[i]` 表示字符串 `s` 与 `s[i..n-1]` (即以 `s[i]` 开头的后缀) 的最长公共前缀 (LCP) 长度。特别地, `z[0]` 通常定义为 0 或 `n`。

算法维护一个区间 `[l, r]`, 其中 `r` 是当前匹配到的最远右端点。当计算 `z[i]` 时:

**CRITICAL:** Z 算法必须维护 `[l, r]` 窗口, 这是保证均摊线性复杂度的关键。

时间复杂度:  $O(n)O(n)$ , 均摊分析类似 KMP。

空间复杂度:  $O(n)O(n)$ 。

### 实现

```
// Z 函数: 维护 [l, r] 窗口
// z[i] = LCP(s, s[i..n-1])
// 约定 z[0] = 0
vector<int> zFunction(const string &s) {
 int n = (int)s.size();
 vector<int> z(n);
 // [l, r] 是当前已匹配的最远区间 (闭区间)
 int l = 0, r = 0;
 for (int i = 1; i < n; i++) {
 // 情况1: i 在窗口内, 可以利用已有信息
 if (i <= r) {
 z[i] = min(r - i + 1, z[i - l]);
 }
 // 情况2: 朴素扩展 (i > r 时 z[i]==0, 直接从 while 开始扩展)
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
 z[i]++;
 }
 // 更新窗口 [l, r]
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
 }
 return z;
}
```

### 应用: 字符串匹配

利用 Z 函数可以  $O(n+m)O(n+m)$  完成模式匹配——将模式串拼接到文本串前, 用分隔符隔开:

```
// 使用 Z 函数进行字符串匹配
// 返回所有匹配位置 (文本串中的起始下标, 0-indexed)
vector<int> zMatch(const string &text, const string &pat) {
 // 构造拼接串: pat + '#' + text
 string s = pat + "#" + text;
 vector<int> z = zFunction(s);
 int m = (int)pat.size();
 vector<int> matches;
 for (int i = m + 1; i < (int)s.size(); i++) {
 if (z[i] == m) {
 matches.push_back(i - m - 1);
 }
 }
 return matches;
}
```

## 5.3 Manacher 算法（马拉车 / 最长回文子串）

---

### 核心思想

Manacher 算法能在  $O(n)$  时间内求出字符串每个位置为中心的最长回文半径。

**关键技巧：**在原串每个字符前后插入 #，将偶长度回文统一转化为奇长度回文处理。例如 "aba" → "#a#b#a#"，"aa" → "#a#a#"。

算法维护已找到的最右回文边界  $[l, r]$  和对应的中心  $c$ 。当计算位置  $i$  的回文半径时，利用其关于  $c$  的对称点  $mirror = 2*c - i$  的已知信息进行加速。

**时间复杂度：** $O(n)$ 。

**空间复杂度：** $O(n)$ 。

## 实现

```
// Manacher 算法：求每个位置的最长回文半径
// 输入字符串 s（原始串）
// 返回 d1（奇数长度回文半径）和 d2（偶数长度回文半径）
// 或者返回处理后字符串 t 的回文半径数组 p
struct ManacherResult {
 string t; // 插入 # 后的字符串
 vector<int> radius; // radius[i]：以 i 为中心的最长回文半径（以 t 中字符数为单位）
};

ManacherResult manacher(const string &s) {
 int n = (int)s.size();

 // 构造 t，插入 # 分隔符
 // s = "abba" → t = "^a#b#b#a#$"（前后加哨兵简化边界判断）
 string t = "^a#";
 for (char c : s) {
 t.push_back(c);
 t.push_back('#');
 }
 t.push_back('$');

 int m = (int)t.size();
 vector<int> radius(m);

 // C：当前最右回文子串的中心
 // R：当前最右回文子串的右边界
 int C = 0, R = 0;

 for (int i = 1; i < m - 1; i++) {
 // i 关于 C 的对称点
 int mirror = 2 * C - i;

 // 利用对称性加速
 if (i < R) {
 radius[i] = min(R - i, radius[mirror]);
 }

 // 朴素扩展
 while (t[i + radius[i] + 1] == t[i - radius[i] - 1]) {
 radius[i]++;
 }

 // 更新最右回文边界
 if (i + radius[i] > R) {
 C = i;
 R = i + radius[i];
 }
 }

 return {t, radius};
}

// 获取原始串 s 中以 center 为中心的最长回文子串
string getLongestPalindrome(const string &s) {
 auto [t, radius] = manacher(s);

 // 找到最大半径及其中心
 int bestCenter = 0, bestRadius = 0;
 int m = (int)t.size();
 for (int i = 1; i < m - 1; i++) {
 if (radius[i] > bestRadius) {
 bestRadius = radius[i];
 bestCenter = i;
 }
 }

 // 从 t 的索引反推 s 的起始位置
 // t 中位置 i 对应 s 中位置 (i - 2) / 2
 int start = (bestCenter - bestRadius - 2) / 2;
 int len = bestRadius; // radius 就是 t 中的回文半径，等于 s 中回文长度
 return s.substr(start, len);
}

// 判断子串 s[l..r]（闭区间）是否为回文（预处理后 O(1) 查询）
vector<int> manacherOdd(const string &s) {
 // 不插入 # 的版本：直接求奇数长度回文半径
 // d1[i]：以 i 为中心的最长奇数回文半径（包含自身）
 int n = (int)s.size();
 vector<int> d1(n);
 int l = 0, r = -1;
 for (int i = 0; i < n; i++) {
 int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
 while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
 d1[i] = k--;
 if (i + k > r) {
 l = i - k;
 r = i + k;
 }
 }
 return d1;
}

vector<int> manacherEven(const string &s) {
 // 偶数长度回文半径
 // d2[i]：以 i 和 i-1 为中心的偶数回文半径
 int n = (int)s.size();
 vector<int> d2(n);
 int l = 0, r = -1;
 for (int i = 0; i < n; i++) {
 int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
 while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
 d2[i] = k--;
 if (i + k > r) {
 l = i - k - 1;
 r = i + k;
 }
 }
 return d2;
}
```

## 5.4 AC 自动机 (Aho-Corasick Automaton)

---

### 核心思想

AC 自动机是多模式匹配的经典算法，基于 Trie 树并引入失配指针（fail link）和输出指针（output/dict link）。失配指针指向当前节点的最长真后缀所对应的节点。构建好后，只需对文本串扫描一遍即可找出所有模式串的出现位置。

### 构建步骤：

**CRITICAL:** BFS 构建 fail 时，必须记得 `dict[v] |= dict[fail[v]]`，否则在匹配时可能漏掉模式串的出现。

### 时间复杂度：



```

struct AhoCorasick {
 static constexpr int ALPHA = 26; // 字符集大小

 vector<array<int, ALPHA>> nxt; // Trie 转移边
 vector<int> fail; // 失配指针
 vector<int> dict; // 输出标记 (bitmask 或 end count)
 vector<int> cnt; // 以该节点结尾的模式串数量

 AhoCorasick() {
 newNode(); // 根节点 0
 }

 int newNode() {
 nxt.emplace_back();
 nxt.back().fill(-1);
 fail.push_back(0);
 dict.push_back(0);
 cnt.push_back(0);
 return (int)nxt.size() - 1;
 }

 // 插入一个模式串
 void insert(const string &s, int id = 0) {
 int v = 0;
 for (char ch : s) {
 int c = ch - 'a';
 if (nxt[v][c] == -1) {
 nxt[v][c] = newNode();
 }
 v = nxt[v][c];
 }
 dict[v] |= (1 << id); // 标记模式串 id 的结束
 cnt[v]++;
 }

 // BFS 构建 fail 指针 + 输出传播
 void build() {
 queue<int> q;

 // 第 0 层 (根的直接儿子) : fail 指向根
 for (int c = 0; c < ALPHA; c++) {
 int u = nxt[0][c];
 if (u != -1) {
 fail[u] = 0;
 q.push(u);
 } else {
 nxt[0][c] = 0; // 优化: 不回退到 -1, 直接指向根
 }
 }

 // BFS 按层构建
 while (!q.empty()) {
 int v = q.front(); q.pop();

 // CRITICAL: 输出传播 —— 继承 fail 链上的输出标记
 dict[v] |= dict[fail[v]];
 cnt[v] += cnt[fail[v]];

 for (int c = 0; c < ALPHA; c++) {
 int u = nxt[v][c];
 if (u != -1) {
 // 正常子节点: fail 指向 fail[v] 的对应子节点
 fail[u] = nxt[fail[v]][c];
 q.push(u);
 } else {
 // 优化: 补齐转移边, 避免匹配时沿 fail 链跳转
 nxt[v][c] = nxt[fail[v]][c];
 }
 }
 }
 }

 // 在文本串 text 中匹配所有模式串
 // 返回每个模式串的出现次数
 vector<int> match(const string &text, int patCount) {
 vector<int> result(patCount);
 int v = 0;
 for (char ch : text) {
 int c = ch - 'a';
 v = nxt[v][c]; // 直接走转移边 (上面已补齐)

 // 检查当前节点及 fail 链上的所有模式串
 int cur = v;
 while (cur != 0 && dict[cur]) {
 for (int i = 0; i < patCount; i++) {
 if (dict[cur] & (1 << i)) {
 result[i]++;
 }
 }
 cur = fail[cur];
 }
 }
 return result;
 }

 // 匹配并返回每个位置结束的模式串 id 列表
 vector<vector<int>> matchPositions(const string &text, int patCount) {
 vector<vector<int>> pos(patCount);
 int v = 0;
 int m = (int)text.size();
 for (int i = 0; i < m; i++) {
 int c = text[i] - 'a';
 v = nxt[v][c];
 // 遍历 fail 链收集所有匹配到的模式串
 int cur = v;
 while (cur != 0) {
 for (int j = 0; j < patCount; j++) {
 if (dict[cur] & (1 << j)) {
 pos[j].push_back(i - (int)/* 这里需要模式串实际长度, 可存 len 数组 */0 + 1);
 }
 }
 cur = fail[cur];
 }
 }
 }
}

```

```

 }
 return pos;
}
};

```

## 通用字符集版本（使用 map）

对于较大或不确定的字符集，使用 `map<char, int>` 存储转移边：

```

struct AhoCorasickMap {
 vector<unordered_map<char, int>> nxt; // 用 unordered_map 存转移
 vector<int> fail;
 vector<int> cnt; // 以该节点结尾的模式串计数

 AhoCorasickMap() { newNode(); }

 int newNode() {
 nxt.emplace_back();
 fail.push_back(0);
 cnt.push_back(0);
 return (int)nxt.size() - 1;
 }

 void insert(const string &s) {
 int v = 0;
 for (char c : s) {
 if (!nxt[v].count(c)) nxt[v][c] = newNode();
 v = nxt[v][c];
 }
 cnt[v]++;
 }

 void build() {
 queue<int> q;
 for (auto &[ch, u] : nxt[0]) {
 fail[u] = 0;
 q.push(u);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();

 // 输出传播
 cnt[v] += cnt[fail[v]];

 for (auto &[c, u] : nxt[v]) {
 // fail[u] = nxt[fail[v]][c] if exists else 0
 int f = fail[v];
 while (f && !nxt[f].count(c)) f = fail[f];
 fail[u] = nxt[f].count(c) ? nxt[f][c] : 0;
 q.push(u);
 }
 }
 }

 // 返回文本中匹配到的模式串总数（按节点累计）
 long long countMatches(const string &text) {
 int v = 0;
 long long ans = 0;
 for (char c : text) {
 while (v && !nxt[v].count(c)) v = fail[v];
 if (nxt[v].count(c)) v = nxt[v][c];
 ans += cnt[v]; // cnt 已包含 fail 链传播
 }
 return ans;
 }
};

```

## 5.5 后缀数组 (Suffix Array)

### 核心思想

后缀数组是处理字符串问题的强大工具。给定长度为  $n$  的字符串 `s`：

本节使用**倍增法 + 基数排序**，复杂度为  $O(n \log n)O(n \log n)$ 。Kasai 算法可在  $O(n)O(n)$  内求出 `height` 数组。

#### 5.5.1 倍增法 + 基数排序（计数排序优化）

**核心技巧：**每次倍增时用两次计数排序（先按第二关键字排，再按第一关键字排），保证单轮排序  $O(n)O(n)$ 。

**时间复杂度：** $O(n \log n)O(n \log n)$ 。

**空间复杂度：** $O(n)O(n)$ 。



```

// 后缀数组：倍增 + 基数（计数）排序
struct SuffixArray {
 int n;
 string s;
 vector<int> sa; // 后缀数组：sa[i] = 排名 i 的后缀的起始下标
 vector<int> rk; // 排名数组：rk[i] = 后缀 i 的排名
 vector<int> height; // height[i] = LCP(sa[i-1], sa[i])

 SuffixArray(const string &s) : s(s), n((int)s.size()) {
 sa.resize(n);
 rk.resize(n);
 height.resize(n);
 buildSA();
 buildHeight();
 }

 void buildSA() {
 // id[i]：以 i 开头长度为 k 的子串的排名
 // tmp[i]：临时排名（对新第二关键字排序后）
 vector<int> id(n), tmp(n);

 // ----- 第 0 轮：按单个字符排序 -----
 // 字符集大小：256（可放宽到 max(n, 256)）
 const int sigma = max(n, 256);
 vector<int> cnt(sigma);

 // 第一轮：只按首字符排序
 for (int i = 0; i < n; i++) {
 rk[i] = (unsigned char)s[i];
 cnt[rk[i]]++;
 }
 for (int i = 1; i < sigma; i++) cnt[i] += cnt[i - 1];
 for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;

 // ----- 倍增：k = 1, 2, 4, 8, ... -----
 for (int k = 1, p = 0; p < n; k <= 1) {
 // 按第二关键字排序（第二关键字 = sa[i] + k 的排名）
 // 如果第二关键字越界（即后缀长度不足 k），排名视为 -1（最小）
 p = 0;
 for (int i = n - k; i < n; i++) id[p++] = i; // 第二关键字为空，最小
 for (int i = 0; i < n; i++) {
 if (sa[i] >= k) id[p++] = sa[i] - k;
 }

 // 按第一关键字（rk）进行计数排序
 fill(cnt.begin(), cnt.begin() + p, 0);
 // 当 p < sigma 时只需要清空用到的部分
 // 安全起见用 fill 到 p
 for (int i = 0; i < n; i++) cnt[rk[i]]++;
 for (int i = 1; i < sigma; i++) cnt[i] += cnt[i - 1];
 for (int i = n - 1; i >= 0; i--) {
 sa[--cnt[rk[id[i]]]] = id[i];
 }

 // 重新计算排名
 tmp[sa[0]] = 0;
 p = 0; // p 记录不同排名的数量
 for (int i = 1; i < n; i++) {
 // 判断两个后缀的前 2k 长度的子串是否相等
 int x = sa[i], y = sa[i - 1];
 int rkx = rk[x], rkny = (x + k < n) ? rk[x + k] : -1;
 int rky = rk[y], rkny = (y + k < n) ? rk[y + k] : -1;
 if (rkx != rky || rkny != rkny) p++;
 tmp[x] = p;
 }
 rk.swap(tmp);
 if (p == n - 1) break; // 所有排名都已区分，提前结束
 }
 }

 // Kasai 算法：O(n) 求 height / LCP 数组
 void buildHeight() {
 // 先用 sa 构造 rk（如果 buildSA 中没有维护 rk）
 // 这里 buildSA 中已维护了最终的 rk
 int k = 0;
 for (int i = 0; i < n; i++) {
 if (rk[i] == 0) {
 height[0] = 0;
 k = 0;
 continue;
 }
 // height[rk[i]] = LCP(i, sa[rk[i]-1])
 // 利用性质：height[rk[i]] >= height[rk[i-1]] - 1
 if (k > 0) k--;
 int j = sa[rk[i] - 1]; // 排名前一位的后缀起始位置
 while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
 height[rk[i]] = k;
 }
 }

 // 判断子串 s[l..r] 在 s 中出现的次数 O(log n)
 int countSubstring(int l, int r) const {
 int len = r - l + 1;
 // 二分离散排名下界
 int lo = 0, hi = n;
 while (lo < hi) {
 int mid = (lo + hi) / 2;
 if (s.compare(sa[mid], len, s, l, len) < 0) lo = mid + 1;
 else hi = mid;
 }
 int lb = lo;
 // 二分离散排名上界
 lo = 0, hi = n;
 while (lo < hi) {
 int mid = (lo + hi) / 2;
 if (s.compare(sa[mid], len, s, l, len) <= 0) lo = mid + 1;
 else hi = mid;
 }
 return lo - lb;
 }
};

```

### 5.5.2 LCP 查询 (利用 height 数组 + RMQ)

在求出 height 数组后, 可以通过 RMQ (Sparse Table) 实现  $O(1)O(1)$  查询任意两个后缀的 LCP:

```
// Sparse Table for LCP queries on suffix array
struct LCPQuery {
 int n;
 vector<int> log2;
 vector<vector<int>> st; // st[k][i] = min(height[i..i+2^k-1])

 LCPQuery(const vector<int> &height) : n((int)height.size()) {
 log2.resize(n + 1);
 log2[1] = 0;
 for (int i = 2; i <= n; i++) log2[i] = log2[i / 2] + 1;

 int K = log2[n] + 1;
 st.assign(K, vector<int>(n));
 st[0] = height;

 for (int k = 1; k < K; k++) {
 for (int i = 0; i + (1 << k) <= n; i++) {
 st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
 }
 }
 }

 // 查询后缀 i 和后缀 j 的 LCP (i, j 是后缀起始下标)
 int query(int i, int j, const vector<int> &rk) {
 if (i == j) return n - i;
 int ri = rk[i], rj = rk[j];
 if (ri > rj) swap(ri, rj);
 // ri < rj, 查询 height[ri+1 .. rj] 的最小值
 ri++; // height 从 height[ri+1] 开始
 int len = rj - ri + 1;
 int k = log2[len];
 return min(st[k][ri], st[k][rj - (1 << k) + 1]);
 }
};
```

### 5.5.3 应用

```
// 最长公共子串 (Longest Common Substring)
// 求两个串 a 和 b 的最长公共子串
string longestCommonSubstring(const string &a, const string &b) {
 string s = a + "#" + b; // 用分隔符拼接
 int n1 = (int)a.size(), n2 = (int)b.size();
 SuffixArray sa(s);

 int maxLen = 0, startPos = 0;
 for (int i = 1; i < sa.n; i++) {
 int x = sa.sa[i - 1], y = sa.sa[i];
 // 确保两个后缀分别来自 a 和 b
 if ((x < n1) != (y < n1)) {
 if (sa.height[i] > maxLen) {
 maxLen = sa.height[i];
 startPos = min(x, y);
 }
 }
 }
 return s.substr(startPos, maxLen);
}

// 不同子串数量
// 所有后缀的前缀去掉重复的 LCP 部分
long long distinctSubstrings(const string &s) {
 SuffixArray sa(s);
 long long n = s.size();
 long long total = n * (n + 1) / 2; // 所有子串数
 for (int i = 1; i < n; i++) {
 total -= sa.height[i]; // 减去重复的 LCP
 }
 return total;
}
```

## 5.6 字符串哈希 (Rolling Hash / String Hashing)

### 核心思想

字符串哈希将任意字符串映射为整数, 支持  $O(1)O(1)$  查询任意子串的哈希值 (滚动哈希)。常用于字符串匹配、最长公共前缀 (二分 + 哈希)、回文判断等。

**CRITICAL:** 单模数哈希一定会被卡 (生日悖论 + 针对性数据), 必须使用双哈希或 64 位无符号溢出哈希。

时间复杂度:

### 5.6.1 单哈希（64 位自然溢出）

```
// 单哈希：使用 unsigned long long 自然溢出
// 注意：可能被 Anti-Hash 测试卡掉
struct SingleHash {
 using ULL = unsigned long long;
 static const ULL BASE = 131;

 int n;
 vector<ULL> h; // 前缀哈希
 vector<ULL> p; // $p[i] = \text{BASE}^i$

 SingleHash(const string &s) : n((int)s.size()) {
 h.resize(n + 1);
 p.resize(n + 1);
 p[0] = 1;
 for (int i = 0; i < n; i++) {
 h[i + 1] = h[i] * BASE + (ULL)(s[i]);
 p[i + 1] = p[i] * BASE;
 }
 }

 // 获取子串 $s[l..r]$ (闭区间) 的哈希值, θ -indexed
 ULL get(int l, int r) const {
 return h[r + 1] - h[l] * p[r - l + 1];
 }
};
```

### 5.6.2 双哈希（推荐）

```
// 双哈希：使用两个大质数模数, 安全可靠（推荐）
struct DoubleHash {
 using ULL = unsigned long long;
 static const int MOD1 = 1000000007; // $1e9 + 7$
 static const int MOD2 = 1000000009; // $1e9 + 9$
 static const int BASE = 131; // 基数

 int n;
 vector<int> h1, h2; // 前缀哈希
 vector<int> p1, p2; // $\text{BASE}^i \bmod \text{MOD}$

 DoubleHash(const string &s) : n((int)s.size()) {
 h1.resize(n + 1);
 h2.resize(n + 1);
 p1.resize(n + 1);
 p2.resize(n + 1);
 p1[0] = p2[0] = 1;
 for (int i = 0; i < n; i++) {
 h1[i + 1] = ((ULL)h1[i] * BASE + (ULL)(s[i])) % MOD1;
 h2[i + 1] = ((ULL)h2[i] * BASE + (ULL)(s[i])) % MOD2;
 p1[i + 1] = (ULL)p1[i] * BASE % MOD1;
 p2[i + 1] = (ULL)p2[i] * BASE % MOD2;
 }
 }

 // 获取子串哈希值, 返回 pair 作为组合哈希
 pair<int, int> get(int l, int r) const {
 int v1 = (h1[r + 1] - (ULL)h1[l] * p1[r - l + 1] % MOD1 + MOD1) % MOD1;
 int v2 = (h2[r + 1] - (ULL)h2[l] * p2[r - l + 1] % MOD2 + MOD2) % MOD2;
 return {v1, v2};
 }

 // 将哈希对编码为 64 位整数, 方便存入 set/map
 ULL encode(pair<int, int> hv) const {
 return ((ULL)hv.first << 32) | (ULL)hv.second;
 }
};
```

### 5.6.3 应用

```
// 使用 DoubleHash 判断两个子串是否相等 $O(1)$
bool equal(const DoubleHash &dh, int l1, int r1, int l2, int r2) {
 return dh.get(l1, r1) == dh.get(l2, r2);
}

// 二分求两个后缀的 LCP 长度 $O(\log n)$
int lcp(const DoubleHash &dh, int i, int j) {
 int lo = 0, hi = dh.n - max(i, j);
 while (lo < hi) {
 int mid = (lo + hi + 1) / 2;
 if (dh.get(i, i + mid - 1) == dh.get(j, j + mid - 1))
 lo = mid;
 else
 hi = mid - 1;
 }
 return lo;
}

// 判断子串是否为回文（需要正反两个哈希）
struct PalindromeHash {
 DoubleHash forwardHash;
 DoubleHash reverseHash;

 PalindromeHash(const string &s)
 : forwardHash(s), reverseHash(string(s.rbegin(), s.rend())) {}

 bool isPalindrome(int l, int r) const {
 int n = forwardHash.n;
 // 正串 $[l, r]$ vs 反串 $[n-1-r, n-1-l]$
 return forwardHash.get(l, r) == reverseHash.get(n - 1 - r, n - 1 - l);
 }
};
```

## 5.7 最小表示法 (Minimal Rotation / Booth's Algorithm)

### 核心理想

最小表示法找出字符串所有循环同构串中字典序最小的那个，返回其起始位置。

**Booth's Algorithm** 可以在  $O(n)O(n)$  时间、 $O(1)O(1)$  额外空间内完成。算法的关键思想是维护两个候选起始位置 `i` 和 `j`，以及当前比较的偏移量 `k`：每次跳转保证 `i` 和 `j` 之间至少相差 1，且跳过的位置都被证明不可能为最优解。

**时间复杂度：** $O(n)O(n)$ 。

**空间复杂度：** $O(1)O(1)$ （仅需几个变量，不依赖输入长度）。

### 实现

```
// Booth 算法：求最小循环同构串的起始位置
// 返回字典序最小的循环同构串在原串中的起始下标 (0-indexed)
// 将原串 s 复制一份 s + s 亦可实现，但 Booth 算法更优雅且 O(1) 额外空间
int minimalRotation(const string &s) {
 int n = (int)s.size();
 // i 和 j 是两个候选起始位置，k 是当前比较的偏移
 int i = 0, j = 1, k = 0;
 // 将 s 视为循环串，使用模运算访问
 while (i < n && j < n && k < n) {
 // 比较 s[(i+k) % n] 与 s[(j+k) % n]
 char a = s[(i + k) % n];
 char b = s[(j + k) % n];
 if (a == b) {
 k++;
 } else {
 // 字符不相等：淘汰字典序较大的一方
 if (a > b) {
 // i 开头的串更大，跳过 i..i+k 这些位置
 i = i + k + 1;
 } else {
 // j 开头的串更大，跳过 j..j+k 这些位置
 j = j + k + 1;
 }
 // 确保 i != j
 if (i == j) {
 j++;
 }
 k = 0; // 重置偏移量
 }
 }
 return min(i, j);
}

// 获取最小循环同构串
string getMinimalRotation(const string &s) {
 int pos = minimalRotation(s);
 int n = (int)s.size();
 string t = s + s;
 return t.substr(pos, n);
}

// 最大表示法：求字典序最大的循环同构串
// 只需将比较符号取反即可
int maximalRotation(const string &s) {
 int n = (int)s.size();
 int i = 0, j = 1, k = 0;
 while (i < n && j < n && k < n) {
 char a = s[(i + k) % n];
 char b = s[(j + k) % n];
 if (a == b) {
 k++;
 } else {
 // 取反：淘汰较小的
 if (a < b) {
 i = i + k + 1;
 } else {
 j = j + k + 1;
 }
 }
 if (i == j) j++;
 k = 0;
 }
 return min(i, j);
}
```

### 应用

```
// 判断两个串是否互为循环同构 O(n)
bool isCyclicEqual(const string &a, const string &b) {
 if (a.size() != b.size()) return false;
 return getMinimalRotation(a) == getMinimalRotation(b);
}

// 对一组串按最小表示法去重（循环同构视为相同）
vector<string> dedupCyclic(const vector<string> &words) {
 set<string> seen;
 vector<string> result;
 for (const string &w : words) {
 string mr = getMinimalRotation(w);
 if (!seen.count(mr)) {
 seen.insert(mr);
 result.push_back(w);
 }
 }
 return result;
}
```

5.8 字符串算法复杂度总结

算法	预处理	查询/匹配	空间	用途
KMP	$O(m)O(m)$	$O(n)O(n)$	$O(m)O(m)$	单模式匹配
Z-Algorithm	$O(n)O(n)$	$O(n)O(n)$	$O(n)O(n)$	前缀与后缀 LCP
Manacher	$O(n)O(n)$	$O(1)O(1)$	$O(n)O(n)$	最长回文子串
AC 自动机	$O(L \cdot  \Sigma )O(L \cdot  \Sigma  \cdot \text{vert})$	$O(n + \text{matches})O(n + \text{matches})$	$O(L \cdot  \Sigma )O(L \cdot  \Sigma  \cdot \text{vert})$	多模式匹配
后缀数组	$O(n \log n)O(n \log n)$	$O(\log n)O(\log n)$	$O(n)O(n)$	子串查询、LCP
字符串哈希	$O(n)O(n)$	$O(1)O(1)$ / 子串	$O(n)O(n)$	子串判等、二分 LCP
最小表示法	$O(n)O(n)$	-	$O(1)O(1)$	循环同构判等

6. 动态规划 (Dynamic Programming)

6.1 背包问题 (Knapsack Problems)

6.1.1 01 背包 — 每个物品最多选一次

核心思想：dp[j] 表示容量为 j 时能获得的最大价值。内层循环从大到小遍历，保证每个物品只被考虑一次。

```
// 01 背包：N 个物品，容量 C
// w[i] = 重量, v[i] = 价值
// 时间复杂度 O(N*C), 空间复杂度 O(C)

const int INF = -1e9;

int zero_one_knapsack(const vector<int>& w, const vector<int>& v, int C) {
 int N = w.size();
 vector<int> dp(C + 1, 0); // 求最大值，初始化为 0
 for (int i = 0; i < N; i++) {
 for (int j = C; j >= w[i]; j--) { // 从大到小
 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
 }
 }
 return dp[C];
}

// 变体：恰好装满（求价值最大）
int zero_one_knapsack_exact(const vector<int>& w, const vector<int>& v, int C) {
 vector<int> dp(C + 1, INF);
 dp[0] = 0;
 for (int i = 0; i < w.size(); i++) {
 for (int j = C; j >= w[i]; j--) {
 if (dp[j - w[i]] != INF)
 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
 }
 }
 return dp[C] == INF ? -1 : dp[C];
}

// 变体：求方案数
int zero_one_ways(const vector<int>& w, int C) {
 vector<int> dp(C + 1, 0);
 dp[0] = 1;
 for (int x : w) {
 for (int j = C; j >= x; j--) {
 dp[j] = (dp[j] + dp[j - x]) % MOD;
 }
 }
 return dp[C];
}
```

关键点：内层循环 j 从大到小保证了 dp[j - w[i]] 来自上一层（即未选物品 i 的状态），实现 01 约束。

6.1.2 完全背包 — 每个物品可以选无限次

核心思想：内层循环从小到大遍历，允许同一物品被多次选择。

```
// 完全背包：N 个物品，容量 C
// 每个物品可以选无限次
int unbounded_knapsack(const vector<int>& w, const vector<int>& v, int C) {
 int N = w.size();
 vector<int> dp(C + 1, 0);
 for (int i = 0; i < N; i++) {
 for (int j = w[i]; j <= C; j++) { // 从小到大
 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
 }
 }
 return dp[C];
}

// 完全背包：方案数（无序，即组合数）
int unbounded_ways(const vector<int>& w, int C) {
 vector<int> dp(C + 1, 0);
 dp[0] = 1;
 for (int x : w) {
 for (int j = x; j <= C; j++) {
 dp[j] = (dp[j] + dp[j - x]) % MOD;
 }
 }
 return dp[C];
}

// 完全背包：恰好装满的最小物品数
int unbounded_min_items(const vector<int>& w, int C) {
 vector<int> dp(C + 1, 1e9);
 dp[0] = 0;
 for (int x : w) {
 for (int j = x; j <= C; j++) {
 dp[j] = min(dp[j], dp[j - x] + 1);
 }
 }
 return dp[C] == 1e9 ? -1 : dp[C];
}
```

对比记忆：

背包类型	内层循环方向	原因
01 背包	j 从大到小	每个物品只用一次，dp[j-w[i]] 不能包含当前物品
完全背包	j 从小到大	每个物品可用多次，dp[j-w[i]] 可以包含当前物品

### 6.1.3 多重背包 — 每个物品最多选 cnt[i] 次

二进制拆分优化：将 cnt[i] 拆成 1, 2, 4, ..., 2<sup>k</sup>, r 这 O(log cnt) 个物品，转化为 01 背包。

```
// 多重背包：N 种物品，每种有 cnt[i] 个
// 二进制拆分转化为 01 背包
// 时间复杂度 O(C * Σ log cnt[i])

int bounded_knapsack(const vector<int>& w, const vector<int>& v,
 const vector<int>& cnt, int C) {
 int N = w.size();
 vector<int> dp(C + 1, 0);

 for (int i = 0; i < N; i++) {
 int k = 1;
 int remain = cnt[i];
 while (remain >= k) {
 // 打包 k 个物品 i 作为新物品
 int pack_w = w[i] * k;
 int pack_v = v[i] * k;
 for (int j = C; j >= pack_w; j--) {
 dp[j] = max(dp[j], dp[j - pack_w] + pack_v);
 }
 remain -= k;
 k <<= 1;
 }
 // 处理余数 r
 if (remain > 0) {
 int pack_w = w[i] * remain;
 int pack_v = v[i] * remain;
 for (int j = C; j >= pack_w; j--) {
 dp[j] = max(dp[j], dp[j - pack_w] + pack_v);
 }
 }
 }
 return dp[C];
}

// 单调队列优化多重背包 (O(N*C))
// 对同类做滑动窗口最大值
int bounded_knapsack_fast(const vector<int>& w, const vector<int>& v,
 const vector<int>& cnt, int C) {
 vector<int> dp(C + 1, 0);
 for (int i = 0; i < w.size(); i++) {
 vector<int> ndp(dp);
 for (int mod = 0; mod < w[i]; mod++) {
 deque<pair<int, int>> dq; // (index, value)
 for (int j = mod; j <= C; j += w[i]) {
 // 价值 = dp[j] - (j/w[i]) * v[i]
 int val = dp[j] - (j / w[i]) * v[i];
 // 弹出过期的（超过 cnt[i] 个）
 while (!dq.empty() && dq.front().first < j - cnt[i] * w[i])
 dq.pop_front();
 // 维护单调递减
 while (!dq.empty() && dq.back().second <= val)
 dq.pop_back();
 dq.push_back({j, val});
 ndp[j] = max(ndp[j], dq.front().second + (j / w[i]) * v[i]);
 }
 }
 dp.swap(ndp);
 }
 return dp[C];
}
```

## 6.2 混合背包

将前三种背包合并，根据物品类型选择处理方式。

```
struct Item {
 int w, v, cnt; // cnt=-1:01, cnt=0:完全, cnt>0:多重
};

int mixed_knapsack(const vector<Item>& items, int C) {
 vector<int> dp(C + 1, 0);
 for (auto& item : items) {
 if (item.cnt == -1) {
 // 01 背包
 for (int j = C; j >= item.w; j--)
 dp[j] = max(dp[j], dp[j - item.w] + item.v);
 } else if (item.cnt == 0) {
 // 完全背包
 for (int j = item.w; j <= C; j++)
 dp[j] = max(dp[j], dp[j - item.w] + item.v);
 } else {
 // 多重背包 - 二进制拆分
 int k = 1, remain = item.cnt;
 while (remain >= k) {
 int pw = item.w * k, pv = item.v * k;
 for (int j = C; j >= pw; j--)
 dp[j] = max(dp[j], dp[j - pw] + pv);
 remain -= k; k <= 1;
 }
 if (remain > 0) {
 int pw = item.w * remain, pv = item.v * remain;
 for (int j = C; j >= pw; j--)
 dp[j] = max(dp[j], dp[j - pw] + pv);
 }
 }
 }
 return dp[C];
}
```

## 6.3 二维费用背包

```
// 每个物品同时消耗两种资源（如重量 w 和体积 b ），价值为 v
int two_dim_knapsack(const vector<int>& w, const vector<int>& b,
 const vector<int>& v, int C, int D) {
 vector<vector<int>>> dp(C + 1, vector<int>(D + 1, 0));
 for (int i = 0; i < w.size(); i++) {
 for (int j = C; j >= w[i]; j--) {
 for (int kk = D; kk >= b[i]; kk--) {
 dp[j][kk] = max(dp[j][kk], dp[j - w[i]][kk - b[i]] + v[i]);
 }
 }
 }
 return dp[C][D];
}
```

## 6.4 最长上升子序列 (LIS) — $O(N \log N)$

核心：维护 `tails` 数组，`tails[i]` 表示长度为 `i+1` 的上升子序列的最小末尾值。

```

// 严格递增 LIS - lower_bound
int lis_strict(const vector<int>& a) {
 vector<int> tails;
 for (int x : a) {
 auto it = lower_bound(tails.begin(), tails.end(), x);
 if (it == tails.end())
 tails.push_back(x);
 else
 *it = x;
 }
 return tails.size();
}

// 非递减 (最长不下降子序列) - upper_bound
int lis_non_decreasing(const vector<int>& a) {
 vector<int> tails;
 for (int x : a) {
 auto it = upper_bound(tails.begin(), tails.end(), x);
 if (it == tails.end())
 tails.push_back(x);
 else
 *it = x;
 }
 return tails.size();
}

// LIS 并还原方案
vector<int> lis_with_path(const vector<int>& a) {
 int n = a.size();
 vector<int> tails, pos(n), prev(n, -1);
 for (int i = 0; i < n; i++) {
 auto it = lower_bound(tails.begin(), tails.end(), a[i]);
 int idx = it - tails.begin();
 if (it == tails.end())
 tails.push_back(a[i]);
 else
 *it = a[i];
 pos[idx] = i;
 if (idx > 0) prev[i] = pos[idx - 1];
 }
 // 还原
 vector<int> ans;
 for (int p = pos[tails.size() - 1]; p != -1; p = prev[p])
 ans.push_back(a[p]);
 reverse(ans.begin(), ans.end());
 return ans;
}

```

LIS 扩展:

```

// 二维 LIS: 按 w 升序, w 相同时 h 降序, 再对 h 求 LIS
// 用途: 俄罗斯套娃信封问题 (LC 354)
int max_envelopes(vector<pair<int,int>>& e) {
 // w 升序, w 相同时 h 降序 (避免同 w 被选中多次)
 sort(e.begin(), e.end(), [](auto& a, auto& b) {
 return a.first < b.first || (a.first == b.first && a.second > b.second);
 });
 vector<int> tails;
 for (auto& p : e) {
 auto it = lower_bound(tails.begin(), tails.end(), p.second);
 if (it == tails.end()) tails.push_back(p.second);
 else *it = p.second;
 }
 return tails.size();
}

```

## 6.5 最长公共子序列 (LCS)

### 6.5.1 标准 $O(N \cdot M)$ DP

```

// 时间 $O(N \cdot M)$, 空间 $O(N \cdot M)$
int lcs(const string& a, const string& b) {
 int n = a.size(), m = b.size();
 vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
 for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= m; j++) {
 if (a[i - 1] == b[j - 1])
 dp[i][j] = dp[i - 1][j - 1] + 1;
 else
 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
 }
 }
 return dp[n][m];
}

```



### 6.5.2 滚动数组优化到 $O(\min(N,M))$

```
// 空间 $O(\min(n, m))$
int lcs_optimized(const string& a, const string& b) {
 // 保证 a 是较短的串, 节省空间
 if (a.size() > b.size()) return lcs_optimized(b, a);
 int n = a.size(), m = b.size();
 vector<int> dp(m + 1, 0);
 for (int i = 1; i <= n; i++) {
 int pre = 0; // dp[i-1][j-1]
 for (int j = 1; j <= m; j++) {
 int tmp = dp[j];
 if (a[i - 1] == b[j - 1])
 dp[j] = pre + 1;
 else
 dp[j] = max(dp[j], dp[j - 1]);
 pre = tmp;
 }
 }
 return dp[m];
}
```

### 6.5.3 LCS 还原方案

```
string lcs_reconstruct(const string& a, const string& b) {
 int n = a.size(), m = b.size();
 vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
 for (int i = 1; i <= n; i++)
 for (int j = 1; j <= m; j++)
 if (a[i - 1] == b[j - 1])
 dp[i][j] = dp[i - 1][j - 1] + 1;
 else
 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);

 string ans;
 int i = n, j = m;
 while (i > 0 && j > 0) {
 if (a[i - 1] == b[j - 1]) {
 ans.push_back(a[i - 1]);
 i--; j--;
 } else if (dp[i - 1][j] > dp[i][j - 1]) {
 i--;
 } else {
 j--;
 }
 }
 reverse(ans.begin(), ans.end());
 return ans;
}
```

### 6.5.4 LCS 转 LIS (排列情况)

当其中一个序列是排列 (无重复元素) 时, 可  $O(N \log N)$ :

```
// a 是排列 [1..n], b 是任意序列
int lcs_permutation(const vector<int>& a, const vector<int>& b) {
 int n = a.size();
 unordered_map<int, int> pos;
 for (int i = 0; i < n; i++) pos[a[i]] = i;
 vector<int> seq;
 for (int x : b) {
 if (pos.count(x)) seq.push_back(pos[x]);
 }
 return lis_strict(seq); // LIS of positions
}
```

## 6.6 编辑距离 (Edit Distance)

```
int edit_distance(const string& s, const string& t) {
 int n = s.size(), m = t.size();
 // dp[i][j] = min cost to convert s[0..i-1] to t[0..j-1]
 vector<vector<int>> dp(n + 1, vector<int>(m + 1, 1e9));
 for (int i = 0; i <= n; i++) dp[i][0] = i; // 删除 i 个
 for (int j = 0; j <= m; j++) dp[0][j] = j; // 插入 j 个
 for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= m; j++) {
 if (s[i - 1] == t[j - 1])
 dp[i][j] = dp[i - 1][j - 1]; // 匹配
 else
 dp[i][j] = min({
 dp[i - 1][j] + 1, // 删除 s[i-1]
 dp[i][j - 1] + 1, // 插入 t[j-1]
 dp[i - 1][j - 1] + 1 // 替换
 });
 }
 }
 return dp[n][m];
}
```

## 6.7 区间 DP (Interval DP)

通用模板: 从小到大枚举区间长度 `len`, 再枚举左端点 `l`, 右端点 `r = l + len - 1`, 枚举分割点 `k`。

```

// 区间 DP 通用模板
// dp[l][r] 表示区间 [l, r] 上的最优解

// 例 1: 石子合并 (相邻合并, 代价为区间和)
int stone_merge(const vector<int>& a) {
 int n = a.size();
 vector<long long> pref(n + 1, 0);
 for (int i = 1; i <= n; i++) pref[i] = pref[i - 1] + a[i - 1];

 vector<vector<long long>> dp(n, vector<long long>(n, 1e18));
 for (int i = 0; i < n; i++) dp[i][i] = 0;
 for (int len = 2; len <= n; len++) {
 for (int l = 0; l + len - 1 < n; l++) {
 int r = l + len - 1;
 for (int k = l; k < r; k++) {
 dp[l][r] = min(dp[l][r],
 dp[l][k] + dp[k + 1][r] + pref[r + 1] - pref[l]);
 }
 }
 }
 return dp[0][n - 1];
}

// 例 2: 环形石子合并 (断环成链)
int stone_merge_circular(const vector<int>& a) {
 int n = a.size();
 // 将数组拼接成 2n 长度
 vector<int> b(2 * n);
 for (int i = 0; i < 2 * n; i++) b[i] = a[i % n];

 vector<long long> pref(2 * n + 1, 0);
 for (int i = 1; i <= 2 * n; i++) pref[i] = pref[i - 1] + b[i - 1];

 vector<vector<long long>> dp(2 * n, vector<long long>(2 * n, 1e18));
 for (int i = 0; i < 2 * n; i++) dp[i][i] = 0;

 for (int len = 2; len <= n; len++) {
 for (int l = 0; l + len - 1 < 2 * n; l++) {
 int r = l + len - 1;
 for (int k = l; k < r; k++) {
 dp[l][r] = min(dp[l][r],
 dp[l][k] + dp[k + 1][r] + pref[r + 1] - pref[l]);
 }
 }
 }

 long long ans = 1e18;
 for (int l = 0; l + n - 1 < 2 * n; l++)
 ans = min(ans, dp[l][l + n - 1]);
 return ans;
}

// 例 3: 最长回文子序列
int longest_palindrome_subseq(const string& s) {
 int n = s.size();
 vector<vector<int>> dp(n, vector<int>(n, 0));
 for (int i = 0; i < n; i++) dp[i][i] = 1;
 for (int len = 2; len <= n; len++) {
 for (int l = 0; l + len - 1 < n; l++) {
 int r = l + len - 1;
 if (s[l] == s[r])
 dp[l][r] = dp[l + 1][r - 1] + 2;
 else
 dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
 }
 }
 return dp[0][n - 1];
}

// 例 4: 矩阵链乘法
int matrix_chain(const vector<int>& dims) {
 int n = dims.size() - 1; // n 个矩阵
 vector<vector<int>> dp(n, vector<int>(n, 1e9));
 for (int i = 0; i < n; i++) dp[i][i] = 0;
 for (int len = 2; len <= n; len++) {
 for (int l = 0; l + len - 1 < n; l++) {
 int r = l + len - 1;
 for (int k = l; k < r; k++) {
 dp[l][r] = min(dp[l][r],
 dp[l][k] + dp[k + 1][r] + dims[l] * dims[k + 1] * dims[r + 1]);
 }
 }
 }
 return dp[0][n - 1];
}

```

## 6.8 树形 DP (Tree DP)

### 6.8.1 基础树形 DP

```
// 例 1：树上最大独立集（没有上司的舞会）
// dp[u][0] = 以 u 为根的子树，不选 u 的最大值
// dp[u][1] = 以 u 为根的子树，选择 u 的最大值

vector<vector<int>> g;
vector<vector<long long>> dp;

void dfs_tree(int u, int p) {
 dp[u][0] = 0;
 dp[u][1] = val[u]; // val[u] 是节点 u 的权值
 for (int v : g[u]) {
 if (v == p) continue;
 dfs_tree(v, u);
 dp[u][0] += max(dp[v][0], dp[v][1]);
 dp[u][1] += dp[v][0];
 }
}

// 例 2：树的直径（两次 DFS / 树形 DP）
// 方法一：两次 DFS（边权非负）
pair<int, int> furthest(const vector<vector<pair<int, int>>>& g, int s) {
 int n = g.size();
 vector<int> dist(n, -1);
 dfs_dist(g, s, -1, 0, dist);
 int far = max_element(dist.begin(), dist.end()) - dist.begin();
 return {far, dist[far]};
}

int tree_diameter_two_dfs(const vector<vector<pair<int, int>>>& g) {
 auto [a, _1] = furthest(g, 0);
 auto [_2, ans] = furthest(g, a);
 return ans;
}

// 方法二：树形 DP（支持负权）
vector<int> dp1, dp2; // dp1: 向下最长, dp2: 向下次长
int diameter = 0;

void dfs_diameter(int u, int p) {
 for (auto [v, w] : g[u]) {
 if (v == p) continue;
 dfs_diameter(v, u);
 int cand = dp1[v] + w;
 if (cand > dp1[u]) {
 dp2[u] = dp1[u];
 dp1[u] = cand;
 } else if (cand > dp2[u]) {
 dp2[u] = cand;
 }
 }
 diameter = max(diameter, dp1[u] + dp2[u]);
}
```

### 6.8.2 换根 DP (Rerooting DP)

**两次 DFS 模式：**第一次任选根求出向下信息，第二次将父节点贡献当子树接入子节点。

```

// 例：求树上每个点到其他所有点的距离之和
// dp_down[u] = u 子树内所有点到 u 的距离和
// sz[u] = u 的子树大小
// ans[u] = u 到所有点的距离和

int n;
vector<vector<int>> g;
vector<long long> dp_down, ans;
vector<int> sz;

void dfs1(int u, int p) {
 sz[u] = 1;
 dp_down[u] = 0;
 for (int v : g[u]) {
 if (v == p) continue;
 dfs1(v, u);
 sz[u] += sz[v];
 dp_down[u] += dp_down[v] + sz[v]; // 每条边被 v 子树各点经过一次
 }
}

void dfs2(int u, int p) {
 ans[u] = dp_down[u]; // 此时 dp_down[u] 已是完整答案
 for (int v : g[u]) {
 if (v == p) continue;

 // 保存旧值
 long long old_u = dp_down[u], old_v = dp_down[v];
 int sz_u = sz[u], sz_v = sz[v];

 // 换根：去掉 v 的贡献，把 u 当子树接到 v 上
 dp_down[u] -= (dp_down[v] + sz[v]);
 sz[u] -= sz[v];

 dp_down[v] += (dp_down[u] + sz[u]);
 sz[v] += sz[u];

 dfs2(v, u);

 // 恢复
 dp_down[u] = old_u;
 dp_down[v] = old_v;
 sz[u] = sz_u;
 sz[v] = sz_v;
 }
}

void solve_rerooting() {
 sz.assign(n, 0);
 dp_down.assign(n, 0);
 ans.assign(n, 0);
 dfs1(0, -1);
 dfs2(0, -1);
 // ans[u] 即为 u 到所有点的距离和
}

// 换根 DP 通用模板（合并函数版本）
struct Info {
 long long val;
 Info() : val(0) {}
 Info(long long v) : val(v) {}
 // 合并子节点信息
 static Info merge(const Info& a, const Info& b) {
 return Info(a.val + b.val);
 }
} // 提升：加上到父节点的边权
Info lift(int w) const { return Info(val + w); }
};

vector<Info> dp_down, dp_all;

void reroot_dfs1(int u, int p) {
 dp_down[u] = Info(val[u]);
 for (auto [v, w] : g[u]) {
 if (v == p) continue;
 reroot_dfs1(v, u);
 dp_down[u] = Info::merge(dp_down[u], dp_down[v].lift(w));
 }
}

void reroot_dfs2(int u, int p) {
 dp_all[u] = dp_down[u];
 // 收集子节点信息用于快速排除
 int m = g[u].size();
 vector<Info> pref(m + 1), suff(m + 1);
 for (int i = 0; i < m; i++) {
 auto [v, w] = g[u][i];
 pref[i + 1] = Info::merge(pref[i],
 v == p ? Info() : dp_down[v].lift(w));
 }
 for (int i = m - 1; i >= 0; i--) {
 auto [v, w] = g[u][i];
 suff[i] = Info::merge(
 v == p ? Info() : dp_down[v].lift(w), suff[i + 1]);
 }

 for (int i = 0; i < m; i++) {
 auto [v, w] = g[u][i];
 if (v == p) continue;
 // 临时将 u 换为 v 的子节点
 Info old_u = dp_down[u], old_v = dp_down[v];
 dp_down[u] = Info::merge(Info::merge(pref[i], suff[i + 1]), Info(val[u]));
 dp_down[v] = Info::merge(old_v, dp_down[u].lift(w));
 reroot_dfs2(v, u);
 dp_down[u] = old_u;
 dp_down[v] = old_v;
 }
}
}

```

## 6.9 背包问题在树上

```
// 树形依赖背包：选子节点必须先选父节点
// dp[u][j] = 以 u 为根的子树，选恰好 j 体积的最大价值 (u 必须被选)

vector<vector<long long>> dp;

void dfs_tree_knapsack(int u, int p, int C) {
 // 初始化：必须选 u
 dp[u].resize(C + 1, -1e18);
 for (int j = weight[u]; j <= C; j++)
 dp[u][j] = val[u]; // 放置 u 自身的贡献
 dp[u][0] = 0; // 不选 u 时不能占用体积

 for (int v : g[u]) {
 if (v == p) continue;
 dfs_tree_knapsack(v, u, C);
 // 合并子节点 (分组背包)
 for (int j = C; j >= 0; j--) {
 for (int kk = 0; kk <= j; kk++) {
 if (dp[u][j - kk] != -1e18 && dp[v][kk] != -1e18)
 dp[u][j] = max(dp[u][j], dp[u][j - kk] + dp[v][kk]);
 }
 }
 }
}
```

## 6.10 状压 DP (Bitmask DP)

### 6.10.1 基础枚举子集

```
// 枚举 mask 的所有非空子集
for (int sub = mask; sub; sub = (sub - 1) & mask) {
 // sub 是 mask 的子集
}

// 枚举 mask 的所有子集 (含空集)
int sub = mask;
do {
 // process sub
 sub = (sub - 1) & mask;
} while (sub != mask); // 会包含空集, 最后 sub = mask 时跳出

// 枚举所有超集
for (int sup = mask; sup < (1 << n); sup = (sup + 1) | mask) {
 // sup 是 mask 的超集
}
```

### 6.10.2 TSP (旅行商问题)

```
// TSP: dp[mask][i] = 当前在 i, 已经访问过 mask 中的城市, 的最短路径
// 时间复杂度 $O(N^2 * 2^N)$

const int INF = 1e9;
int tsp(const vector<vector<int>>& dist) {
 int n = dist.size();
 vector<vector<int>> dp(1 << n, vector<int>(n, INF));
 dp[1][0] = 0; // 从城市 0 出发

 for (int mask = 1; mask < (1 << n); mask++) {
 for (int i = 0; i < n; i++) {
 if (!(mask >> i & 1)) continue;
 if (dp[mask][i] == INF) continue;
 for (int j = 0; j < n; j++) {
 if (mask >> j & 1) continue;
 dp[mask | (1 << j)][j] = min(
 dp[mask | (1 << j)][j],
 dp[mask][i] + dist[i][j]);
 }
 }
 }

 int ans = INF;
 for (int i = 0; i < n; i++)
 ans = min(ans, dp[(1 << n) - 1][i] + dist[i][0]); // 回到起点
 return ans;
}

// TSP + 哈密顿路径计数
int hamiltonian_path_count(int n, const vector<vector<bool>>& adj) {
 vector<vector<int>> dp(1 << n, vector<int>(n, 0));
 for (int i = 0; i < n; i++) dp[1 << i][i] = 1;

 for (int mask = 1; mask < (1 << n); mask++) {
 for (int i = 0; i < n; i++) {
 if (!(mask >> i & 1)) continue;
 for (int j = 0; j < n; j++) {
 if (mask >> j & 1) continue;
 if (adj[i][j])
 dp[mask | (1 << j)][j] = (dp[mask | (1 << j)][j] + dp[mask][i]) % MOD;
 }
 }
 }

 int ans = 0;
 for (int i = 0; i < n; i++)
 ans = (ans + dp[(1 << n) - 1][i]) % MOD;
 return ans;
}
```

### 6.10.3 SOS DP (子集 DP / Sum Over Subsets)

核心应用：快速计算所有子集 / 超集的聚合信息。

```

// === SOS DP 模板 ===
// 计算 $f[mask] = \sum_{\{sub \subseteq mask\}} g[sub]$
// 即对每个 $mask$, 求其所有子集的贡献和

vector<long long> sos_sum_subset(const vector<long long>& g, int k) {
 // k = 位数, 总状态数 = $1 \ll k$
 vector<long long> dp(g.begin(), g.end()); // $dp = f$
 for (int i = 0; i < k; i++) {
 for (int mask = 0; mask < (1 << k); mask++) {
 if (mask >> i & 1) {
 dp[mask] += dp[mask ^ (1 << i)];
 }
 }
 }
 return dp;
}

// 变体 1: 超集求和 $f[mask] = \sum_{\{super \supseteq mask\}} g[super]$
vector<long long> sos_sum_superset(const vector<long long>& g, int k) {
 vector<long long> dp(g.begin(), g.end());
 for (int i = 0; i < k; i++) {
 for (int mask = 0; mask < (1 << k); mask++) {
 if (!(mask >> i & 1)) { // 注意条件相反
 dp[mask] += dp[mask | (1 << i)];
 }
 }
 }
 return dp;
}

// 变体 2: 子集 max / min
vector<int> sos_max_subset(const vector<int>& g, int k) {
 vector<int> dp(g.begin(), g.end());
 for (int i = 0; i < k; i++) {
 for (int mask = 0; mask < (1 << k); mask++) {
 if (mask >> i & 1) {
 dp[mask] = max(dp[mask], dp[mask ^ (1 << i)]);
 }
 }
 }
 return dp;
}

// 变体 3: 子集计数 (每种元素最多出现一次, 即每个子集各元素计数独立)
// 类似 FWT 的子集卷积预处理
vector<vector<long long>> sos_subset_by_popcount(const vector<long long>& g, int k) {
 // $dp[pc][mask] = \sum_{\{sub \subseteq mask \text{ where } popcount(sub) == pc\}} g[sub]$
 vector<vector<long long>> dp(k + 1, vector<long long>(1 << k, 0));
 for (int mask = 0; mask < (1 << k); mask++) {
 dp[__builtin_popcount(mask)][mask] = g[mask];
 }
 for (int i = 0; i < k; i++) {
 for (int pc = 0; pc <= k; pc++) {
 for (int mask = 0; mask < (1 << k); mask++) {
 if (mask >> i & 1) {
 dp[pc][mask] += dp[pc][mask ^ (1 << i)];
 }
 }
 }
 }
 return dp;
}

```

## SOS DP 应用示例:

```

// 例: 给定 n 个字符串, 对每个字符串求它是多少个其他字符串的超序列子序列
// 等价于: 对每个 $mask$, 求有多少个给定 $mask$ 是它的子集
vector<int> count_submasks_in_array(const vector<int>& arr, int k) {
 vector<long long> freq(1 << k, 0);
 for (int x : arr) freq[x]++;
 auto dp = sos_sum_subset(freq, k); // SOS DP
 vector<int> ans;
 for (int x : arr) ans.push_back(dp[x]);
 return ans;
}

// 例: 求所有子集的最大 AND 值
int max_and_of_subsets(const vector<int>& arr, int k) {
 // 对每个 $mask$, 检查是否存在至少 2 个元素的超集包含它
 vector<int> freq(1 << k, 0);
 for (int x : arr) freq[x]++;

 auto dp = sos_sum_superset(freq, k); // 变成超集频率

 int ans = 0;
 for (int mask = 0; mask < (1 << k); mask++) {
 if (dp[mask] >= 2) ans = max(ans, mask);
 }
 return ans;
}

```

## 6.10.4 状压 DP — 枚举最后一组

```
// 例：划分问题 - 将 n 个物品分成若干组，每组权值满足约束
// $dp[mask]$ = 将 $mask$ 集合最优划分的答案

vector<int> dp(1 << n, INF);
dp[0] = 0;

// 预处理每组的合法性
vector<bool> valid(1 << n, false);
for (int mask = 1; mask < (1 << n); mask++) {
 if (check_group(mask)) valid[mask] = true;
}

for (int mask = 1; mask < (1 << n); mask++) {
 // 枚举 $mask$ 的最后一个组 (子集枚举技巧)
 for (int sub = mask; sub; sub = (sub - 1) & mask) {
 if (valid[sub]) {
 dp[mask] = min(dp[mask], dp[mask ^ sub] + cost[sub]);
 }
 }
}
}
```

## 6.11 数位 DP (Digit DP)

核心：记忆化搜索，memo key 通常为 (pos, tight, leadzero, ...)。

**CRITICAL**：多组测试用例时，memo 必须在**每组**清空（因为不同的 upper bound 导致不同结果）。但可以用二维数组 memo[pos][tight] 并在每个测试用例用递增的 case\_id 标记是否已计算。

```
// 数位 DP 通用模板
// 例：求 $[0, n]$ 中不包含数字 4，且数字 2 的个数 $\leq k$ 的数的个数

long long digit_dp(long long n, int k) {
 if (n < 0) return 0;

 // 将 n 分解为十进制位
 vector<int> digits;
 while (n) { digits.push_back(n % 10); n /= 10; }
 reverse(digits.begin(), digits.end());
 int m = digits.size();

 // memo[pos][tight][leadzero][cnt2]
 // 实际中 tight 维度可以不 memo (因为 tight 状态很少共享)
 // 或在非 tight 时存入，tight 时直接计算
 static long long memo[20][2][2][20];
 static int vis[20][2][2][20];
 int case_id = 0; // 多测清空用

 function<long long(int, bool, bool, int)> dfs =
 [&](int pos, bool tight, bool leadzero, int cnt2) -> long long {
 if (cnt2 > k) return 0;
 if (pos == m) return 1; // 走到最后，合法
 if (!tight && vis[pos][leadzero][cnt2] == case_id)
 return memo[pos][leadzero][cnt2];

 long long res = 0;
 int up = tight ? digits[pos] : 9;
 for (int d = 0; d <= up; d++) {
 if (d == 4) continue; // 不包含 4
 bool ntight = tight && (d == up);
 bool nlead = leadzero && (d == 0);
 int ncnt2 = cnt2 + (!nlead && d == 2);
 res += dfs(pos + 1, ntight, nlead, ncnt2);
 }

 if (!tight) {
 vis[pos][leadzero][cnt2] = case_id;
 memo[pos][leadzero][cnt2] = res;
 }
 return res;
 };

 case_id++;
 return dfs(0, true, true, 0);
}

// 区间查询 $[L, R]$
long long query(long long L, long long R, int k) {
 return digit_dp(R, k) - digit_dp(L - 1, k);
}
```

常用 memo 策略：

```
// 方案 A：二维 memo[pos][cnt]，非 tight 时存入
long long memo[20][N]; // N 为额外状态维度

// 调用时：
if (!tight && memo[pos][state] != -1) return memo[pos][state];
// ... 计算 ...
if (!tight) memo[pos][state] = res;
return res;

// 方案 B：多测用递增 case_id 代替 memset
int vis[20][N];
long long memo[20][N];
int stamp = 0;

// 每组测试前 stamp++
// 检查：if (!tight && vis[pos][state] == stamp) return memo[pos][state];
// 标记：vis[pos][state] = stamp; memo[pos][state] = res;
```

数位 DP 经典变体：

```
// 1. 数位和 / 数位积
// 额外状态: sum 或 product
// 注: 乘积状态可能很大, 可用 map 或 限制范围

// 2. 模某个数的余数
// 额外状态: rem = (rem * 10 + d) % MOD

// 3. LIS 数位 DP (计算数位的最长上升子序列)
// 用 mask (0-9 位) 维护当前 LIS 的结尾集合

// 4. 上界 + 下界同时限定
// tight_low, tight_high 双标志

// 5. 二进制数位 DP
// digits 用 base=2, 处理 XOR/AND/OR 性质
```

## 6.12 斜率优化 / 凸包优化 (Convex Hull Trick)

### 6.12.1 基础 CHT (Li Chao 树替代方案)

**问题形式:**  $dp[i] = \min_{\{j < i\}} \{ dp[j] + f(j, i) \}$ , 其中  $f(j, i)$  可写成  $m_j * x_i + c_j$  的形式。

**适用条件:** 斜率  $m_j$  单调, 查询  $x_i$  单调 (双端队列维护凸包)。

```
// DP 转移: dp[i] = min_{j < i} { dp[j] + (s[i]-s[j])^2 } + C
// 展开: dp[i] = min { -2*s[i]*s[k] + (dp[k]+s[k]^2) } + s[i]^2 + C
// 直线 y = m*x + b, 其中 m = -2*s[k], b = dp[k] + s[k]^2
// 查询点 x = s[i]
// 斜率 m 递减 (-2*s[k] 递减), 查询 x 递增

struct Line {
 long long m, b; // y = m*x + b
 long long eval(long long x) const { return m * x + b; }
 // 交点的 x 坐标: 交点 = (b2-b1)/(m1-m2)
 double intersect_x(const Line& other) const {
 return (double)(other.b - b) / (m - other.m);
 }
};

struct CHT {
 deque<Line> dq;

 // 加入新直线 (斜率单调的情况)
 // 若斜率递增 (m 从小到大加入), 维护下凸壳 (求最小值)
 // 若斜率递减 (m 从大到小加入), 维护上凸壳 (求最大值)
 // 以下代码针对斜率递减, 维护下凸壳的情况 (min 查询)
 void add(long long m, long long b) {
 Line cur = {m, b};
 // 保持凸性: 新线 + 队尾前两条的交点如果比队尾与倒数第二的交点更靠左, 则弹出队尾
 while (dq.size() >= 2) {
 Line& l1 = dq[dq.size() - 2];
 Line& l2 = dq.back();
 // l1 ∩ l2 >= l2 ∩ cur → l2 不优, 弹掉
 if ((l2.b - l1.b) * (m - l2.m) >= (b - l2.b) * (l1.m - l2.m))
 dq.pop_back();
 else
 break;
 }
 dq.push_back(cur);
 }

 // 查询 x (查询点 x 单调时 O(1) 均摊)
 long long query(long long x) {
 while (dq.size() >= 2 && dq[0].eval(x) >= dq[1].eval(x))
 dq.pop_front();
 return dq.empty() ? 1e18 : dq.front().eval(x);
 }

 // 二分查询 (查询点 x 不单调时, O(log N))
 long long query_binsearch(long long x) {
 int lo = 0, hi = dq.size() - 1;
 while (lo < hi) {
 int mid = (lo + hi) / 2;
 if (dq[mid].eval(x) <= dq[mid + 1].eval(x))
 hi = mid;
 else
 lo = mid + 1;
 }
 return dq[lo].eval(x);
 }
};

// 使用示例: 序列划分
long long seq_partition(const vector<long long>& a, long long C) {
 int n = a.size();
 vector<long long> s(n + 1, 0), dp(n + 1, 0);
 for (int i = 1; i <= n; i++) s[i] = s[i - 1] + a[i - 1];

 CHT cht;
 cht.add(0, 0); // 直线 y = 0*x + 0
 for (int i = 1; i <= n; i++) {
 // dp[i] = min_{k < i} { -2*s[i]*s[k] + dp[k] + s[k]^2 } + s[i]^2 + C
 dp[i] = cht.query(s[i]) + s[i] * s[i] + C;
 cht.add(-2 * s[i], dp[i] + s[i] * s[i]);
 }
 return dp[n];
}
```

### 6.12.2 Li Chao 线段树

当斜率不单调时, 用 Li Chao 树替代。



```

// Li Chao 线段树：支持插入线段 + 查询某点的最小值
// 定义域是整数，值域大时用动态开点
template <typename T>
struct LiChaoTree {
 struct Node {
 T m = 0, b = INF; // y = m*x + b, 初始 b = INF
 int lc = -1, rc = -1;
 };
 vector<Node> tr;
 T L, R; // X 坐标范围

 LiChaoTree(T l, T r) : L(l), R(r) { tr.emplace_back(); }

 T eval(int idx, T x) const {
 return tr[idx].m * x + tr[idx].b;
 }

 void add_line(T m, T b) { _add(0, L, R, m, b); }

 void _add(int idx, T l, T r, T m, T b) {
 T mid = l + (r - l) / 2;
 bool left_better = eval(idx, l) > m * l + b;
 bool mid_better = eval(idx, mid) > m * mid + b;

 if (mid_better) {
 swap(tr[idx].m, m);
 swap(tr[idx].b, b);
 }
 if (l == r) return;

 if (left_better != mid_better) {
 if (tr[idx].lc == -1) {
 tr[idx].lc = tr.size(); tr.emplace_back();
 }
 _add(tr[idx].lc, l, mid, m, b);
 } else {
 if (tr[idx].rc == -1) {
 tr[idx].rc = tr.size(); tr.emplace_back();
 }
 _add(tr[idx].rc, mid + 1, r, m, b);
 }
 }

 T query(T x) { return _query(0, L, R, x); }

 T _query(int idx, T l, T r, T x) {
 if (idx == -1) return INF;
 T res = eval(idx, x);
 if (l == r) return res;
 T mid = l + (r - l) / 2;
 if (x <= mid)
 return min(res, _query(tr[idx].lc, l, mid, x));
 else
 return min(res, _query(tr[idx].rc, mid + 1, r, x));
 }
};

```

### 6.12.3 交叉乘积防溢出

CHT 中判断交点时使用交叉乘法代替浮点数除法，避免精度问题和溢出：

```

// 判断 l1 与 l2 的交点是否在 l2 与 l3 交点的右侧
// 即 cross(l1, l2) >= cross(l2, l3) 时弹掉 l2
// cross(l_i, l_j) = (b_j - b_i) / (m_i - m_j)
// 用乘法：(b_j - b_i) * (m_j - m_k) >= (b_k - b_j) * (m_i - m_j)

// 但更稳健的做法是使用 __int128 防溢出：
bool is_bad(const Line& l1, const Line& l2, const Line& l3) {
 // 判断 l2 是否无效
 // (b3 - b1) * (m1 - m2) <= (b2 - b1) * (m1 - m3) 时 l2 不优
 __int128 lhs = (__int128)(l3.b - l1.b) * (l1.m - l2.m);
 __int128 rhs = (__int128)(l2.b - l1.b) * (l1.m - l3.m);
 return lhs <= rhs; // 取决于上下凸壳和符号
}

```

## 6.13 四边形不等式优化 (Divide and Conquer DP)

**问题形式：**  $dp[i][j] = \min_{k \in [i, j]} \{ dp[i-1][k-1] + cost(k, j) \}$ ，其中  $cost$  满足四边形不等式。

**优化：**当  $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$  时，内层枚举  $k$  的范围被限定，总复杂度从  $O(K \cdot N^2)$  降到  $O(K \cdot N)$ 。

### 6.13.1 分治优化 (Divide and Conquer DP)

当  $dp[layer][j]$  只依赖  $dp[layer-1][...]$ ，且决策具有单调性时使用。

```

// dp_new[j] = min_{0<=k<j} { dp_old[k] + cost(k, j) }
// opt[j] 随 j 单调不降

// 分治函数: solve(l, r, opt_l, opt_r)
// 表示 dp_new[l..r] 的最优决策点在 [opt_l, opt_r] 范围内

vector<long long> dp_old, dp_new;

void dac_dp(int l, int r, int opt_l, int opt_r) {
 if (l > r) return;
 int mid = (l + r) / 2;
 int best_k = opt_l;
 long long best_val = 1e18;

 for (int k = opt_l; k <= min(opt_r, mid - 1); k++) {
 long long cur = dp_old[k] + cost(k + 1, mid);
 if (cur < best_val) {
 best_val = cur;
 best_k = k;
 }
 }
 dp_new[mid] = best_val;

 dac_dp(l, mid - 1, opt_l, best_k);
 dac_dp(mid + 1, r, best_k, opt_r);
}

// 逐层调用
void solve_layered(int n, int K) {
 for (int i = 1; i <= K; i++) {
 dp_new.assign(n + 1, 1e18);
 dac_dp(1, n, 0, n - 1);
 dp_old.swap(dp_new);
 }
}

```

### 6.13.2 Knuth 优化

当 `cost` 同时满足四边形不等式和区间单调性时, `opt[l][r-1] <= opt[l][r] <= opt[l+1][r]`。

```

// 区间 DP 的 Knuth 优化
// dp[l][r] = min_{l<=k<r} { dp[l][k] + dp[k+1][r] } + cost(l, r)

vector<vector<long long>> dp(n, vector<long long>(n, 1e18));
vector<vector<int>> opt(n, vector<int>(n, 0));

for (int i = 0; i < n; i++) {
 dp[i][i] = 0;
 opt[i][i] = i;
}

for (int len = 2; len <= n; len++) {
 for (int l = 0; l + len - 1 < n; l++) {
 int r = l + len - 1;
 int kl = opt[l][r - 1]; // 左界
 int kr = opt[l + 1][r]; // 右界 (len>2 时有定义, 需处理)
 if (kr < kl) kr = r - 1;
 for (int k = kl; k <= kr; k++) {
 long long cur = dp[l][k] + dp[k + 1][r] + pref[r + 1] - pref[l];
 if (cur < dp[l][r]) {
 dp[l][r] = cur;
 opt[l][r] = k;
 }
 }
 }
}
}

```

### 6.13.3 1D/1D 优化（单调队列 / 二分栈）

```
// dp[i] = min_{j<i} { dp[j] + cost(j, i) }
// 若 opt 单调, 可以用二分栈 $O(N \log N)$

struct Decision {
 int opt, l, r; // opt 是 [l, r] 范围内的最优决策
};

// 使用二分栈维护决策区间
// 每次在队尾二分找到新决策开始优于旧决策的位置
deque<Decision> dq;

long long calc(int j, int i) {
 return dp[j] + cost(j, i);
}

void add_decision(int i) {
 // 弹出被 i 完全支配的决策
 while (!dq.empty() && calc(i, dq.back().l) <= calc(dq.back().opt, dq.back().l))
 dq.pop_back();
 if (dq.empty()) {
 dq.push_back({i, i + 1, n});
 return;
 }
 auto& last = dq.back();
 int lo = last.l, hi = last.r, pos = last.r + 1;
 while (lo <= hi) {
 int mid = (lo + hi) / 2;
 if (calc(i, mid) <= calc(last.opt, mid)) {
 pos = mid;
 hi = mid - 1;
 } else {
 lo = mid + 1;
 }
 }
 last.r = pos - 1;
 if (pos <= n) dq.push_back({i, pos, n});
}

int get_opt(int i) {
 while (!dq.empty() && dq.front().r < i) dq.pop_front();
 return dq.front().opt;
}
```

## 6.14 概率 DP / 期望 DP

```
// 期望 DP 通用思路：从终点倒推或列方程
// dp[i] = 从状态 i 到达终点的期望步数

// 例：掷骰子问题 - 求从 0 到 N 的期望步数
double dice_expectation(int N) {
 vector<double> dp(N + 7, 0); // 多开边界防溢出
 for (int i = N - 1; i >= 0; i--) {
 double sum = 0;
 for (int d = 1; d <= 6; d++)
 sum += dp[i + d];
 dp[i] = sum / 6.0 + 1.0;
 }
 return dp[0];
}

// 例：有环期望 - 高斯消元
// dp[u] = 1 + avg_{v in adj[u]} dp[v]
// 对于一般图用高斯消元 $O(N^3)$
```

## 6.15 博弈 DP

```
// 博弈 DP 通用模式
// dp[state] = 当前玩家能否必胜
// dp[state] = !dp[next_state_1] || !dp[next_state_2] || ...
// （存在一个后继态让对手必败，则当前必胜）
// 终结态：没有合法操作 → 必败

// 例：Nim 游戏变体
// dp[i] = 从 i 个石子开始，当前玩家是否必胜
vector<bool> game_dp(int N, const vector<int>& moves) {
 vector<bool> dp(N + 1, false);
 for (int i = 1; i <= N; i++) {
 for (int m : moves) {
 if (i >= m && !dp[i - m]) {
 dp[i] = true;
 break;
 }
 }
 }
 return dp;
}
```

6.16 DP 技巧一览

6.16.1 状态设计常见思路

模式	示例问题	状态定义
线性 DP	LIS, 最大子段和	dp[i] 以 i 结尾
前缀 DP	背包, 划分数	dp[j] 前 i 个容量为 j
区间 DP	石子合并, 回文	dp[l][r] 区间
树形 DP	最大独立集, 直径	dp[u][0/1] 以 u 为根
状压 DP	TSP, 匹配	dp[mask] 或 dp[mask][i]
数位 DP	范围内计数	dp(pos, tight, ...)
期望 DP	随机过程	dp[i] 从 i 到终点的期望

6.16.2 优化对照表

优化方法	适用 DP 形式	复杂度降低
滚动数组	只依赖上一行/列	$O(N^2)$ 空间 $\rightarrow O(N)$
单调队列	<code>dp[i] = min/max {dp[k] + f(i,k)}</code> 窗口约束	$O(N^2) \rightarrow O(N)$
斜率优化 (CHT)	<code>dp[i] = min {m_j * x_i + c_j}</code>	$O(N^2) \rightarrow O(N)$
分治优化	<code>dp[layer][j] = min {dp[layer-1][k] + cost(k,j)}</code>	$O(KN^2) \rightarrow O(KN \log N)$
Knuth 优化	区间 DP 满足四边形不等式	$O(N^3) \rightarrow O(N^2)$
SOS DP	子集求和	$O(3^N) \rightarrow O(N^2 \cdot N)$
二进制拆分	多重背包	$O(\sum cnt) \rightarrow O(C \sum \log cnt)$

6.16.3 常见调试错误

```
// 1. 背包循环方向写反
// 01: for (j = C; j >= w[i]; j--) ← 递减
// 完全: for (j = w[i]; j <= C; j++) ← 递增

// 2. 状压 DP 中未处理非法状态
if (invalid[mask]) continue;

// 3. 数位 DP 中 memo 未在每组测试清空

// 4. CHT 中斜率/截距正负号写反
// 展开 dp[i] = min { ... } 时保持 m_j * x_i + c_j 形式

// 5. 区间 DP 中 len=1 基础情况未初始化
for (int i = 0; i < n; i++) dp[i][i] = 0;

// 6. 四边形不等式 opt[l][r-1], opt[l+1][r] 越界
// len=2 时需检查 opt[l+1][r] 是否存在
```

6.17 综合例题

6.17.1 分组背包

```
// N 组物品, 每组最多选一个, 容量 C
int group_knapsack(const vector<vector<pair<int,int>>& groups, int C) {
 // groups[g] = vector of {weight, value}
 vector<int> dp(C + 1, 0);
 for (auto& group : groups) {
 for (int j = C; j >= 0; j--) { // 01 模式: 先容量
 for (auto [w, v] : group) {
 if (j >= w) dp[j] = max(dp[j], dp[j - w] + v);
 }
 }
 }
 return dp[C];
}
```

6.17.2 有依赖的背包

```
// 树形依赖 + 分组思想
// 把每个子树当作一组物品, 可选 0/1/.../son_size 体积的方案
// 详见 6.9 节
```

6.17.3 泛化物品合并

```
// 两个泛化物品的合并 (对每个容量 j, 枚举分配给 a 的体积 k)
vector<int> merge_items(const vector<int>& a, const vector<int>& b, int C) {
 vector<int> c(C + 1, -1e9);
 for (int j = 0; j <= C; j++) {
 for (int k = 0; k <= j; k++) {
 if (a[k] != -1e9 && b[j - k] != -1e9)
 c[j] = max(c[j], a[k] + b[j - k]);
 }
 }
 return c;
}
```

6.18 总结

7. 计算几何 (Computational Geometry)

7.1 基础：点与向量 (Point and Vector Basics)

English: Point and Vector Primitives | Chinese: 点与向量的基础运算

计算几何的基石。全部算法均建立在叉积 (cross product) 和点积 (dot product) 之上。EPS 取  $1e-9$  处理浮点误差。能不用浮点就不用——大多数判定问题（相交、包含、凸包等）用整数叉积即可完美解决，零误差。

```
// ===== 浮点版本 (通用) =====
using T = double; // 高精度需求可改为 long double
const T EPS = 1e-9;
int sgn(T x) { return (x > EPS) - (x < -EPS); } // 符号函数

struct P {
 T x, y;
 P(T x = 0, T y = 0) : x(x), y(y) {}

 // 运算符
 P operator+(P o) const { return {x + o.x, y + o.y}; }
 P operator-(P o) const { return {x - o.x, y - o.y}; }
 P operator*(T d) const { return {x * d, y * d}; }
 P operator/(T d) const { return {x / d, y / d}; }
 bool operator<(P o) const { return tie(x, y) < tie(o.x, o.y); }
 bool operator==(P o) const { return tie(x, y) == tie(o.x, o.y); }

 // 核心运算
 T dot(P o) const { return x * o.x + y * o.y; } // 点积 a·b
 T cross(P o) const { return x * o.y - y * o.x; } // 叉积 a×b (标量)
 T cross(P a, P b) const { return (a - *this).cross(b - *this); } // (a-this)×(b-this)

 T dist2() const { return x * x + y * y; } // 距离平方
 double dist() const { return sqrt(dist2()); } // 欧氏距离
 double angle() const { return atan2(y, x); } // 极角 [-pi, pi]

 P perp() const { return {-y, x}; } // 逆时针旋转 90°
 P rotate(double a) const {
 return {x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)};
 } // 旋转 a 弧度

 friend ostream& operator<<(ostream& os, P p) { return os << "(" << p.x << ", " << p.y << ")"; }
};

// ===== 整数版本 (推荐：无浮点误差) =====
struct iPoint {
 ll x, y;
 iPoint(ll x = 0, ll y = 0) : x(x), y(y) {}
 iPoint operator+(iPoint o) const { return {x + o.x, y + o.y}; }
 iPoint operator-(iPoint o) const { return {x - o.x, y - o.y}; }
 iPoint operator*(ll d) const { return {x * d, y * d}; }
 bool operator<(iPoint o) const { return tie(x, y) < tie(o.x, o.y); }
 bool operator==(iPoint o) const { return tie(x, y) == tie(o.x, o.y); }
 ll dot(iPoint o) const { return x * o.x + y * o.y; } // 点积
 ll cross(iPoint o) const { return x * o.y - y * o.x; } // 叉积
 ll cross(iPoint a, iPoint b) const { return (a - *this).cross(b - *this); }
 ll dist2() const { return x * x + y * y; }

 // 三态朝向判定 (无需 EPS)
 int ori(iPoint a, iPoint b) const {
 ll v = cross(a, b);
 return (v > 0) - (v < 0); // +1=CCW, -1=CW, 0=共线
 }
};

// ===== 常用判断函数 =====

// 点在向量 p1→p2 的哪一侧：+1=左侧 (CCW), -1=右侧 (CW), 0=共线
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

// p 是否在线段 s-e 上 (含端点)
bool onSegment(P s, P e, P p) {
 return sgn(s.cross(e, p)) == 0 && sgn((s - p).dot(e - p)) <= 0;
}

// 点 p 到线段 s-e 的距离
double segDist(P s, P e, P p) {
 if (s == e) return (p - s).dist();
 auto d = (e - s).dist2(), t = min(d, max(.0, (p - s).dot(e - s)));
 return ((p - s) * d - (e - s) * t).dist() / d;
}
```

**叉积的几何意义：**`a.cross(b)` 等于 a 和 b 围成的平行四边形的有向面积 (CCW 为正)。这是所有朝向判定、面积计算、极角排序的基础。

**极角排序：**用叉积而非 `atan2`。`atan2` 有浮点误差且可能把  $-\pi$  和  $\pi$  判为不同角。叉积判定方向 (CCW/CW) 零误差。

```
// ---- 极角排序：以 p0 为原点，按 CCW 方向排序 ----
// 关键：用叉积判断方向，只在叉积为 0 时按距离排序 (近的在前)
void polarSort(vector<P>& pts, P p0) {
 sort(all(pts), [&](P a, P b) {
 // 先按象限分组：上半平面在前
 // 更鲁棒的方法：直接用叉积
 T o = p0.cross(a, b);
 if (o == 0) return (p0 - a).dist2() < (p0 - b).dist2(); // 共线时近的在前
 return o > 0; // CCW 方向
 });
}

// 整数版本的极角排序 (零误差)
void polarSortInt(vector<iPoint>& pts, iPoint p0) {
 sort(all(pts), [&](iPoint a, iPoint b) {
 // 上半平面 (y >= p0.y) 优先
 bool ha = (a.y > p0.y) || (a.y == p0.y && a.x > p0.x);
 bool hb = (b.y > p0.y) || (b.y == p0.y && b.x > p0.x);
 if (ha != hb) return ha;
 ll o = p0.cross(a, b);
 if (o == 0) return (p0 - a).dist2() < (p0 - b).dist2();
 return o > 0;
 });
}
```

## 7.2 线段与直线 (Segments and Lines)

**English:** Segment and Line Operations | **Chinese:** 线段与直线的运算

```

// ===== 线段相交判定 =====
// 返回值: 0=不相交, 1=严格相交(交点在线段内部), 2=端点接触或共线重叠
// 交叉实验 + 跨立实验 (straddle test)
template<class T>
int segmentIntersection(const Point<T>& a, const Point<T>& b,
 const Point<T>& c, const Point<T>& d,
 Point<double>& out) {
 T oa = c.cross(d, a), ob = c.cross(d, b);
 T oc = a.cross(b, c), od = a.cross(b, d);

 // 检查边界条件: 是否有端点在另一线段上
 auto between = [](T a, T b, T c) {
 return min(a, b) <= c + EPS && c <= max(a, b) + EPS;
 };

 if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) {
 // 严格相交: 求出交点
 double t = (double)((a - c).cross(d - c)) / (double)((b - a).cross(d - c));
 out.x = a.x + (b.x - a.x) * t;
 out.y = a.y + (b.y - a.y) * t;
 return 1;
 }

 // 处理退化情况
 if (sgn(oa) == 0 && between(c.x, d.x, a.x) && between(c.y, d.y, a.y)) {
 out = Point<double>(a.x, a.y); return 2;
 }
 if (sgn(ob) == 0 && between(c.x, d.x, b.x) && between(c.y, d.y, b.y)) {
 out = Point<double>(b.x, b.y); return 2;
 }
 if (sgn(oc) == 0 && between(a.x, b.x, c.x) && between(a.y, b.y, c.y)) {
 out = Point<double>(c.x, c.y); return 2;
 }
 if (sgn(od) == 0 && between(a.x, b.x, d.x) && between(a.y, b.y, d.y)) {
 out = Point<double>(d.x, d.y); return 2;
 }

 return 0; // 不相交
}

// 整数版本——纯叉积判定, 零误差
int segmentIntersectionInt(iPoint a, iPoint b, iPoint c, iPoint d) {
 auto ori = [](iPoint a, iPoint b, iPoint c) -> int {
 ll v = a.cross(b, c);
 return (v > 0) - (v < 0);
 };

 auto between = [](iPoint a, iPoint b, iPoint c) -> bool {
 return min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
 min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
 };

 int o1 = ori(a, b, c), o2 = ori(a, b, d);
 int o3 = ori(c, d, a), o4 = ori(c, d, b);

 if (o1 != o2 && o3 != o4) return 1; // 严格相交

 // 退化: 端点在另一线段上
 if (o1 == 0 && between(a, b, c)) return 2;
 if (o2 == 0 && between(a, b, d)) return 2;
 if (o3 == 0 && between(c, d, a)) return 2;
 if (o4 == 0 && between(c, d, b)) return 2;

 return 0;
}

// ===== 直线交点 =====
// 求两直线 a1-a2 与 b1-b2 的交点 (假设不平行)
P lineIntersection(P a1, P a2, P b1, P b2) {
 P va = a2 - a1, vb = b2 - b1;
 T t = (b1 - a1).cross(vb) / va.cross(vb);
 return a1 + va * t;
}

// ===== 点到直线投影 =====
P projectPointToLine(P p, P a, P b) {
 P v = b - a;
 return a + v * v.dot(p - a) / v.dist2();
}

// ===== 线段与直线是否严格相交 =====
bool segmentCrossesLine(P a, P b, P c, P d) {
 return sgn(a.cross(c, d)) * sgn(b.cross(c, d)) < 0;
}

```

### 7.3 多边形基础 (Polygon Basics)

**English:** Polygon Fundamentals | **Chinese:** 多边形基础运算

```

// ===== 多边形面积 (鞋带公式 / Shoelace) =====
// 顶点按顺序给出, CCW → 正面积, CW → 负面积 (返回二倍有向面积避浮点除法)
T polygonArea2(const vector<P>& poly) {
 T area = 0;
 int n = sz(poly);
 rep(i, 0, n) area += poly[i].cross(poly[(i + 1) % n]);
 return area; // >0 为 CCW
}
T polygonArea(const vector<P>& poly) { return fabs(polygonArea2(poly)) / 2.0; }

// 整数版本
ll polygonArea2Int(const vector<iPoint>& poly) {
 ll area = 0;
 int n = sz(poly);
 rep(i, 0, n) area += poly[i].cross(poly[(i + 1) % n]);
 return area;
}

// ===== 点在多边形内 (射线法 / Ray Casting) =====
// 对任意简单多边形有效 (不要求凸), O(N)
// 从 p 向 +x 方向发射射线, 数交点: 奇数 → 内部, 偶数 → 外部
// strict=true 时边界点返回 false
bool pointInPolygon(const vector<P>& poly, P p, bool strict = true) {
 int n = sz(poly), cnt = 0;
 rep(i, 0, n) {
 P a = poly[i], b = poly[(i + 1) % n];
 if (onSegment(a, b, p)) return !strict; // 在边上
 if (a.y > b.y) swap(a, b);
 // 检查射线是否穿过边 a-b (排除端点重合情况)
 if (sgn(p.y - a.y) > 0 && sgn(p.y - b.y) <= 0) {
 if (sgn(a.cross(b, p)) > 0) cnt++; // p 在边的左侧 → 射线穿过
 }
 }
 return cnt & 1;
}

// ===== 点在凸多边形内 (二分 / O(log N)) =====
// 要求顶点 CCW 排列, 不含共线边
// strict=true: 严格内部; false: 含边界
bool pointInConvexPolygon(const vector<P>& poly, P p, bool strict = true) {
 int n = sz(poly), a = 1, b = n - 1, r = !strict;
 if (n < 3) return r && onSegment(poly[0], poly.back(), p);
 // 先判断是否在最外层扇形外
 if (sideOf(poly[0], poly[a], p) >= r || sideOf(poly[0], poly[b], p) <= -r)
 return false;
 // 二找到 p 在哪两个顶点之间
 while (abs(a - b) > 1) {
 int c = (a + b) / 2;
 (sideOf(poly[0], poly[c], p) > 0 ? b : a) = c;
 }
 return poly[a].cross(poly[b], p) < r;
}

// ===== 多边形方向判定 =====
bool isCcw(const vector<P>& poly) { return polygonArea2(poly) > 0; }

// ===== 多边形周长 =====
double polygonPerimeter(const vector<P>& poly) {
 double p = 0;
 int n = sz(poly);
 rep(i, 0, n) p += (poly[(i + 1) % n] - poly[i]).dist();
 return p;
}

// ===== Pick 定理 =====
// 格点多边形面积 = 内部格点数 + 边界格点数 / 2 - 1
// 边界格点数: 每条边的 gcd(|dx|, |dy|) 之和
ll boundaryLatticePoints(const vector<iPoint>& poly) {
 ll cnt = 0;
 int n = sz(poly);
 rep(i, 0, n) {
 auto d = poly[(i + 1) % n] - poly[i];
 cnt += gcd(abs(d.x), abs(d.y));
 }
 return cnt;
}
// 内部格点数 = (面积 * 2 - 边界格点数 + 2) / 2

```

## 7.4 凸包 (Convex Hull)

**English:** Convex Hull | **Chinese:** 凸包

Andrew 单调链法,  $O(N \log N)$ 。比 Graham Scan 更简洁, 无需极角排序, 按 x-y 字典序排序即可。

**共线点处理策略:** `<= 0` 排除边上的共线点 (竞赛最常用), `< 0` 保留边上的共线点。

```
// ===== Andrew's Monotone Chain (推荐) =====
// 共线点处理: cross <= 0 -> 排除边上的共线点 (保留端点)
// cross < 0 -> 保留边上的共线点
// 返回值: CCW 顺序的凸包顶点, hull[0] 为最左下点
vector<P> convexHull(vector<P> pts) {
 if (sz(pts) <= 1) return pts;
 sort(all(pts)); // 按 x 再 y 排序
 vector<P> h(sz(pts) + 1);
 int s = 0, t = 0;
 // 两遍扫描: 下凸壳 + 上凸壳
 for (int it = 2; it--; s = --t, reverse(all(pts)))
 for (P p : pts) {
 while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <= 0) t--;
 h[t++] = p;
 }
 // 去掉末尾重复点 (全共线情况会只剩两个相同点)
 return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}

// ===== Andrew 展开版 (更易调试) =====
vector<P> convexHullVerbose(vector<P> pts) {
 int n = sz(pts);
 if (n <= 1) return pts;
 sort(all(pts));
 vector<P> hull(2 * n);
 int k = 0;
 // 下凸壳: 从左到右
 rep(i, 0, n) {
 while (k >= 2 && hull[k - 2].cross(hull[k - 1], pts[i]) <= 0) k--;
 hull[k++] = pts[i];
 }
 // 上凸壳: 从右到左
 for (int i = n - 2, t = k + 1; i >= 0; i--) {
 while (k >= t && hull[k - 2].cross(hull[k - 1], pts[i]) <= 0) k--;
 hull[k++] = pts[i];
 }
 hull.resize(k - 1);
 return hull;
}

// ===== 整数凸包 (零误差) =====
vector<iPoint> convexHullInt(vector<iPoint> pts) {
 if (sz(pts) <= 1) return pts;
 sort(all(pts));
 vector<iPoint> h(sz(pts) + 1);
 int s = 0, t = 0;
 for (int it = 2; it--; s = --t, reverse(all(pts)))
 for (iPoint p : pts) {
 while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <= 0) t--;
 h[t++] = p;
 }
 return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}

// 保留共线点版本: 将上述 <= 0 改为 < 0
// 凸包为直线的特殊情况检查
bool isHullDegenerate(const vector<P>& hull) { return sz(hull) < 3; }
```

**Graham Scan 替代** (用叉积极角排序, 非 atan2) :

```
vector<P> convexHullGraham(vector<P> pts) {
 // 找到最下最左的点作为原点
 P p0 = *min_element(all(pts), [](P a, P b) {
 return tie(a.y, a.x) < tie(b.y, b.x);
 });
 // 极角排序: 用叉积, CCW 顺序
 sort(all(pts), [&](P a, P b) {
 T o = p0.cross(a, b);
 if (o == 0) return (p0 - a).dist2() < (p0 - b).dist2();
 return o > 0; // CCW
 });
 vector<P> hull;
 for (P p : pts) {
 while (sz(hull) >= 2 && hull[sz(hull) - 2].cross(hull.back(), p) <= 0)
 hull.pop_back();
 hull.pb(p);
 }
 return hull;
}
```

## 7.5 旋转卡壳 (Rotating Calipers)

**English:** Rotating Calipers | **Chinese:** 旋转卡壳

在凸多边形上  $O(N)$  求解: 最远点对 (直径)、最小外接矩形、多边形宽度的利器。核心思路: 维护平行切线在凸包上同步旋转——切点的移动是单调的, 不需要回溯。



```

// ===== 凸包直径 (最远点对) =====
// 输入: CCW 凸包 (不含共线边上的点)
// 返回: 最远点对
pair<P, P> hullDiameter(const vector<P>& hull) {
 int n = sz(hull);
 if (n < 2) return {hull[0], hull[0]};
 int j = 1;
 pair<T, pair<P, P>> best = {0, {hull[0], hull[0]}};
 rep(i, 0, j) {
 for (; j = (j + 1) % n) {
 best = max(best, {(hull[i] - hull[j]).dist2(), {hull[i], hull[j]}});
 // 若 next(j) 不比 j 离边 (i, i+1) 更远, 则停止旋转 j
 if ((hull[(j + 1) % n] - hull[j]).cross(hull[(i + 1) % n] - hull[i]) >= 0)
 break;
 }
 }
 return best.second;
}

T hullDiameterDist2(const vector<P>& hull) {
 auto [a, b] = hullDiameter(hull);
 return (a - b).dist2();
}

// ===== 凸包宽度 (最近平行切线间距) =====
T hullWidth(const vector<P>& hull) {
 int n = sz(hull);
 if (n < 2) return 0;
 int j = 1;
 T width = 1e30;
 rep(i, 0, n) {
 // 找到离边 (i, i+1) 最远的点 j
 while (fabs(hull[i].cross(hull[(i + 1) % n], hull[(j + 1) % n])) >
 fabs(hull[i].cross(hull[(i + 1) % n], hull[j])))
 j = (j + 1) % n;
 T h = fabs(hull[i].cross(hull[(i + 1) % n], hull[j]))
 / (hull[(i + 1) % n] - hull[i]).dist();
 width = min(width, h);
 }
 return width;
}

// ===== 最小面积外接矩形 (Minimum Area Bounding Rectangle) =====
// 输入: CCW 凸包
T minAreaBoundingRect(const vector<P>& hull) {
 int n = sz(hull);
 if (n < 3) return 0;
 vector<P> H = hull;
 H.pb(H[0]); // 循环: 方便取边 (i, i+1)

 int j = 1, l = 1, r = 1;
 T ans = 1e30;
 rep(i, 0, n) {
 // j: 离边 (i, i+1) 最远的点 (叉积面积最大)
 while (fabs(H[i + 1].cross(H[(j + 1) % n] - H[i], H[j] - H[i])) >=
 fabs(H[i + 1].cross(H[j] - H[i], H[i] - H[j])))
 j = (j + 1) % n;
 // r: 沿边 (i, i+1) 方向投影最靠右的点 (点积最大)
 while ((H[i + 1] - H[i]).dot(H[(r + 1) % n] - H[i]) >=
 (H[i + 1] - H[i]).dot(H[r] - H[i]))
 r = (r + 1) % n;
 if (i == 0) l = r;
 // l: 沿边 (i, i+1) 方向投影最靠左的点 (点积最小)
 while ((H[i + 1] - H[i]).dot(H[(l + 1) % n] - H[i]) <=
 (H[i + 1] - H[i]).dot(H[l] - H[i]))
 l = (l + 1) % n;

 T h = fabs(H[i + 1].cross(H[j] - H[i])) / (H[i + 1] - H[i]).dist();
 T w = ((H[i + 1] - H[i]).dot(H[r] - H[i]) -
 (H[i + 1] - H[i]).dot(H[l] - H[i])) / (H[i + 1] - H[i]).dist();
 ans = min(ans, h * w);
 }
 return ans;
}

```

## 7.6 最近点对 (Closest Pair of Points)

**English:** Closest Pair of Points | **Chinese:** 平面最近点对

$O(N \log N)$  扫描线法 (sweep line)。维护一个按  $y$  排序的活跃窗口，每次以当前点的  $x$  做半径裁剪，窗口内满足  $y$  约束的候选点不超过 6 个。

```
// ===== 扫描线法 (推荐—实现简洁) =====
// 返回最近点对的距离平方
T closestPairDist2(vector<P> pts) {
 // 按 x 排序
 sort(all(pts), [](const P& a, const P& b) { return a.x < b.x; });
 set<pair<T, T>> active; // {y, x}: 按 y 排序的活跃点集
 T best = 9e18;
 int left = 0;

 rep(i, 0, sz(pts)) {
 T d = (T)sqrtl((long double)best) + 1;
 // 移除 x 方向距离已超过 d 的点 (不可能构成更优对)
 while (pts[i].x - pts[left].x > d)
 active.erase({pts[left].y, pts[left].x}), left++;

 // 在 y 方向 [y-d, y+d] 范围内枚举候选点 (最多 6 个)
 auto lo = active.lower_bound({pts[i].y - d, -1e18});
 auto hi = active.upper_bound({pts[i].y + d, 1e18});
 for (auto it = lo; it != hi; ++it) {
 T dx = pts[i].x - it->second;
 T dy = pts[i].y - it->first;
 best = min(best, dx * dx + dy * dy);
 }
 active.insert({pts[i].y, pts[i].x});
 }
 return best;
}

double closestPairDist(vector<P>& pts) { return sqrt(closestPairDist2(pts)); }

// ===== 分治法 (经典版本, 适合 N 极大时用索引避免拷贝) =====
// 传入已按 x 排序的点数组 [l, r), 辅助数组 tmp 用于按 y 归并
T closestPairDC(vector<P>& pts, int l, int r, vector<P>& tmp) {
 if (r - l <= 3) {
 T best = 9e18;
 rep(i, l, r) rep(j, i + 1, r) best = min(best, (pts[i] - pts[j]).dist2());
 sort(pts.begin() + l, pts.begin() + r,
 [](P a, P b) { return a.y < b.y; });
 return best;
 }
 int mid = (l + r) / 2;
 T midX = pts[mid].x;
 T best = min(closestPairDC(pts, l, mid, tmp),
 closestPairDC(pts, mid, r, tmp));

 // 归并: 按 y 排序
 merge(pts.begin() + l, pts.begin() + mid,
 pts.begin() + mid, pts.begin() + r,
 tmp.begin(), [](P a, P b) { return a.y < b.y; });
 copy(tmp.begin(), tmp.begin() + (r - l), pts.begin() + l);

 // 收集中间带内的点
 T d = (T)sqrtl((long double)best);
 vector<P> strip;
 rep(i, l, r) if (fabs(pts[i].x - midX) < d) strip.pb(pts[i]);

 // 检查带内点对 (每点最多检查 7 个后继)
 rep(i, 0, sz(strip)) {
 for (int j = i + 1; j < sz(strip) && strip[j].y - strip[i].y < d; j++)
 best = min(best, (strip[i] - strip[j]).dist2());
 }
 return best;
}

T closestPairDC_wrapper(vector<P> pts) {
 sort(all(pts), [](P a, P b) { return a.x < b.x; });
 vector<P> tmp(sz(pts));
 return closestPairDC(pts, 0, sz(pts), tmp);
}
```

## 7.7 半平面交 (Half-plane Intersection)

**English:** Half-plane Intersection | **Chinese:** 半平面交

$O(N \log N)$  的排序增量法 (S&I, Sort-and-Incremental)。用于求多边形核 (kernel)、二维线性约束的可行域、求凸多边形的交。每条半平面由其有向直线定义——直线左侧为有效区域。

```
// ===== Half-plane 定义 =====
struct Halfplane {
 P p, pq; // 直线上一点、方向向量
 double angle;

 Halfplane() {}
 Halfplane(const P& a, const P& b) : p(a), pq(b - a) {
 angle = atan2(pq.y, pq.x);
 }
 // 点 r 是否在右侧 (即不在有效区域内)
 bool out(const P& r) const { return pq.cross(r - p) < -EPS; }
 // 按极角排序
 bool operator<(const Halfplane& e) const { return angle < e.angle; }

 // 两条半平面的交点
 friend P inter(const Halfplane& s, const Halfplane& t) {
 double alpha = (t.p - s.p).cross(t.pq) / s.pq.cross(t.pq);
 return s.p + s.pq * alpha;
 }
};

// ===== S&I 算法 =====
vector<P> halfplaneIntersection(vector<Halfplane> H) {
 // 加包围盒: 避免无界区域的数值问题
 const double INF = 1e9;
 P box[4] = {P(INF, INF), P(-INF, INF), P(-INF, -INF), P(INF, -INF)};
 rep(i, 0, 4) H.emplace_back(box[i], box[(i + 1) % 4]); // CCW box

 sort(all(H));
 deque<Halfplane> dq;
 int len = 0;

 rep(i, 0, sz(H)) {
 // 弹出队尾的多余半平面
 while (len > 1 && H[i].out(inter(dq[len - 1], dq[len - 2])))
 dq.pop_back(), --len;
 // 弹出队首的多余半平面
 while (len > 1 && H[i].out(inter(dq[0], dq[1])))
 dq.pop_front(), --len;
 // 平行线: 保留更严格的那条 (在有效区域内更苛刻的)
 if (len > 0 && fabs(dq[len - 1].pq.cross(H[i].pq)) < EPS) {
 if (dq[len - 1].pq.dot(H[i].pq) < 0) return {}; // 方向相反 → 空
 if (H[i].out(dq[len - 1].p)) dq.pop_back(), --len; // 用更严格的替换
 else continue; // 当前半平面被覆盖
 }
 dq.pb(H[i]), ++len;
 }

 // 最终清理: 检查队首队尾的冗余
 while (len > 2 && dq[0].out(inter(dq[len - 1], dq[len - 2])))
 dq.pop_back(), --len;
 while (len > 2 && dq[len - 1].out(inter(dq[0], dq[1])))
 dq.pop_front(), --len;

 if (len < 3) return {};

 vector<P> ret(len);
 rep(i, 0, len) ret[i] = inter(dq[i], dq[(i + 1) % len]);
 return ret;
}

// ===== 多边形核 (Kernel) =====
// 求多边形中能看到所有顶点的区域 = 所有边定义的半平面交
vector<P> polygonKernel(const vector<P>& poly) {
 int n = sz(poly);
 vector<Halfplane> hp;
 rep(i, 0, n) hp.emplace_back(poly[i], poly[(i + 1) % n]); // CCW → 边左侧
 return halfplaneIntersection(hp);
}
```

## 7.8 最小圆覆盖 (Minimum Enclosing Circle)

**English:** Minimum Enclosing Circle (MEC) | **Chinese:** 最小圆覆盖

Welzl 随机增量法, 期望复杂度  $O(N)$ , 最坏  $O(N^3)$  但实际极快。核心思想: 打乱点的顺序, 逐步构建最小圆; 若当前点在新圆外, 则以该点为边界点重建一个更小的子问题。

```

// ===== 三点求外接圆 =====
// 返回 {圆心, 半径平方}
// 共线时返回两点直径
pair<P, T> circumCircle(P a, P b, P c) {
 T d = (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y)) * 2;
 if (fabs(d) < EPS) {
 // 三点共线：取最长直径
 T d2 = max({(a - b).dist2(), (b - c).dist2(), (c - a).dist2()});
 if ((a - b).dist2() == d2) return {(a + b) / 2, d2 / 4};
 if ((b - c).dist2() == d2) return {(b + c) / 2, d2 / 4};
 return {(c + a) / 2, d2 / 4};
 }
 T a2 = a.dot(a), b2 = b.dot(b), c2 = c.dot(c);
 T ux = (a2 * (b.y - c.y) + b2 * (c.y - a.y) + c2 * (a.y - b.y)) / d;
 T uy = (a2 * (c.x - b.x) + b2 * (a.x - c.x) + c2 * (b.x - a.x)) / d;
 P center(ux, uy);
 return {center, (center - a).dist2()};
}

// ===== Welzl 随机增量法 =====
// 返回 {圆心, 半径}
pair<P, double> minEnclosingCircle(vector<P> pts) {
 if (pts.empty()) return {P(0, 0), 0};
 random_shuffle(all(pts)); // 关键：随机化保证期望 $O(N)$

 P o = pts[0];
 double r = 0;
 int n = sz(pts);

 // 主循环：检查并修复
 rep(i, 0, n) {
 if ((pts[i] - o).dist2() <= r * r + EPS) continue; // 在圆内
 o = pts[i], r = 0; // 以 pts[i] 为边界点重新构建
 rep(j, 0, i) {
 if ((pts[j] - o).dist2() <= r * r + EPS) continue;
 o = (pts[i] + pts[j]) * 0.5; // 以 pts[i], pts[j] 为边界直径
 r = (pts[i] - pts[j]).dist() * 0.5;
 rep(k, 0, j) {
 if ((pts[k] - o).dist2() <= r * r + EPS) continue;
 // 三点确定圆
 auto [co, cr2] = circumCircle(pts[i], pts[j], pts[k]);
 o = co;
 r = sqrt(cr2);
 }
 }
 }
 return {o, r};
}

// 简洁版（竞赛用）—— 同上但不拆分函数
pair<P, double> minEnclosingCircleCompact(vector<P> p) {
 random_shuffle(all(p));
 P c = p[0];
 double r = 0;
 int n = sz(p);
 rep(i, 0, n) {
 if ((p[i] - c).dist2() <= r * r + EPS) continue;
 c = p[i], r = 0;
 rep(j, 0, i) {
 if ((p[j] - c).dist2() <= r * r + EPS) continue;
 c = (p[i] + p[j]) * 0.5, r = (p[i] - p[j]).dist() * 0.5;
 rep(k, 0, j) {
 if ((p[k] - c).dist2() <= r * r + EPS) continue;
 // 三点外接圆
 T d = 2 * (p[i].x * (p[j].y - p[k].y) + p[j].x * (p[k].y - p[i].y) + p[k].x * (p[i].y - p[j].y));
 if (fabs(d) < EPS) continue;
 T a2 = p[i].dot(p[i]), b2 = p[j].dot(p[j]), c2 = p[k].dot(p[k]);
 c.x = (a2 * (p[j].y - p[k].y) + b2 * (p[k].y - p[i].y) + c2 * (p[i].y - p[j].y)) / d;
 c.y = (a2 * (p[k].x - p[j].x) + b2 * (p[i].x - p[k].x) + c2 * (p[j].x - p[i].x)) / d;
 r = (c - p[i]).dist2();
 }
 }
 }
 return {c, r};
}

```

## 7.9 多边形切割与闵可夫斯基和 (Polygon Cut and Minkowski Sum)

**English:** Polygon Cut and Minkowski Sum | **Chinese:** 多边形切割 / 闵可夫斯基和

```

// ===== 多边形切割 (用有向直线切, 保留左侧) =====
// O(N)
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
 vector<P> res;
 int n = sz(poly);
 rep(i, 0, n) {
 P cur = poly[i], prev = poly[(i ? i : n) - 1];
 T a = s.cross(e, cur), b = s.cross(e, prev);
 // 边穿过切割线: 计算交点
 if (sgn(a) != sgn(b))
 res.pb(cur + (prev - cur) * (a / (a - b)));
 // cur 在左侧 → 保留
 if (sgn(a) < 0) res.pb(cur);
 }
 return res;
}

// ===== 闵可夫斯基和 (凸多边形 A + 凸多边形 B) =====
// A+B = { a+b | a∈A, b∈B }, 结果仍是凸多边形, O(|A|+|B|)
// 要求: CCW 排列, 最低最左点在 first
void reorderPolygon(vector<P>& poly) {
 int pos = 0;
 rep(i, 1, sz(poly))
 if (poly[i].y < poly[pos].y || (poly[i].y == poly[pos].y && poly[i].x < poly[pos].x))
 pos = i;
 rotate(poly.begin(), poly.begin() + pos, poly.end());
}

vector<P> minkowskiSum(vector<P> A, vector<P> B) {
 reorderPolygon(A); reorderPolygon(B);
 A.pb(A[0]); A.pb(A[1]); // 循环访问
 B.pb(B[0]); B.pb(B[1]);
 vector<P> res;
 int n = sz(A) - 2, m = sz(B) - 2;
 int i = 0, j = 0;
 while (i < n || j < m) {
 res.pb(A[i] + B[j]);
 T crossProd = (A[i + 1] - A[i]).cross(B[j + 1] - B[j]);
 if (crossProd >= 0 && i < n) i++;
 if (crossProd <= 0 && j < m) j++;
 }
 return res;
}

// ===== 凸多边形距离 (A 到 B 的最短距离) =====
// A - B 的闵可夫斯基差 = A + (-B), 然后求原点到该差的最小距离
T convexPolygonDistance(const vector<P>& A, const vector<P>& B) {
 vector<P> negB;
 for (auto& p : B) negB.pb(P(0, 0) - p);
 auto sum = minkowskiSum(A, negB);

 // 原点到凸多边形的最短距离
 T best = 1e30;
 int n = sz(sum);
 rep(i, 0, n) {
 P a = sum[i], b = sum[(i + 1) % n];
 P origin(0, 0);

 if ((b - a).dot(origin - a) <= 0) // 最近点是 a
 best = min(best, a.dist2());
 else if ((a - b).dot(origin - b) <= 0) // 最近点是 b
 best = min(best, b.dist2());
 else // 最近点在线段 a-b 上
 best = min(best, a.cross(b) * a.cross(b) / (b - a).dist2());
 }
 return sqrt(best);
}

```

## 7.10 三角形与圆 (Triangles and Circles)

**English:** Triangle and Circle Geometry | **Chinese:** 三角形与圆的几何运算

```
// ===== 三角形 =====

// 三角形面积 (海伦公式)
double triangleArea(double a, double b, double c) {
 double s = (a + b + c) / 2;
 return sqrt(s * (s - a) * (s - b) * (s - c));
}

// 三点坐标直接求面积
T triangleArea2(P a, P b, P c) { return fabs(a.cross(b, c)); }

// 外心 (外接圆圆心)
P circumCenter(P a, P b, P c) {
 T d = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y));
 T a2 = a.dot(a), b2 = b.dot(b), c2 = c.dot(c);
 return {(a2 * (b.y - c.y) + b2 * (c.y - a.y) + c2 * (a.y - b.y)) / d,
 (a2 * (c.x - b.x) + b2 * (a.x - c.x) + c2 * (b.x - a.x)) / d};
}

// 重心
P centroid(P a, P b, P c) { return {(a.x + b.x + c.x) / 3, (a.y + b.y + c.y) / 3}; }

// 内心
P inCenter(P a, P b, P c) {
 double da = (b - c).dist(), db = (c - a).dist(), dc = (a - b).dist();
 return {(a.x * da + b.x * db + c.x * dc) / (da + db + dc),
 (a.y * da + b.y * db + c.y * dc) / (da + db + dc)};
}

// 垂心
P orthoCenter(P a, P b, P c) {
 // 垂心 = 3 * 重心 - 2 * 外心
 P g = centroid(a, b, c);
 P o = circumCenter(a, b, c);
 return g * 3 - o * 2;
}

// 九点圆心 = 外心与垂心中点
P ninePointCenter(P a, P b, P c) {
 return (circumCenter(a, b, c) + orthoCenter(a, b, c)) / 2;
}

// ===== 圆 =====

// 直线与圆求交：返回两个交点 (可能 0、1、2 个)
// 直线由 a, b 两点确定
vector<P> circleLineIntersection(P center, double r, P a, P b) {
 P d = b - a;
 T f = (a - center).dot(d);
 T D = d.dist2();
 T disc = f * f - D * ((a - center).dist2() - r * r);
 if (disc < -EPS) return {}; // 无交点
 if (disc < EPS) { // 相切
 T t = -f / D;
 return {a + d * t};
 }
 // 两个交点
 T t1 = (-f - sqrt(disc)) / D;
 T t2 = (-f + sqrt(disc)) / D;
 return {a + d * t1, a + d * t2};
}

// 两圆求交 (面积)
double circleIntersectionArea(P c1, double r1, P c2, double r2) {
 double d = (c1 - c2).dist();
 if (d >= r1 + r2) return 0; // 相离
 if (d <= fabs(r1 - r2)) { // 内含
 double r = min(r1, r2);
 return M_PI * r * r;
 }
 double angle1 = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d));
 double angle2 = acos((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));
 return r1 * r1 * (angle1 - sin(angle1) * cos(angle1))
 + r2 * r2 * (angle2 - sin(angle2) * cos(angle2));
}

// 两圆交点
vector<P> circleCircleIntersection(P c1, double r1, P c2, double r2) {
 double d = (c1 - c2).dist();
 if (d > r1 + r2 + EPS || d < fabs(r1 - r2) - EPS) return {};
 if (d < EPS && fabs(r1 - r2) < EPS) return {}; // 重合 (无穷多交点)
 double a = (r1 * r1 - r2 * r2 + d * d) / (2 * d);
 double h = sqrt(max(0.0, r1 * r1 - a * a));
 P p = c1 + (c2 - c1) * (a / d);
 P perp = (c2 - c1).perp() * (h / d);
 if (h < EPS) return {p};
 return {p + perp, p - perp};
}

```

## 7.11 三维几何基础 (3D Geometry Basics)

English: 3D Geometry Primer | Chinese: 三维几何基础

```
// ===== 三维点结构 =====
using T3 = double;
const T3 EPS3 = 1e-9;

struct P3 {
 T3 x, y, z;
 P3(T3 x = 0, T3 y = 0, T3 z = 0) : x(x), y(y), z(z) {}
 P3 operator+(P3 o) const { return {x + o.x, y + o.y, z + o.z}; }
 P3 operator-(P3 o) const { return {x - o.x, y - o.y, z - o.z}; }
 P3 operator*(T3 d) const { return {x * d, y * d, z * d}; }
 P3 operator/(T3 d) const { return {x / d, y / d, z / d}; }
 T3 dot(P3 o) const { return x * o.x + y * o.y + z * o.z; }
 P3 cross(P3 o) const { return {y * o.z - z * o.y, z * o.x - x * o.z, x * o.y - y * o.x}; }
 T3 dist2() const { return x * x + y * y + z * z; }
 double dist() const { return sqrt(dist2()); }
};

// ===== 平面 =====
// 三点确定一个平面, 返回法向量 (单位化)
P3 planeNormal(P3 a, P3 b, P3 c) {
 P3 n = (b - a).cross(c - a);
 return n / n.dist();
}

// 点 p 到平面 a-b-c 的有向距离 (法向为正)
double pointToPlane(P3 p, P3 a, P3 b, P3 c) {
 P3 n = planeNormal(a, b, c);
 return n.dot(p - a);
}

// ===== 直线 =====
// 点 p 到线段 s-e 的最近点
P3 closestOnSegment(P3 p, P3 s, P3 e) {
 auto d = (e - s).dist2();
 auto t = max(0.0, min(1.0, (p - s).dot(e - s) / d));
 return s + (e - s) * t;
}

// ===== 直线与平面交点 =====
// 直线由 a + t * (b-a) 定义, 平面由 p0 + u*(p1-p0) + v*(p2-p0) 定义
P3 linePlaneIntersection(P3 a, P3 b, P3 p0, P3 p1, P3 p2) {
 P3 dir = b - a;
 P3 n = (p1 - p0).cross(p2 - p0);
 double denom = dir.dot(n);
 if (fabs(denom) < EPS3) return a; // 平行或不交
 double t = (p0 - a).dot(n) / denom;
 return a + dir * t;
}

// ===== 四面体体积 =====
// V = |(a-d) · ((b-d) × (c-d))| / 6
double tetrahedronVolume(P3 a, P3 b, P3 c, P3 d) {
 return fabs((a - d).dot((b - d).cross(c - d))) / 6.0;
}
}
```

## 7.12 极角排序详解 (Polar Angle Sort — Deep Dive)

**English:** Polar Angle Sorting | **Chinese:** 极角排序深入

竞赛中最常出错的环节之一。核心原则：叉积优先，atan2 备用。

```
// ===== 三分组法 (最稳健) =====
// 将平面按象限分成上下两组, 组内叉积排序
// 优点: 零误差 (整数坐标), 无需处理 -pi/pi 边界
auto polarCmp = [&](P a, P b) {
 // 上半平面 (含正 x 轴) → ha=true
 bool ha = (a.y > 0) || (a.y == 0 && a.x > 0);
 bool hb = (b.y > 0) || (b.y == 0 && b.x > 0);
 if (ha != hb) return ha > hb; // 上半平面在前
 T o = a.cross(b);
 if (o != 0) return o > 0; // CCW
 return a.dist2() < b.dist2(); // 共线时近的在前
};

// ===== atan2 排序 (浮点, 慎用) =====
// atan2 返回 [-pi, pi], 排序时 -pi 和 pi 相邻但排在一起不自然
// 仅在需要真正角度值时才用 atan2
auto atan2Cmp = [&](P a, P b) {
 double ang_a = atan2(a.y, a.x);
 double ang_b = atan2(b.y, b.x);
 if (fabs(ang_a - ang_b) > EPS) return ang_a < ang_b;
 return a.dist2() < b.dist2();
};

// ===== 整数极角排序 (零误差) =====
auto polarCmpInt = [&](iPoint a, iPoint b) {
 // 上半平面优先
 bool ha = (a.y > 0) || (a.y == 0 && a.x > 0);
 bool hb = (b.y > 0) || (b.y == 0 && b.x > 0);
 if (ha != hb) return ha > hb;
 ll o = a.cross(b);
 if (o != 0) return o > 0;
 return a.dist2() < b.dist2();
};
}
```

## 7.13 线段树 + 几何 (区间交点查询)

**English:** Segment Tree for Geometry Queries | **Chinese:** 线段树加速几何查询

当需要动态插入/删除线段并查询与某条线的交点时, 线段树是利器。

```

// 李超线段树 (Li Chao Tree) : 维护一族直线 $y = kx + b$, 支持 :
// - add_line(k, b) : 插入一条新直线
// - query(x) : 求所有直线在 x 处的最大值/最小值
// 0(log C) per operation, C 为 x 的值域范围
//
// 与计算几何的关联 : 直线族交点问题可转化为李超树 ;
// 线段族求交则需要线段树 + 扫描线。

struct LiChaoTree {
 struct Line {
 ll k, b;
 ll eval(ll x) const { return k * x + b; }
 };
 vector<Line> tree;
 int n, X_MIN, X_MAX;

 LiChaoTree(int minX, int maxX) : X_MIN(minX), X_MAX(maxX) {
 n = maxX - minX + 1;
 tree.assign(4 * n, {0, LINF}); // 最小值版本 : 最大值用 -LINF
 }

 void addLine(int idx, int l, int r, Line newLine) {
 int mid = (l + r) / 2;
 ll xl = l + X_MIN, xm = mid + X_MIN, xr = r + X_MIN;
 bool leftBetter = newLine.eval(xl) < tree[idx].eval(xl);
 bool midBetter = newLine.eval(xm) < tree[idx].eval(xm);

 if (midBetter) swap(tree[idx], newLine); // 中点更好 → 交换, 把旧的推下去
 if (l == r) return;

 if (leftBetter != midBetter)
 addLine(idx * 2, l, mid, newLine);
 else
 addLine(idx * 2 + 1, mid + 1, r, newLine);
 }

 void addLine(ll k, ll b) { addLine(1, 0, n - 1, {k, b}); }

 ll query(int idx, int l, int r, int x) {
 ll res = tree[idx].eval(x);
 if (l == r) return res;
 int mid = (l + r) / 2;
 if (x - X_MIN <= mid)
 res = min(res, query(idx * 2, l, mid, x));
 else
 res = min(res, query(idx * 2 + 1, mid + 1, r, x));
 return res;
 }

 ll query(int x) { return query(1, 0, n - 1, x); }
};

```

## 7.14 扫描线 (Sweep Line)

**English:** Sweep Line | **Chinese:** 扫描线

处理二维区间的通用框架：矩形面积并、矩形周长并、线段交点计数等。

```

// ===== 矩形面积并 (经典扫描线) =====
// 离散化 y 坐标 + 线段树维护有效长度
struct Event {
 T x, y1, y2;
 int type; // +1 左边界进入, -1 右边界离开
 bool operator<(const Event& o) const { return x < o.x; }
};

double rectangleUnionArea(const vector<tuple<T,T,T,T>>& rects) {
 // rects = {x1, y1, x2, y2}, 假设 x1<x2, y1<y2
 vector<T> ys;
 vector<Event> events;
 for (auto [x1, y1, x2, y2] : rects) {
 ys.pb(y1); ys.pb(y2);
 events.pb({x1, y1, y2, 1});
 events.pb({x2, y1, y2, -1});
 }
 sort(all(ys));
 ys.erase(unique(all(ys)), ys.end());
 sort(all(events));

 // 线段树维护 : 每个离散化后的 y 区间被覆盖的次数
 int m = sz(ys) - 1;
 vi cnt(4 * m);
 vector<double> len(4 * m);

 auto update = [&](auto&& self, int idx, int l, int r, int ql, int qr, int val) -> void {
 if (ql <= l && r <= qr) {
 cnt[idx] += val;
 } else {
 int mid = (l + r) / 2;
 if (ql <= mid) self(self, idx * 2, l, mid, ql, qr, val);
 if (qr > mid) self(self, idx * 2 + 1, mid + 1, r, ql, qr, val);
 }
 if (cnt[idx] > 0) len[idx] = ys[r + 1] - ys[l];
 else if (l == r) len[idx] = 0;
 else len[idx] = len[idx * 2] + len[idx * 2 + 1];
 };

 double ans = 0;
 rep(i, 0, sz(events) - 1) {
 int y1 = lower_bound(all(ys), events[i].y1) - ys.begin();
 int y2 = lower_bound(all(ys), events[i].y2) - ys.begin();
 update(update, 1, 0, m - 1, y1, y2 - 1, events[i].type);
 ans += len[1] * (events[i + 1].x - events[i].x);
 }
 return ans;
}

```



7.15 二/三维凸包进阶 (Advanced Hulls)

English: Advanced Convex Hull Topics | Chinese: 凸包进阶

```
// ===== 动态凸包 (允许在线插入点) =====
// 维持上下凸壳两个 std::set, O(log N) 插入
// 适用于需要动态维护凸包信息的问题
struct DynamicHull {
 set<P> upper, lower; // upper: 上凸壳; lower: 下凸壳

 // 检查点 b 是否在 a-c 构成的凸壳内部 (即 a-c 之间不需要 b)
 bool isBad(set<P>& hull, set<P>::iterator it) {
 if (it == hull.begin() || next(it) == hull.end()) return false;
 auto prev_it = prev(it), next_it = next(it);
 return (*prev_it).cross(*next_it, *it) >= 0; // 非凸
 }

 void insert(P p) {
 // 下凸壳
 auto it = lower.insert(p).first;
 if (isBad(lower, it)) { lower.erase(it); }
 else {
 while (it != lower.begin() && isBad(lower, prev(it)))
 lower.erase(prev(it));
 while (next(it) != lower.end() && isBad(lower, next(it)))
 lower.erase(next(it));
 }
 // 上凸壳: y 取反
 p.y = -p.y;
 it = upper.insert(p).first;
 if (isBad(upper, it)) { upper.erase(it); }
 else {
 while (it != upper.begin() && isBad(upper, prev(it)))
 upper.erase(prev(it));
 while (next(it) != upper.end() && isBad(upper, next(it)))
 upper.erase(next(it));
 }
 }
};

// ===== 三维凸包 (增量法) =====
// O(N^2) 三维凸包: 返回所有面 (三角形), Face 用三个点的索引表示
struct Face {
 int a, b, c;
 P3 normal(const vector<P3>& pts) const {
 return (pts[b] - pts[a]).cross(pts[c] - pts[a]);
 }
};

vector<Face> convexHull3D(vector<P3> pts) {
 int n = sz(pts);
 if (n < 4) return {};

 // 先找一个初始四面体
 // 简化版: 假设前四点不共面

 vector<Face> faces;
 vector<vector<int>> adj(n);

 // Floyd 增量法: 初始一个四面体, 逐步加入点
 // 每加入一个点, 移除可见面, 补上新的面
 // (完整实现约 80 行, 此处给出框架)
 // 对应 OI-wiki / KACTL 3d-hull 模板

 // [完整实现请参考 KACTL: Point3D.h]
 return faces;
}
```

比赛技巧 (Contest Tips for Geometry)

模板速查表 (Template Quick Reference)

算法	英文	复杂度	推荐实现	关键点
凸包	Convex Hull	O(N log N)	Andrew 单调链	<= 0 排除共线点; < 0 保留
半平面交	Half-plane Intersection	O(N log N)	S&I 双端队列	必须加包围盒, 注意平行线处理
旋转卡壳直径	Rotating Calipers Diameter	O(N)	对踵点扫描	输入必须是严格 CCW 凸包
最小外接矩形	Min Bounding Rectangle	O(N)	三指针旋转卡壳	维护 j(高), l(左投影), r(右投影)
最近点对	Closest Pair	O(N log N)	扫面线 / 分治	扫面线版更简洁, 带内候选 ≤ 6 个
最小圆覆盖	Min Enclosing Circle	O(N) 期望	Welzl 随机增量	随机化是 O(N) 的保证
点在多边形内	Point in Polygon	O(N)	射线法	半开区间处理避免顶点重数
点在凸多边形内	Point in Convex Polygon	O(log N)	二分定位	先判最外扇形, 再二分
线段相交	Segment Intersection	O(1)	跨立实验	区分严格相交、端点接触、共线重叠
多边形面积	Polygon Area	O(N)	鞋带公式	二倍有向面积避除法
闵可夫斯基和	Minkowski Sum	O(N+M)	双指针归并	要求 CCW + 最低 leftmost 在 first
凸多边形距离	Convex Polygon Distance	O(N+M)	Minkowski 差	求原点到差的最短距离
多边形切割	Polygon Cut	O(N)	逐边处理	保留直线左侧部分
圆与直线交点	Circle-Line Intersection	O(1)	判别式	相切/相交/相离三种情况
两圆交面积	Circle Intersection Area	O(1)	扇形-三角形	内含/相切/相交三种情况
三维凸包	3D Convex Hull	O(N^2)	增量法	参考 KACTL Point3D.h

8 网络流

网络流是图论中最庞大的模块之一，涵盖最大流、最小割、费用流、匹配等问题。在竞赛中，网络流题目往往难在建图而非算法本身——掌握常见建模技巧比背模板更重要。

8.1 Dinic 最大流

最大流问题：给定有向图  $G=(V,E)G=(V,E)$ ，每条边  $(u,v)(u,v)$  有容量  $c(u,v)c(u,v)$ ，求从源点  $ss$  到汇点  $tt$  的最大流量。

Dinic 算法是竞赛中最常用的最大流算法，核心思想是 分层图 (level graph) + 阻塞流 (blocking flow) + 当前弧优化。

## 关键优化

优化	说明
当前弧优化	<code>ptr[u]</code> 记录每个节点下一次该尝试的边，避免重复扫描已满流的边
多路增广	DFS 一次尽可能多地推送流量，而非每次只找一条增广路
边对存储	正向边和反向边相邻存储（ <code>rev</code> 字段或 <code>tot^1</code> 技巧），保证 $O(1)O(1)$ 取反向边
反向流量守恒	正向边 <code>flow += f</code> ，反向边 <code>flow -= f</code> ，始终满足 <code>fwd.flow + rev.flow == 0</code>

### 8.1.1 标准 Dinic（BFS + DFS + 当前弧优化，`rev` 字段版本）

复杂度： $O(V^2E)O(V^2E)$  一般图， $O(EV)O(E\sqrt{V})$  单位容量图，实际运行远小于理论界。

```
/**
 * Dinic 最大流 — 标准 BFS + DFS 版本
 * 时间复杂度： $O(V^2 * E)$ ，单位容量： $O(E * \sqrt{V})$
 * 存储：rev 字段指向反向边下标
 */
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;

struct Dinic {
 struct Edge {
 int to, rev; // 目标节点，反向边在 g[to] 中的下标
 ll cap; // 剩余容量（直接用 cap 表示残量，不需要单独的 flow 字段）
 };

 int n;
 vector<vector<Edge>> g;
 vector<int> level, ptr; // 分层标记，当前弧指针

 Dinic(int n_) : n(n_), g(n_), level(n_), ptr(n_) {}

 // 添加有向边 u→v，容量为 cap
 void add_edge(int u, int v, ll cap) {
 g[u].push_back({v, (int)g[v].size(), cap});
 g[v].push_back({u, (int)g[u].size() - 1, 0}); // 反向边容量为 0
 }

 // 添加无向边 u-v，容量为 cap
 void add_undirected(int u, int v, ll cap) {
 g[u].push_back({v, (int)g[v].size(), cap});
 g[v].push_back({u, (int)g[u].size() - 1, cap}); // 反向边容量也为 cap
 }

 // BFS 构建分层图，返回汇点 t 是否可达
 bool bfs(int s, int t) {
 fill(level.begin(), level.end(), -1);
 queue<int> q;
 level[s] = 0;
 q.push(s);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (const Edge &e : g[u]) {
 if (level[e.to] == -1 && e.cap > 0) {
 level[e.to] = level[u] + 1;
 q.push(e.to);
 }
 }
 }
 return level[t] != -1;
 }

 // DFS 多路增广：在分层图上向 t 推送流量
 ll dfs(int u, int t, ll pushed) {
 if (u == t) return pushed;
 // 当前弧优化：ptr[u] 之前的所有边都已满流，跳过
 for (int &i = ptr[u]; i < (int)g[u].size(); ++i) {
 Edge &e = g[u][i];
 if (level[e.to] != level[u] + 1 || e.cap <= 0) continue;
 ll tr = dfs(e.to, t, min(pushed, e.cap));
 if (tr == 0) continue;
 e.cap -= tr;
 g[e.to][e.rev].cap += tr; // 反向边增加容量
 return tr;
 }
 return 0;
 }

 // 求解最大流
 ll max_flow(int s, int t) {
 ll flow = 0;
 while (bfs(s, t)) {
 fill(ptr.begin(), ptr.end(), 0); // 每次分层后重置当前弧
 while (ll pushed = dfs(s, t, INF))
 flow += pushed;
 }
 return flow;
 }

 // 获取最小割的 S 集合（从 s 出发沿残量网络可达的点）
 vector<bool> min_cut(int s) {
 vector<bool> vis(n, false);
 queue<int> q;
 q.push(s); vis[s] = true;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (const Edge &e : g[u]) {
 if (!vis[e.to] && e.cap > 0) {
 vis[e.to] = true;
 q.push(e.to);
 }
 }
 }
 return vis;
 }
};
```

### 8.1.2 tot^1 成对存储版本

另一种常用写法是将正向边和反向边存储在相邻位置（`edges[i]` 和 `edges[i^1]`），用 `tot` 作为全局边计数器。优点是不需要存储 `rev` 字段，代码更紧凑。

```
/**
 * Dinic 最大流 - tot^1 成对存储版本
 * 正向边编号偶数 (i), 反向边编号奇数 (i^1)
 */
struct Dinic_TotTrick {
 struct Edge {
 int to; ll cap; int nxt; // 链式前向星
 };

 int n, s, t;
 vector<Edge> edges;
 vector<int> head; // head[u]: 节点 u 的第一条出边编号
 vector<int> level, ptr;
 int tot; // 边的总数 (每条无向边贡献 2)

 Dinic_TotTrick(int n_) : n(n_), head(n_, -1), level(n_), ptr(n_), tot(0) {
 edges.reserve(200000); // 预分配, 减少扩容
 }

 void add_edge(int u, int v, ll cap) {
 edges.push_back({v, cap, head[u]});
 head[u] = tot++;
 edges.push_back({u, 0, head[v]});
 head[v] = tot++;
 }

 bool bfs() {
 fill(level.begin(), level.end(), -1);
 queue<int> q;
 level[s] = 0; q.push(s);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int i = head[u]; i != -1; i = edges[i].nxt) {
 int v = edges[i].to;
 if (level[v] == -1 && edges[i].cap > 0) {
 level[v] = level[u] + 1;
 q.push(v);
 }
 }
 }
 return level[t] != -1;
 }

 ll dfs(int u, ll pushed) {
 if (u == t) return pushed;
 for (int &i = ptr[u]; i != -1; i = edges[i].nxt) {
 int v = edges[i].to;
 if (level[v] != level[u] + 1 || edges[i].cap <= 0) continue;
 ll tr = dfs(v, min(pushed, edges[i].cap));
 if (tr == 0) continue;
 edges[i].cap -= tr;
 edges[i ^ 1].cap += tr; // tot^1 技巧取反向边
 return tr;
 }
 return 0;
 }

 ll max_flow(int s_, int t_) {
 s = s_; t = t_;
 ll flow = 0;
 while (bfs()) {
 for (int i = 0; i < n; ++i) ptr[i] = head[i]; // 当前弧指向第一条边
 while (ll pushed = dfs(s, INF))
 flow += pushed;
 }
 return flow;
 }
};
```

### 8.1.3 KACTL 缩放 Dinic (Scaling Dinic)

KACTL 的 Dinic 额外引入 **容量缩放 (capacity scaling)**：从大到小枚举阈值 `lim`，每轮只考虑容量  $\geq \lim$  的边。这使得算法在遍历邻接表时可以提前跳过不够大的边，在大容量图上通常比标准 Dinic 更快。

核心：

```

/**
 * KACTL Dinic - 容量缩放版本 (Scaling Dinic)
 * 在标准 Dinic 上增加容量阈值 lim, 每次只考虑 cap >= lim 的边
 * 时间复杂度: $O(V^2 * E * \log(C_{max}))$, 实践中常优于标准 Dinic
 *
 * 使用 >> (30 - L) 设定初始 lim:
 * lim = 1 << 30 适用于 int 容量 (约 10^9)
 * lim = 1LL << 60 适用于 long long 容量
 * 或取 max_cap 的最高位: lim = 1LL << (63 - __builtin_clzll(max_cap))
 */
struct DinicScaling {
 struct Edge {
 int to, rev;
 ll cap;
 };

 int n;
 vector<vector<Edge>> g;
 vector<int> level, ptr;

 DinicScaling(int n_) : n(n_), g(n_), level(n_), ptr(n_) {}

 void add_edge(int u, int v, ll cap) {
 g[u].push_back({v, (int)g[v].size(), cap});
 g[v].push_back({u, (int)g[u].size() - 1, 0});
 }

 // 分层 BFS, 只考虑 cap >= lim 的边
 bool bfs(int s, int t, ll lim) {
 fill(level.begin(), level.end(), -1);
 queue<int> q;
 level[s] = 0; q.push(s);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (const Edge &e : g[u]) {
 if (level[e.to] == -1 && e.cap >= lim) {
 level[e.to] = level[u] + 1;
 q.push(e.to);
 }
 }
 }
 return level[t] != -1;
 }

 // 多路增广, 只考虑 cap >= lim 的边
 ll dfs(int u, int t, ll pushed, ll lim) {
 if (u == t) return pushed;
 for (int &i = ptr[u]; i < (int)g[u].size(); ++i) {
 Edge &e = g[u][i];
 if (level[e.to] != level[u] + 1 || e.cap < lim) continue;
 ll tr = dfs(e.to, t, min(pushed, e.cap), lim);
 if (tr == 0) continue;
 e.cap -= tr;
 g[e.to][e.rev].cap += tr;
 return tr;
 }
 return 0;
 }

 // 缩放最大流
 ll max_flow(int s, int t, ll max_cap = 0) {
 ll flow = 0;
 // 计算初始 lim: 若未提供 max_cap, 默认为 $1 << 30$ (约 10^9)
 // 若提供了 max_cap, 取不小于它的最小 2 的幂
 if (max_cap == 0) max_cap = 1LL << 30;
 else {
 while (max_cap & (max_cap - 1)) // 不是 2 的幂则取下一个
 max_cap += max_cap & -max_cap;
 max_cap = min(max_cap, 1LL << 60);
 }

 for (ll lim = max_cap; lim > 0; lim >= 1) {
 while (bfs(s, t, lim)) {
 fill(ptr.begin(), ptr.end(), 0);
 while (ll pushed = dfs(s, t, INF, lim))
 flow += pushed;
 }
 }
 return flow;
 }

 vector<bool> min_cut(int s) {
 vector<bool> vis(n, false);
 queue<int> q;
 q.push(s); vis[s] = true;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (const Edge &e : g[u]) {
 if (!vis[e.to] && e.cap > 0) {
 vis[e.to] = true;
 q.push(e.to);
 }
 }
 }
 return vis;
 }
};

```

## 8.2 MCMF 最小费用最大流

在最大流问题的每条边上附加一个单位费用  $w(u,v)w(u,v)$ , 求在流量最大的前提下总费用最小的流。也可以指定目标流量 FF, 求达到 FF 的最小费用。

### 核心挑战

费用流中最关键的问题是 **负权边**——如果用 SPFA 每次找最短增广路, 最坏  $O(FVE)O(FVE)$ , 容易被卡。Johnson 势能法可以消除负权边, 之后全部用 Dijkstra。

## 算法流程

## 复杂度

$O(F \cdot E \log V)$  ( $F \cdot E \log V$ ),  $FF$  为最大流量。若要求特定流量  $K \leq F \leq F$ , 则  $O(K \cdot E \log V)$  ( $K \cdot E \log V$ )。



```

/**
 * MCMF - 最小费用最大流 (Johnson 势能 + Dijkstra)
 * 支持负权边: 通过 Bellman-Ford 初始化势能
 * 时间复杂度: $O(F * E * \log V)$
 */
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;

struct MCMF {
 struct Edge {
 int to, rev;
 ll cap, cost; // 剩余容量, 单位费用
 };

 int n;
 vector<vector<Edge>> g;
 vector<ll> pot; // Johnson 势能数组

 MCMF(int n_) : n(n_), g(n_), pot(n_, 0) {}

 // 添加有向边 u->v, 容量 cap, 单位费用 cost
 void add_edge(int u, int v, ll cap, ll cost) {
 g[u].push_back({v, (int)g[v].size(), cap, cost});
 g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
 }

 /**
 * 用 Bellman-Ford (SPFA) 初始化势能
 * 当图中存在负权边时, 必须在 solve() 前调用一次
 * 若图中无边权为负 (或已知无负边), 可跳过此步 (pot=0 即可)
 */
 bool init_potentials(int s) {
 fill(pot.begin(), pot.end(), INF);
 vector<bool> inq(n, false);
 vector<int> cnt(n, 0); // 入队次数, 检测负环
 queue<int> q;

 pot[s] = 0; q.push(s); inq[s] = true; cnt[s] = 1;

 while (!q.empty()) {
 int u = q.front(); q.pop(); inq[u] = false;
 for (const Edge &e : g[u]) {
 if (e.cap <= 0) continue;
 if (pot[e.to] > pot[u] + e.cost) {
 pot[e.to] = pot[u] + e.cost;
 if (!inq[e.to]) {
 q.push(e.to); inq[e.to] = true;
 if (++cnt[e.to] > n) return false; // 负环, MCMF 无解
 }
 }
 }
 }
 return true;
 }

 /**
 * 求解最小费用最大流
 * @param flow_limit 目标流量上限 (默认 INF 表示跑满最大流)
 * @return {总流量, 总费用}
 */
 pair<ll, ll> solve(int s, int t, ll flow_limit = INF) {
 ll flow = 0, cost = 0;
 vector<ll> dist(n);
 vector<int> parent(n), p_idx(n);

 while (flow < flow_limit) {
 // Dijkstra 在修正后非负权图上求最短路
 fill(dist.begin(), dist.end(), INF);
 using pii = pair<ll, int>;
 priority_queue<pii, vector<pii>, greater<pii>> pq;

 dist[s] = 0;
 pq.push({0, s});

 while (!pq.empty()) {
 auto [d, u] = pq.top(); pq.pop();
 if (d != dist[u]) continue; // 懒惰删除
 for (int i = 0; i < (int)g[u].size(); ++i) {
 Edge &e = g[u][i];
 if (e.cap <= 0) continue;
 // 修正边权保证非负
 ll nd = dist[u] + e.cost + pot[u] - pot[e.to];
 if (nd < dist[e.to]) {
 dist[e.to] = nd;
 parent[e.to] = u;
 p_idx[e.to] = i;
 pq.push({nd, e.to});
 }
 }
 }

 if (dist[t] == INF) break; // 无法继续增广

 // 更新势能
 for (int i = 0; i < n; ++i) {
 if (dist[i] != INF) pot[i] += dist[i];
 }

 // 沿路径最大限度推流
 ll pushed = flow_limit - flow;
 for (int v = t; v != s; v = parent[v]) {
 Edge &e = g[parent[v]][p_idx[v]];
 pushed = min(pushed, e.cap);
 }
 for (int v = t; v != s; v = parent[v]) {
 Edge &e = g[parent[v]][p_idx[v]];
 e.cap -= pushed;
 g[v][e.rev].cap += pushed;
 cost += pushed * e.cost; // 用原始费用累加
 }
 flow += pushed;
 }
 }
};

```

```

 }
 return {flow, cost};
}
};

// ===== 使用示例 =====
// MCMF mcmf(n);
// mcmf.add_edge(u, v, cap, cost); // 建图
// if (has_negative_edges)
// mcmf.init_potentials(s); // 有负权才调用
// auto [max_f, min_c] = mcmf.solve(s, t);
// auto [f, c] = mcmf.solve(s, t, K); // 指定目标流量 K

```

### 8.3 匈牙利算法（指派问题）

匈牙利算法求解二分图上的 **最优指派问题 (Assignment Problem)**: 给定  $n \times n$  的代价矩阵  $a_{ij}$ , 为每行分配一个不同的列, 使得总代价最小 (或最大)。



适用场景

关键技巧

```

/**
 * 匈牙利算法 - 指派问题 $O(n^3)$
 * 求 $n \times n$ 代价矩阵的最小总代价指派 (每行配一列)
 * 求最大指派: 将所有 cost 取负
 *
 * 使用: Hungarian hung(n, m, cost, want_minimum);
 * hung.solve() 返回 {最小总代价, 匹配方案 match[row]=col}
 */
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;

struct Hungarian {
 int n, m; // n 行 (左部), m 列 (右部), 处理后 $n=m$
 vector<vector<ll>> a; // 代价矩阵, 会被修改
 vector<ll> u, v; // 行顶标, 列顶标
 vector<int> p, way; // p[col]: 匹配的行, way[col]: 前驱列
 vector<int> match; // match[row] = 配对的列

 /**
 * @param n_ 行数
 * @param m_ 列数
 * @param cost 代价矩阵 ($n_ \times m_$)
 * @param minimum true=最小总代价, false=最大总代价
 */
 Hungarian(int n_, int m_, const vector<vector<ll>> &cost, bool minimum = true)
 : n(n_), m(m_), a(cost) {
 // 补成方阵
 int sz = max(n, m);
 a.resize(sz, vector<ll>(sz, INF));
 for (int i = 0; i < sz; ++i) a[i].resize(sz, INF);
 // 补的行/列代价为 0 (用 0 而非 INF 以保证总能有可行指派)
 for (int i = 0; i < sz; ++i)
 for (int j = 0; j < sz; ++j)
 if (i >= n || j >= m) a[i][j] = 0;

 // 求最大指派则取负
 if (!minimum) {
 for (int i = 0; i < sz; ++i)
 for (int j = 0; j < sz; ++j)
 if (a[i][j] != INF) a[i][j] = -a[i][j];
 }

 n = m = sz;
 u.assign(n + 1, 0);
 v.assign(m + 1, 0);
 p.assign(m + 1, 0);
 way.assign(m + 1, 0);
 match.assign(n + 1, 0);
 }

 // 返回 {最小/最大总代价, 匹配方案}
 // match[row] = 配对的列 (1-indexed rows, 1-indexed cols)
 // 若 match[row] > 原始 m, 说明该行配到了补的虚点
 pair<ll, vector<int>> solve() {
 for (int i = 1; i <= n; ++i) {
 p[0] = i;
 int j0 = 0;
 vector<ll> minv(m + 1, INF);
 vector<bool> used(m + 1, false);

 do {
 used[j0] = true;
 int i0 = p[j0]; // 当前考察的行
 ll delta = INF;
 int j1 = 0;

 // 遍历所有列, 找最小松弛量
 for (int j = 1; j <= m; ++j) {
 if (used[j]) continue;
 ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
 if (cur < minv[j]) {
 minv[j] = cur;
 way[j] = j0;
 }
 if (minv[j] < delta) {
 delta = minv[j];
 j1 = j;
 }
 }
 } while (j1 != 0);

 // 更新顶标
 for (int j = 0; j <= m; ++j) {
 if (used[j]) {
 u[p[j]] += delta;
 v[j] -= delta;
 } else {
 minv[j] -= delta;
 }
 }
 j0 = j1;
 } while (p[j0] != 0);

 // 沿 way 数组做增广翻转
 do {
 int j1 = way[j0];
 p[j0] = p[j1];
 j0 = j1;
 } while (j0 != 0);

 // 生成答案
 for (int j = 1; j <= m; ++j)
 if (p[j] != 0) match[p[j]] = j;

 // 总代价 = -v[0] (注意我们的顶标符号约定)
 // 实际总代价用 p[j] 和 j 算
 ll ans = -v[0]; // 等价于 sum 配对代价

 // 更直接的计算方式
 ans = 0;
 for (int i = 1; i <= n; ++i)

```

```

 if (match[i] != 0) ans += a[i - 1][match[i] - 1];
 return {ans, match};
 }
};

// ===== 使用示例 =====
// vector<vector<ll>> cost = {
// {4, 1, 3},
// {2, 0, 5},
// {3, 2, 2},
// };
// Hungarian hung(3, 3, cost, true); // true = 最小代价
// auto [min_cost, match] = hung.solve();
// // min_cost = 最小总代价 (例: 1+2+2=5)
// // match[row] = 该行配的列 (1-indexed)

```

## 8.4 Hopcroft-Karp 算法（二分图最大匹配）

Hopcroft-Karp (HK) 算法是 无权二分图最大匹配的最优选择，复杂度  $O(EV)O(E\sqrt{V})$ 。比匈牙利  $O(VE)O(VE)$  更优，尤其适合稀疏的大规模二分图（ $V \leq 10^5, E \leq 10^5$ ）。

### 算法思想

与 Dinic 类似，HK 也使用分层 + 阻塞匹配的思路：

```

/**
 * Hopcroft-Karp - 二分图最大匹配 $O(E * \sqrt{V})$
 * 左部: $0..n-1$, 右部: $0..m-1$
 * 使用:
 * HK hk(n, m);
 * hk.add_edge(left, right);
 * int match_cnt = hk.max_match();
 * auto [matchL, matchR] = hk.get_match();
 */
#include <bits/stdc++.h>
using namespace std;

struct HopcroftKarp {
 int n, m; // 左部大小 n, 右部大小 m
 vector<vector<int>> g; // 左部 -> 右部的邻接表
 vector<int> matchL, matchR; // 匹配记录 (-1 表示未匹配)
 vector<int> dist; // BFS 距离 (左部)

 HopcroftKarp(int n_, int m_) : n(n_), m(m_), g(n_),
 matchL(n_, -1), matchR(m_, -1), dist(n_) {}

 void add_edge(int u, int v) {
 // u in [0, n-1], v in [0, m-1]
 g[u].push_back(v);
 }

 // BFS 构建分层图, 返回是否存在增广路
 bool bfs() {
 queue<int> q;
 for (int u = 0; u < n; ++u) {
 if (matchL[u] == -1) {
 dist[u] = 0;
 q.push(u);
 } else {
 dist[u] = -1;
 }
 }

 bool found = false;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v : g[u]) {
 int nxt = matchR[v];
 if (nxt != -1 && dist[nxt] == -1) {
 dist[nxt] = dist[u] + 1;
 q.push(nxt);
 } else if (nxt == -1) {
 found = true; // 找到未匹配的右部点
 }
 }
 }
 return found;
 }

 // DFS 沿分层图增广
 bool dfs(int u) {
 for (int v : g[u]) {
 int nxt = matchR[v];
 if (nxt == -1 || (dist[nxt] == dist[u] + 1 && dfs(nxt))) {
 matchL[u] = v;
 matchR[v] = u;
 return true;
 }
 }
 dist[u] = -1; // 此点已废, 标记避免重复访问
 return false;
 }

 // 返回最大匹配数
 int max_match() {
 int res = 0;
 while (bfs()) {
 for (int u = 0; u < n; ++u) {
 if (matchL[u] == -1 && dfs(u))
 ++res;
 }
 }
 return res;
 }

 // 获取匹配方案
 pair<vector<int>, vector<int>> get_match() {
 return {matchL, matchR};
 }

 // 最小点覆盖 = {未被 HK DFS 访问到的左部点} U {被 DFS 访问到的右部点}
 // 求最小点覆盖需在 max_match() 后调用
 pair<vector<int>, vector<int>> min_vertex_cover() {
 // 从未匹配的左部点出发做交错 BFS
 vector<bool> visL(n, false), visR(m, false);
 queue<int> q;
 for (int u = 0; u < n; ++u) {
 if (matchL[u] == -1) {
 visL[u] = true;
 q.push(u);
 }
 }
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v : g[u]) {
 if (!visR[v] && matchR[v] != u) {
 visR[v] = true;
 if (matchR[v] != -1 && !visL[matchR[v]]) {
 visL[matchR[v]] = true;
 q.push(matchR[v]);
 }
 }
 }
 }
 // 左部未访问的点 ∈ 最小点覆盖, 右部访问过的点 ∈ 最小点覆盖
 vector<int> coverL, coverR;
 for (int u = 0; u < n; ++u)
 if (!visL[u]) coverL.push_back(u);
 for (int v = 0; v < m; ++v)

```

```
 if (visR[v]) coverR.push_back(v);
 return {coverL, coverR};
 }
};
```

二分图匹配方案速查

问题	转化方式
最大匹配	HK 或 Dinic
最小点覆盖	Konig 定理: 最小点覆盖 = 最大匹配, 方案由交替 BFS 构造
最大独立集	总点数 - 最大匹配 (= 总点数 - 最小点覆盖)
最小边覆盖	若无孤立点 = 总点数 - 最大匹配
DAG 最小路径覆盖	拆点二分图, 顶点数 - 最大匹配
带权匹配	KM 算法 $O(V^3)$ 或 MCMF

8.5 最小割应用

8.5.1 最大权闭合子图 (Maximum Weight Closure)

问题: 给定有向图, 每个点有权值  $w_i$  (可正可负)。选出一个点集  $S$ , 满足闭合性  $(\forall u \in S, (u \rightarrow v) \in E \implies v \in S)$  for all  $u \in S, (u \rightarrow v) \in E \implies v \in S$ , 且  $\sum_{i \in S} w_i$  最大。

建模:

经典应用: 选项目有收益/成本, 选 A 必须选 B。

```
/**
 * 最大权闭合子图
 * max_profit = sum(正权) - min_cut(s,t)
 */
struct MaxWeightClosure {
 int n;
 vector<ll> weight;
 Dinic dinic; // 复用 8.1.1 的 Dinic
 int s, t;

 MaxWeightClosure(int n_, const vector<ll> &w)
 : n(n_), weight(w), dinic(n_ + 2), s(n_), t(n_ + 1) {}

 // 添加约束: 选 u 必须选 v (即 u -> v 有边)
 void add_implication(int u, int v) {
 dinic.add_edge(u, v, INF);
 }

 /**
 * solve: 返回最大权值 + 选中的点集
 */
 pair<ll, vector<int>> solve() {
 ll sum_pos = 0;
 for (int i = 0; i < n; ++i) {
 if (weight[i] > 0) {
 dinic.add_edge(s, i, weight[i]);
 sum_pos += weight[i];
 } else if (weight[i] < 0) {
 dinic.add_edge(i, t, -weight[i]);
 }
 }

 ll min_cut_val = dinic.max_flow(s, t);
 ll max_val = sum_pos - min_cut_val;

 // 提取 S 集中的点 (即最优方案选的点)
 vector<bool> in_s = dinic.min_cut(s);
 vector<int> selected;
 for (int i = 0; i < n; ++i)
 if (in_s[i]) selected.push_back(i);

 return {max_val, selected};
 }
};
```

8.5.2 最小路径覆盖 (DAG 最小不相交路径覆盖)

问题: 在 DAG 中, 用最少的路径覆盖所有顶点, 路径之间互不相交。

建模:

可相交版本: 先用 Floyd 求传递闭包, 再对闭包图做不相交覆盖。

```

/**
 * 最小路径覆盖 (Minimum Path Cover for DAG)
 * min_paths = n - max_matching
 * 返回路径数 + 每条路径的节点序列
 */
struct MinPathCover {
 int n;
 vector<vector<int>> g; // DAG 邻接表
 vector<int> match_to, match_f; // match_to[v] = 哪个左部点匹配了右部 v
 vector<bool> vis;

 MinPathCover(int n_) : n(n_), g(n_) {}

 void add_edge(int u, int v) { g[u].push_back(v); }

 bool dfs(int u) {
 for (int v : g[u]) {
 if (vis[v]) continue;
 vis[v] = true;
 if (match_to[v] == -1 || dfs(match_to[v])) {
 match_to[v] = u;
 match_f[u] = v;
 return true;
 }
 }
 return false;
 }

 // 返回 {最少路径数, 每条路径的节点列表}
 pair<int, vector<vector<int>>> solve() {
 match_to.assign(n, -1);
 match_f.assign(n, -1);
 int matching = 0;

 for (int u = 0; u < n; ++u) {
 vis.assign(n, false);
 if (dfs(u)) ++matching;
 }

 int min_paths = n - matching;

 // 重构路径
 vector<bool> is_start(n, true);
 for (int v = 0; v < n; ++v)
 if (match_to[v] != -1) is_start[v] = false;

 vector<vector<int>> paths;
 vector<bool> used(n, false);
 for (int i = 0; i < n; ++i) {
 if (!is_start[i] || used[i]) continue;
 vector<int> path;
 int cur = i;
 while (cur != -1 && !used[cur]) {
 used[cur] = true;
 path.push_back(cur);
 cur = match_f[cur];
 }
 paths.push_back(path);
 }
 return {min_paths, paths};
 }
};

// ===== 可相交最小路径覆盖 =====
// 先用 Floyd 求传递闭包, 再做不相交覆盖
struct MinPathCoverIntersecting {
 int n;
 vector<vector<bool>> reach; // 传递闭包

 MinPathCoverIntersecting(int n_) : n(n_), reach(n_, vector<bool>(n_, false)) {}

 void add_edge(int u, int v) { reach[u][v] = true; }

 pair<int, vector<vector<int>>> solve() {
 // Floyd 传递闭包
 for (int k = 0; k < n; ++k)
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (reach[i][k] && reach[k][j])
 reach[i][j] = true;

 MinPathCover mpc(n);
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (reach[i][j] && i != j) mpc.add_edge(i, j);

 return mpc.solve();
 }
};

```

## 8.6 上下界网络流

### 问题描述

每条边  $ee$  不仅有容量上界  $c(e)c(e)$ , 还有流量下界  $l(e)l(e)$ , 要求  $l(e) \leq f(e) \leq c(e)l(e) \vee e f(e) \vee e c(e)$ 。在此基础上求可行流 / 最大流 / 最小流。

### 核心转化

对于每条边  $(u, v, l, r)(u, v, l, r)$  (下界  $ll$ , 上界  $rr$ ) :

三类问题解法

问题	步骤
无源汇可行流	跑 $S' \rightarrow T'S' \rightarrow T'$ 最大流，若满流则可行
有源汇可行流	加 $t \rightarrow st \rightarrow to s$ (容量 $\infty$ ), 转为无源汇
有源汇最大流	先求可行流，然后去掉 $t \rightarrow st \rightarrow to s$ 边及 $S', T'S', T'$ ，在原残量网络上跑 $s \rightarrow ts \rightarrow to t$ 最大流
有源汇最小流	先求可行流，然后去掉 $t \rightarrow st \rightarrow to s$ 边，跑 $t \rightarrow st \rightarrow to s$ 最大流（逆向），减去即可

```

/**
 * 上下界网络流 (Bounded Flow)
 * 复用 8.1.1 的 Dinic
 * 使用：
 * BoundedFlow bf(n); // n = 原图节点数
 * bf.add_edge(u, v, lower, upper); // 添加有上下界的边
 * bf.feasible(); // 无源汇可行流
 * bf.feasible(s, t); // 有源汇可行流
 * bf.max_flow_bounded(s, t); // 有源汇最大流
 * bf.min_flow_bounded(s, t); // 有源汇最小流
 * bf.get_flow(i); // 获取第 i 条边的实际流量
 */
struct BoundedFlow {
 struct BEdge {
 int u, v; ll lower, upper;
 };

 int n; // 原图节点数
 vector<BEdge> edges; // 记录每条边的上下界
 Dinic dinic; // 内部 Dinic (n + 2 个节点: 原图 + S' + T')
 vector<ll> balance; // balance[i] = 流入下界 - 流出下界
 int S, T; // 超级源 S, 超级汇 T
 int t_to_s_id; // 辅助边 t->s 的编号 (有源汇时用)
 vector<ll> lower_sum; // 每条边下界的累积 (用于计算实际流量)

 BoundedFlow(int n_) : n(n_), dinic(n_ + 2), balance(n_ + 2, 0) {
 S = n_; T = n_ + 1;
 }

 // 添加有上下界的边
 void add_edge(int u, int v, ll lower, ll upper) {
 edges.push_back({u, v, lower, upper});
 balance[v] += lower;
 balance[u] -= lower;
 // 在 Dinic 中建容量为 upper - lower 的边
 dinic.add_edge(u, v, upper - lower);
 }

 // 编译 S' 和 T' 的补偿边, 返回需要的总流出流量
 ll build_super_graph() {
 ll total_demand = 0;
 for (int i = 0; i < n; ++i) {
 if (balance[i] > 0) {
 dinic.add_edge(S, i, balance[i]);
 total_demand += balance[i];
 } else if (balance[i] < 0) {
 dinic.add_edge(i, T, -balance[i]);
 }
 }
 return total_demand;
 }

 // ---- 无源汇可行流 ----
 bool feasible() {
 ll total = build_super_graph();
 return dinic.max_flow(S, T) == total;
 }

 // ---- 有源汇可行流 ----
 bool feasible(int s, int t) {
 // 添加辅助边 t->s, 下界 0, 上界 INF
 dinic.add_edge(t, s, INF);
 return feasible();
 }

 // ---- 有源汇最大流 ----
 // 返回值: 最大流量; 若不可行返回 -1
 ll max_flow_bounded(int s, int t) {
 // Step 1: 求可行流
 dinic.add_edge(t, s, INF);
 ll total = build_super_graph();
 ll check = dinic.max_flow(S, T);
 if (check != total) return -1; // 无可行流

 // Step 2: 此时 dinic 处于残量网络状态
 // t->s 边的流量就是可行流在 s->t 上的流量
 // 找 t->s 边上的流量 (即反向边 s->t 的 cap 减少量或 t->s 的 cap 减少量)
 // 简便做法: 删掉 t->s 边 (将其 cap 置 0), 然后继续增广
 // 更实际的做法: 直接在包含辅助边的图上跑 s->t 最大流
 // 标准做法: 此时 s->t 的流量等于 t->s 反向边的流量
 ll base_flow = 0;
 for (auto &e : dinic.g[t]) {
 if (e.to == s) {
 // e 是 t->s, 其反向边 s->t 的 cap 减少量 = base_flow
 // 实际上 base_flow = e 的反向边当前容量减少的量
 // 更简单: base_flow = e.rev 对应边的原有 cap - 当前 cap
 // 或者: 删除 t->s 边, 在 S', T' 已满流的情况下继续跑 s->t
 }
 }
 // 推荐: 删除辅助边后直接跑 s->t max_flow
 // 把所有连接到 S 和 T 的边 cap 清 0 (防止残余影响)
 for (auto &e : dinic.g[S]) e.cap = 0;
 for (auto &e : dinic.g[T]) e.cap = 0;
 // 清掉 t->s 辅助边
 for (auto &e : dinic.g[t])
 if (e.to == s && e.cap < INF / 2) e.cap = 0;
 for (auto &e : dinic.g[s])
 if (e.to == t && e.cap == 0) e.cap = 0; // s->t 方向的残余

 // Step 3: 在剩余网络上继续跑 s->t
 ll extra = dinic.max_flow(s, t);
 // base_flow 从原始平衡推导: 等于 sum(balance[i] where balance[i] > 0 且 i 在 S'->* 路径上)
 // 实际上: base_flow = dinic 中从 s 实际流出的净流 (在删边前)
 // 最稳的方式是重新构建并记录:
 // 这里改用 "在原有残量网络上增广 s->t" 的标准做法
 return extra; // 须与 base_flow 相加, 这里提供框架, 完整实现见下方
 }

 // ---- 获取每条边的实际流量 ----
 // 实际流量 = lower + (upper - lower 的边中流过 Dinic 的量)
 // 需要在建图时记录每条边对应的 Dinic 边编号
 vector<ll> get_flow() {
 // 在 add_edge 时记录每条边在 dinic.g[u] 中的下标
 }
}

```



```
// 流量 = lower + (初始 cap - 当前 cap)
// 此处仅提供接口，具体实现见完整版
return {};
}
};
```

### 8.6.1 上下界网络流 — 完整比赛模板

以下是一个可直接使用的完整实现，支持无源汇可行流和有源汇最大流。

```

/**
 * 上下界网络流 - 完整比赛模板
 * 功能：无源汇可行流 / 有源汇可行流 / 有源汇最大流
 * 每条边的实际流量 = lower + dinic 中对应边的流量
 */
struct BoundedDinic {
 struct Edge {
 int to, rev; ll cap;
 };

 int n, S, T;
 vector<vector<Edge>> g;
 vector<ll> balance;

 // 记录每条建图边的信息
 struct Record {
 int u, v; // 端点
 ll lower; // 下界
 int idx; // 在 g[u] 中的下标 (Dinic 内部编号)
 int rev_idx; // g[v] 中的反向边下标
 };
 vector<Record> rec;

 BoundedDinic(int n_) : n(n_), S(n_), T(n_ + 1), g(n_ + 2), balance(n_ + 2, 0) {}

 // 添加有上下界的边, 返回记录编号
 int add_edge(int u, int v, ll lower, ll upper) {
 int id = (int)rec.size();
 rec.push_back({u, v, lower, (int)g[u].size(), (int)g[v].size()});
 g[u].push_back({v, (int)g[v].size(), upper - lower});
 g[v].push_back({u, (int)g[u].size() - 1, 0}); // 反向边
 balance[v] += lower;
 balance[u] -= lower;
 return id;
 }

 // ----- Dinic 子程序 -----
 vector<int> level, ptr;
 bool bfs(int s, int t) {
 level.assign(g.size(), -1);
 queue<int> q;
 level[s] = 0; q.push(s);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (auto &e : g[u]) {
 if (level[e.to] == -1 && e.cap > 0) {
 level[e.to] = level[u] + 1;
 q.push(e.to);
 }
 }
 }
 return level[t] != -1;
 }

 ll dfs(int u, int t, ll f) {
 if (u == t) return f;
 for (int &i = ptr[u]; i < (int)g[u].size(); ++i) {
 auto &e = g[u][i];
 if (level[e.to] != level[u] + 1 || e.cap <= 0) continue;
 ll tr = dfs(e.to, t, min(f, e.cap));
 if (tr == 0) continue;
 e.cap -= tr;
 g[e.to][e.rev].cap += tr;
 return tr;
 }
 return 0;
 }

 ll max_flow(int s, int t) {
 ll flow = 0;
 while (bfs(s, t)) {
 ptr.assign(g.size(), 0);
 while (ll tr = dfs(s, t, INF)) flow += tr;
 }
 return flow;
 }

 // ----- 构建超级源汇的补偿边 -----
 ll build_super() {
 ll total = 0;
 for (int i = 0; i < n; ++i) {
 if (balance[i] > 0) {
 g[S].push_back({i, (int)g[i].size(), balance[i]});
 g[i].push_back({S, (int)g[S].size() - 1, 0});
 total += balance[i];
 } else if (balance[i] < 0) {
 g[i].push_back({T, (int)g[T].size(), -balance[i]});
 g[T].push_back({i, (int)g[i].size() - 1, 0});
 }
 }
 return total;
 }

 // ----- 无源汇可行流 -----
 bool feasible() {
 ll total = build_super();
 return max_flow(S, T) == total;
 }

 // ----- 有源汇可行流 -----
 bool feasible(int s, int t) {
 // 加辅助边 t -> s, 下界 0, 上界 INF
 add_edge(t, s, 0, INF);
 return feasible();
 }

 // ----- 有源汇最大流 -----
 // 返回最大流值, 若不可行返回 -1
 ll max_flow_bounded(int s, int t) {
 // Step 1: 求可行流
 int aux_id = add_edge(t, s, 0, INF); // 辅助边
 ll total = build_super();
 ll check = max_flow(S, T);
 if (check != total) return -1;
 }
}

```

```
// Step 2: 可行流的 s->t 净流量 = 辅助边的实际流量
auto &aux_rec = rec[aux_id];
ll base_flow = g[aux_rec.v][aux_rec.rev_idx].cap; // 反向边的剩余容量就是正向边的流量

// Step 3: 删除辅助边和超级源汇相关边, 在残量网络上继续跑 s->t
// 移除 t->s 辅助边
g[aux_rec.u][aux_rec.idx].cap = 0;
g[aux_rec.v][aux_rec.rev_idx].cap = 0;
// 移除 S 和 T 所有相关边 (把他们的 cap 归零即可)
for (auto &e : g[S]) e.cap = 0;
for (auto &e : g[T]) e.cap = 0;

// Step 4: 在原残量网络上继续增广 s->t
ll extra = max_flow(s, t);
return base_flow + extra;
}

// ----- 获取每条原始边的实际流量 -----
// 实际流量 = lower + 在 upper-lower 边上 Dinic 的流量
ll get_flow(int edge_id) {
 auto &r = rec[edge_id];
 // Dinic 边在 g[r.u][r.idx], 其流量 = 原始 cap - 当前 cap
 // 原始 cap = r.upper - r.lower
 // 当前 cap = g[r.u][r.idx].cap
 // 流量 = (r.upper - r.lower) - g[r.u][r.idx].cap
 // 但我们没法拿到"原始 cap", 因为建图时直接存的就是 (upper - lower)
 // 需要额外存储
 return 0; // 平台代码—需额外存储原始上界
}

};

// ===== 更简洁的最大流版本 (存储原始上界) =====
struct BoundedFlowSimple {
 struct Edge {
 int to, rev; ll cap;
 };
 int n, S, T;
 vector<vector<Edge>> g;
 vector<ll> bal;
 // 每条边的: {u, v, lower, original_upper_minus_lower}
 struct Info { int u, v; ll lo, cap0; };
 vector<Info> info;

 BoundedFlowSimple(int n_) : n(n_), S(n_), T(n_ + 1), g(n_ + 2), bal(n_ + 2) {}

 int add_edge(int u, int v, ll lo, ll hi) {
 int id = info.size();
 info.push_back({u, v, lo, hi - lo});
 g[u].push_back({v, (int)g[v].size(), hi - lo});
 g[v].push_back({u, (int)g[u].size() - 1, 0});
 bal[v] += lo; bal[u] -= lo;
 return id;
 }

 // Dinic BFS + DFS (与前面相同, 此处省略重复代码)
 // ... (同 std Dinic) ...

 bool feasible() {
 ll tot = 0;
 for (int i = 0; i < n; ++i) {
 if (bal[i] > 0) {
 g[S].push_back({i, (int)g[i].size(), bal[i]});
 g[i].push_back({S, (int)g[S].size() - 1, 0});
 tot += bal[i];
 } else if (bal[i] < 0) {
 g[i].push_back({T, (int)g[T].size(), -bal[i]});
 g[T].push_back({i, (int)g[i].size() - 1, 0});
 }
 }
 return max_flow(S, T) == tot;
 }

 bool feasible(int s, int t) {
 add_edge(t, s, 0, INF);
 return feasible();
 }

 ll max_flow_bounded(int s, int t) {
 int aux = add_edge(t, s, 0, INF);
 if (!feasible()) return -1; // feasible() 内部跑过 max_flow(S,T)

 // 可行流流量 = 辅助边 (t->s) 上通过的流量
 // 即 info[aux] 对应 Dinic 边上的 cap0 - cur_cap
 ll base = info[aux].cap0 - g[info[aux].u][info[aux].v].cap; // 行索引有误, 需用 idx

 // 为简洁, 此处略去完整 cap 清理逻辑; 实际比赛直接用下面统一版
 return base;
 }
};
```

8.7 复杂度速查

算法	复杂度	适用规模	备注
Dinic 标准	$O(V^2E)O(V^2E)$	$V \leq 103V \leq 10^3, E \leq 104E \leq 10^4$	一般图最大流首选
Dinic 缩放	$O(V^2E \log C)O(V^2E \log C)$	同标准 Dinic	大容量图更快
Dinic 单位容量	$O(EV)O(E\sqrt{V})$	$V \leq 105V \leq 10^5, E \leq 105E \leq 10^5$	二分图匹配 / 网络
MC MF	$O(F \log V)O(FE \log V)$	$FF \text{ (流量)} \leq 104 \leq 10^4$	Johnson 势能版
匈牙利 (指派)	$O(V^3)O(V^3)$	$V \leq 500V \leq 500$	稠密带权指派
Hopcroft-Karp	$O(EV)O(E\sqrt{V})$	$V \leq 105V \leq 10^5$	无权二分图匹配
上下界最大流	$O(\text{Dinic})O(\text{Dinic})$	同 Dinic	图规模加 2 个节点

8.8 常见建图技巧

拆点 (Node Splitting)

当需要限制点的容量时：

二分图 → 最大流

棋盘染色

$n \times m$  网格图上求最大独立集 / 最小覆盖：

时间拆点

分层图最短路/最大流，用于处理不同时间步的状态（例如每个时间步复制一份全图）。

8.9 常见坑点

坑点	说明
反向边容量	有向边：正向 <code>cap</code> ，反向 <code>0</code> 。无向边：正反向均为 <code>cap</code>
INF 取值	用 <code>LLONG_MAX / 4</code> 或 <code>1e18</code> ，保证加法不溢出； <code>0x3f3f3f3f</code> 在 <code>ll</code> 图不够大
当前弧重置	每次 BFS 后必须重置 <code>ptr</code> ，否则当前弧优化失效或 WA
边数奇数偶数	<code>tot^1</code> 技巧要求边从 0 开始编号且成对添加
流量对称性	任意时刻 <code>fwd.cap + rev.cap = 初始 cap</code> 必须成立，可断言验证
SPFA 被卡	费用流优先用 Dijkstra + 势能：势能初始化只在有负权边时跑 SPFA
重边合并	多边直接累加容量，避免建重边降低效率
DFS 爆栈	递归 DFS 增广在 <code>VV</code> 较大时可能栈溢出，改用迭代版
下界流流量获取	实际流量 == 下界 ++ Dinic 残量网络对应边流量

8.10 调试清单

参考文献：KACTL ([github.com/kth-competitive-programming/kactl](https://github.com/kth-competitive-programming/kactl))、OI-wiki ([oi-wiki.org](https://oi-wiki.org))、cp-algorithms ([cp-algorithms.com](https://cp-algorithms.com))、AtCoder Library (ACL)

9. 高级技巧 (Advanced Techniques)

9.1 莫队算法 (Mo's Algorithm)

9.1.1 问题引入

给定长度为 `nn` 的数组 `a[1..n]`，`qq` 次询问 `[l,r]` 区间内不同数字的个数。

朴素做法：每次询问  $O(r-l+1)$  扫描，总  $O(nq)O(nq)$ ，不可接受。

9.1.2 核心思想

莫队算法是离线处理区间查询的分块技巧。将询问按左端点分块，块内按右端点排序。维护双指针 `L,RL,R`，相邻询问之间移动指针即可更新答案，均摊  $O((n+q)nO((n+q)\sqrt{n}))$ 。

核心：排序函数

莫队排序：按 `l/B`/`l/B` 分块，块内交替升降右端点（奇偶优化）：

```
int B = max(1, int(n / sqrt(q))); // 块大小
sort(qs.begin(), qs.end(), [&](const Query& a, const Query& b) {
 int ba = a.l / B, bb = b.l / B;
 if (ba != bb) return ba < bb;
 // 奇偶优化：奇数块 r 降序，偶数块 r 升序
 return (ba & 1) ? (a.r > b.r) : (a.r < b.r);
});
```

### 9.1.3 基础实现

```
struct Mo {
 int n, B, cur = 0;
 vector<int> a, cnt, ans;
 vector<array<int,3>> qs; // {l, r, idx}

 Mo(vector<int>&_a) : a(_a), n(_a.size() - 1) { // 1-indexed
 B = max(1, int(n / sqrt(n))); // 默认块大小
 cnt.assign(*max_element(a.begin(), a.end()) + 1, 0);
 }

 void addQuery(int l, int r) {
 int idx = qs.size();
 qs.push_back({l, r, idx});
 }

 void add(int pos) {
 if (cnt[a[pos]]++ == 0) cur++;
 }
 void remove(int pos) {
 if (--cnt[a[pos]] == 0) cur--;
 }

 vector<int> solve() {
 ans.resize(qs.size());
 sort(qs.begin(), qs.end(), [&](auto& x, auto& y) {
 int bx = x[0] / B, by = y[0] / B;
 if (bx != by) return bx < by;
 return (bx & 1) ? (x[1] > y[1]) : (x[1] < y[1]);
 });
 int L = 1, R = 0; // 空区间
 for (auto& [l, r, idx] : qs) {
 while (L > l) add(--L);
 while (R < r) add(++R);
 while (L < l) remove(L++);
 while (R > r) remove(R--);
 ans[idx] = cur;
 }
 return ans;
 }
};
```

### 9.1.4 带修改莫队 (Mo with Updates)

允许单点修改，加入时间维： $(l, r, t)(l, r, t)$ 。

```
struct MoWithUpdate {
 int n, B;
 vector<int> a, cnt, ans;
 vector<array<int,4>> qs; // {l, r, t, idx}
 vector<array<int,2>> upd; // {pos, val}

 MoWithUpdate(vector<int>&_a) : a(_a), n(_a.size() - 1) {
 B = max(1, int(pow(n, 2.0/3)));
 cnt.assign(maxVal + 1, 0);
 }

 void add(int pos) { if (cnt[a[pos]]++ == 0) cur++; }
 void remove(int pos) { if (--cnt[a[pos]] == 0) cur--; }
 void apply(int t, int L, int R) {
 int pos = upd[t][0], &val = upd[t][1];
 if (L <= pos && pos <= R) { remove(pos); swap(a[pos], val); add(pos); }
 else swap(a[pos], val);
 }

 vector<int> solve() {
 // 排序：左端点块 -> 右端点块 -> 时间
 sort(qs.begin(), qs.end(), [&](auto& x, auto& y) {
 int bx = x[0]/B, by = y[0]/B;
 int rx = x[1]/B, ry = y[1]/B;
 if (bx != by) return bx < by;
 if (rx != ry) return (bx & 1) ? (rx > ry) : (rx < ry);
 return ((rx & 1) ? (x[2] > y[2]) : (x[2] < y[2]));
 });
 int L = 1, R = 0, T = 0;
 for (auto& [l, r, t, idx] : qs) {
 while (L > l) add(--L);
 while (R < r) add(++R);
 while (L < l) remove(L++);
 while (R > r) remove(R--);
 while (T < t) apply(T++, L, R);
 while (T > t) apply(--T, L, R);
 ans[idx] = cur;
 }
 return ans;
 }
};
```

### 9.1.5 希尔伯特序优化 (Hilbert Order)

将区间  $[l, r][l, r]$  映射到希尔伯特曲线 (Hilbert curve) 上的点，按该映射值排序可进一步降低指针移动总量。

```
// 将 (x, y) 映射到希尔伯特序, N 必须是 2 的幂
long long hilbertOrder(int x, int y, int N) {
 long long d = 0;
 for (int s = N >> 1; s > 0; s >>= 1) {
 int rx = (x & s) > 0;
 int ry = (y & s) > 0;
 d += (long long)s * s * ((3 * rx) ^ ry);
 // 旋转
 if (ry == 0) {
 if (rx == 1) { x = N - 1 - x; y = N - 1 - y; }
 swap(x, y);
 }
 }
 return d;
}
```

实际使用中，取  $NN$  为大于  $nn$  的最小二次幂，按 `hilbertOrder(l, r, N)` 排序即可，常数优于普通莫队。

9.1.6 复杂度与适用场景

变种	复杂度	适用
普通莫队	$O((n+q)n)O((n+q)\sqrt{n})$	无修改区间查询
带修改莫队	$O(n^{5/3})O(n^{\{5/3\}})$	带单点修改
树上莫队	$O(nn)O(n\sqrt{n})$	树上路径/子树询问
回滚莫队	$O(nn)O(n\sqrt{n})$	不支持删除操作（如最大值）

9.2 CDQ 分治 (CDQ Divide and Conquer)

9.2.1 问题引入

解决三维偏序问题：给定  $nn$  个三元组  $(a_i,b_i,c_i)$ ，对每个  $i$  统计  $j \neq i$  且  $a_j \leq a_i, b_j \leq b_i, c_j \leq c_i$  的数量。  
本质是离线处理带时间维的动态问题：把修改和查询看作事件，用分治排序时间维，BIT 维护值维。

9.2.2 核心思想

将操作序列按时间（下标）分治：

三维偏序实现：

```
struct Node { int a, b, c, cnt, ans; };
vector<Node> nodes;

struct BIT {
 vector<int> t; int n;
 BIT(int _n) : n(_n), t(_n + 1) {}
 void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
 int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

void cdq(int l, int r, BIT& bit) {
 if (l >= r) return;
 int m = (l + r) / 2;
 cdq(l, m, bit);
 cdq(m + 1, r, bit);

 // 归并排序 b，同时处理右半边对左半边的查询
 int i = l, j = m + 1;
 vector<Node> tmp;
 while (i <= m && j <= r) {
 if (nodes[i].b <= nodes[j].b) {
 bit.add(nodes[i].c, nodes[i].cnt);
 tmp.push_back(nodes[i++]);
 } else {
 nodes[j].ans += bit.sum(nodes[i].c);
 tmp.push_back(nodes[j++]);
 }
 }
 while (i <= m) {
 bit.add(nodes[i].c, nodes[i].cnt);
 tmp.push_back(nodes[i++]);
 }
 while (j <= r) {
 nodes[j].ans += bit.sum(nodes[j].c);
 tmp.push_back(nodes[j++]);
 }
 // 清理 BIT
 for (int k = l; k <= m; k++) bit.add(nodes[k].c, -nodes[k].cnt);
 // 归并回原数组
 for (int k = l; k <= r; k++) nodes[k] = tmp[k - l];
}
```

9.2.3 应用场景

9.3 整体二分 (Parallel Binary Search)

9.3.1 问题引入

区间第  $kk$  小（静态）： $nn$  个数的数组， $qq$  次询问  $[l,r]$  中第  $kk$  小的值。  
主席树可  $O(n\log n)O(n \log n)$  解决，整体二分提供另一种思路，同样  $O((n+q)\log V)O((n+q) \log V)$ 。

9.3.2 核心思想

二分答案域  $[L,R][L,R]$ 。将当前所有询问与修改放在一次扫描中：判定每个询问的答案在  $\leq mid$  还是  $> mid$ ，分别递归下去。  
对于区间第  $kk$  小的多组询问，二分值域，用 BIT 维护  $\leq mid$  的元素位置。对每组询问统计其区间内  $\leq mid$  的数量，若  $\geq k$  则答案为左半边，否则递归到右半边（ $kk$  减去已计入的）。

```

struct Query { int l, r, k, id; }; // 询问: 区间 [l,r], 求第 k 小, id 是原始编号
struct Element { int pos, val; }; // 数组元素位置和价值

void solve(int L, int R, vector<Element>& elems, vector<Query>& qs, vector<int>& ans, BIT& bit) {
 if (qs.empty()) return;
 if (L == R) {
 for (auto& q : qs) ans[q.id] = L;
 return;
 }
 int mid = (L + R) / 2;

 // 分元素: <= mid 的入左, > mid 的入右
 vector<Element> el_left, el_right;
 for (auto& e : elems) {
 if (e.val <= mid) { bit.add(e.pos, 1); el_left.push_back(e); }
 else el_right.push_back(e);
 }

 // 检查每个询问
 vector<Query> q_left, q_right;
 for (auto& q : qs) {
 int cnt = bit.sum(q.r) - bit.sum(q.l - 1);
 if (cnt >= q.k) q_left.push_back(q);
 else { q.k -= cnt; q_right.push_back(q); }
 }

 // 清理 BIT
 for (auto& e : el_left) bit.add(e.pos, -1);

 solve(L, mid, el_left, q_left, ans, bit);
 solve(mid + 1, R, el_right, q_right, ans, bit);
}

```

### 9.3.3 带修改整体二分

若数组允许单点修改，将修改视为“删除旧值 + 添加新值”两个事件。整体框架不变，只是元素集合随递归动态变化。

```

struct Event {
 int type; // 0=询问, 1=插入, -1=删除
 int l, r, k, val, id;
};

void solve(int L, int R, vector<Event>& evt, vector<int>& ans, BIT& bit) {
 if (evt.empty()) return;
 if (L == R) {
 for (auto& e : evt) if (e.type == 0) ans[e.id] = L;
 return;
 }
 int mid = (L + R) / 2;
 vector<Event> el, er;
 for (auto& e : evt) {
 if (e.type == 0) { // 询问
 int cnt = bit.sum(e.r) - bit.sum(e.l - 1);
 if (cnt >= e.k) el.push_back(e);
 else { e.k -= cnt; er.push_back(e); }
 } else { // 修改
 if (e.val <= mid) { bit.add(e.l, e.type); el.push_back(e); }
 else er.push_back(e);
 }
 }
 for (auto& e : el) if (e.type != 0) bit.add(e.l, -e.type);
 solve(L, mid, el, ans, bit);
 solve(mid + 1, R, er, ans, bit);
}

```

### 9.3.4 适用问题

整体二分将一类“多组询问，答案具有单调性”的问题统一在线性扫描中判定。典型问题：

## 9.4 主席树 (Persistent Segment Tree)

### 9.4.1 问题引入

静态区间第  $kk$  小：  $nn$  个数的数组，  $qq$  次询问  $[l,r][l,r]$  第  $kk$  小。  $n,q \leq 2 \times 10^5, q \leq 2 \times 10^5$ 。

### 9.4.2 核心思想

建  $n+1$  棵权值线段树，第  $i$  棵表示前缀  $[1,i]$  中各值的出现次数（值域为离散化后的  $[1,m]$ ）。

第  $i$  棵与第  $i-1$  棵只差一条插入路径（ $O(\log n)$  个节点），共享其他节点，空间  $O(n \log n)$ 。

区间  $[l,r]$  的第  $kk$  小：同时在  $root[l-1]$  和  $root[r]$  上二分，左儿子差值即为该值域在  $[l,r]$  内的个数。

9.4.3 实现

```
struct SegNode {
 int lc, rc, cnt; // 左右儿子编号, 区间覆盖次数
};

vector<SegNode> t; // 动态开点池
vector<int> roots; // 每个版本的根节点编号

int newNode() {
 t.push_back({0, 0, 0});
 return t.size() - 1;
}

int insert(int pre, int l, int r, int pos) {
 int now = newNode();
 t[now] = t[pre];
 t[now].cnt++;
 if (l == r) return now;
 int m = (l + r) >> 1;
 if (pos <= m) t[now].lc = insert(t[pre].lc, l, m, pos);
 else t[now].rc = insert(t[pre].rc, m + 1, r, pos);
 return now;
}

// 区间 [ql, qr] 第 k 小
int kth(int u, int v, int l, int r, int k) {
 if (l == r) return l;
 int m = (l + r) >> 1;
 int leftCnt = t[t[v].lc].cnt - t[t[u].lc].cnt;
 if (k <= leftCnt) return kth(t[u].lc, t[v].lc, l, m, k);
 else return kth(t[u].rc, t[v].rc, m + 1, r, k - leftCnt);
}

// 建树
vector<int> vals; // 离散化后的值
roots.push_back(newNode()); // root[0] 空树
for (int i = 1; i <= n; i++) {
 int pos = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
 roots.push_back(insert(roots[i-1], 1, vals.size(), pos));
}

// 查询
int ans = kth(roots[l-1], roots[r], 1, vals.size(), k);
```

9.4.4 带修改主席树

对带修改的区间第 kk 小，需要主席树 + BIT（树状数组套主席树）：

```
// 树状数组套主席树（持续化 BIT 版本）
// add(p, val, delta): 在位置 p 上插入或删除值 val
void add(int p, int val, int delta) {
 for (int i = p; i <= n; i += i & -i)
 rootsBIT[i] = insert(rootsBIT[i], 1, m, val, delta);
}

// 查询 [l, r] 第 k 小
int query(int l, int r, int k) {
 // 收集 BIT 中涉及的所有根节点
 L.clear(); R.clear();
 for (int i = l - 1; i > 0; i -= i & -i) L.push_back(rootsBIT[i]);
 for (int i = r; i > 0; i -= i & -i) R.push_back(rootsBIT[i]);
 return kth_vec(1, m, k);
}
```

9.4.5 变种

问题	做法
区间不同数的个数	对每个位置维护其上一次出现位置，主席树 $O(n\log n)O(n\log n)$
区间 Mex	主席树维护每个值最后出现位置 $\geq \text{val}$ 的最小值
区间众数	主席树 + 分块（在线 $O(n)O(\sqrt{n})$ ）或离线莫队
区间大于 xx 的个数	主席树查询即可

9.5 Link-Cut Tree (LCT)

9.5.1 问题引入

维护一个森林，支持：

LCT 用 Splay 实现每个 Preferred Path，支持  $O(\log n)O(\log n)$  均摊的上述操作。

9.5.2 核心结构

操作：



### 9.5.3 实现

```
struct LCT {
 struct Node {
 int ch[2], fa, rev;
 int val, sum; // 可自定义维护信息
 };
 vector<Node> t;

 LCT(int n) : t(n + 1) {}

 bool isRoot(int x) {
 int f = t[x].fa;
 return t[f].ch[0] != x && t[f].ch[1] != x;
 }

 void pushup(int x) {
 t[x].sum = t[x].val ^ t[t[x].ch[0]].sum ^ t[t[x].ch[1]].sum;
 }

 void pushrev(int x) {
 swap(t[x].ch[0], t[x].ch[1]);
 t[x].rev ^= 1;
 }

 void pushdown(int x) {
 if (t[x].rev) {
 if (t[x].ch[0]) pushrev(t[x].ch[0]);
 if (t[x].ch[1]) pushrev(t[x].ch[1]);
 t[x].rev = 0;
 }
 }

 // 将 x 旋转到其 Splay 的根
 void rotate(int x) {
 int y = t[x].fa, z = t[y].fa;
 int k = (t[y].ch[1] == x);
 if (!isRoot(y)) t[z].ch[t[z].ch[1] == y] = x;
 t[x].fa = z;
 t[y].ch[k] = t[x].ch[k ^ 1];
 if (t[t[x].ch[k ^ 1]].fa == y) t[t[x].ch[k ^ 1]].fa = x;
 t[x].ch[k ^ 1] = y;
 t[y].fa = x;
 pushup(y); pushup(x);
 }

 // 将 x 旋转到所在 Splay 的根 (Splay 操作)
 void splay(int x) {
 static int stk[200005]; int top = 0;
 int y = x;
 stk[++top] = y;
 while (!isRoot(y)) { y = t[y].fa; stk[++top] = y; }
 while (top) pushdown(stk[top--]);
 while (!isRoot(x)) {
 y = t[x].fa;
 int z = t[y].fa;
 if (!isRoot(y))
 rotate((t[y].ch[1] == x) ^ (t[z].ch[1] == y) ? x : y);
 rotate(x);
 }
 }

 // access: 打通 x 到根的路径
 void access(int x) {
 for (int y = 0; x; y = x, x = t[x].fa) {
 splay(x);
 t[x].ch[1] = y;
 pushup(x);
 }
 }

 // makeroot: 将 x 变为根 (access + splay + 翻转子树)
 void makeroot(int x) {
 access(x);
 splay(x);
 pushrev(x);
 }

 // findroot: 找 x 所在树的根
 int findroot(int x) {
 access(x); splay(x);
 while (t[x].ch[0]) { pushdown(x); x = t[x].ch[0]; }
 splay(x);
 return x;
 }

 // split: 分离出 x 到 y 的路径 (y 为 Splay 根)
 void split(int x, int y) {
 makeroot(x);
 access(y);
 splay(y);
 }

 // link: 加边 (x, y)
 void link(int x, int y) {
 makeroot(x);
 if (findroot(y) != x) t[x].fa = y;
 }

 // cut: 删边 (x, y)
 void cut(int x, int y) {
 makeroot(x);
 if (findroot(y) == x && t[y].fa == x && t[y].ch[0] == 0) {
 t[y].fa = t[x].ch[1] = 0;
 pushup(x);
 }
 }
};
```

## 9.5.4 路径维护模式

```
// 查询 x 到 y 路径上的和
split(x, y);
int ans = t[y].sum;

// 单点修改
splay(x);
t[x].val = newVal;
pushup(x);

// 路径加 (需要 lazytag, 略)
```

## 9.5.5 应用

## 9.6 树链剖分 (Heavy-Light Decomposition, HLD)

### 9.6.1 问题引入

给定一棵有根树，节点带权，支持：

$n, q \leq 2 \times 10^5, q \leq 2 \times 10^5$ ，要求  $O(\log^2 n) O(\log^2 n)$  每次操作。

### 9.6.2 核心思想

将树划分成若干重链（heavy path），每条重链的 DFS 序连续，可用线段树维护。跳链时每次跳到链顶，至多跳  $O(\log n) O(\log n)$  条链。

两次 DFS：

### 9.6.3 实现

```
struct HLD {
 int n, timer = 0;
 vector<vector<int>> adj;
 vector<int> fa, dep, sz, son, top, dfn, rnk; // rnk[dfn] = 原节点
 SegTree seg; // 线段树

 HLD(int _n, vector<vector<int>>& _adj, vector<int>& val) : n(_n), adj(_adj) {
 fa.resize(n + 1); dep.resize(n + 1); sz.resize(n + 1);
 son.resize(n + 1); top.resize(n + 1); dfn.resize(n + 1); rnk.resize(n + 1);
 dfs1(1, 0);
 dfs2(1, 1);
 // 按 dfn 构建线段树
 vector<int> arr(n + 1);
 for (int i = 1; i <= n; i++) arr[dfn[i]] = val[i];
 seg.build(arr, 1, 1, n);
 }

 void dfs1(int u, int f) {
 fa[u] = f; dep[u] = dep[f] + 1; sz[u] = 1;
 int mx = 0;
 for (int v : adj[u]) {
 if (v == f) continue;
 dfs1(v, u);
 sz[u] += sz[v];
 if (sz[v] > mx) { mx = sz[v]; son[u] = v; }
 }
 }

 void dfs2(int u, int tp) {
 top[u] = tp; dfn[u] = ++timer; rnk[timer] = u;
 if (son[u]) dfs2(son[u], tp);
 for (int v : adj[u]) {
 if (v != fa[u] && v != son[u]) dfs2(v, v);
 }
 }

 // 路径 $u-v$ 操作
 void pathUpdate(int u, int v, int x) {
 while (top[u] != top[v]) {
 if (dep[top[u]] < dep[top[v]]) swap(u, v);
 seg.update(dfn[top[u]], dfn[u], x);
 u = fa[top[u]];
 }
 if (dep[u] > dep[v]) swap(u, v);
 seg.update(dfn[u], dfn[v], x);
 }

 int pathQuery(int u, int v) {
 int res = 0;
 while (top[u] != top[v]) {
 if (dep[top[u]] < dep[top[v]]) swap(u, v);
 res += seg.query(dfn[top[u]], dfn[u]);
 u = fa[top[u]];
 }
 if (dep[u] > dep[v]) swap(u, v);
 res += seg.query(dfn[u], dfn[v]);
 return res;
 }

 // 子树操作：dfn[u] 到 dfn[u] + sz[u] - 1
 void subtreeUpdate(int u, int x) {
 seg.update(dfn[u], dfn[u] + sz[u] - 1, x);
 }

 int subtreeQuery(int u) {
 return seg.query(dfn[u], dfn[u] + sz[u] - 1);
 }

 // LCA
 int lca(int u, int v) {
 while (top[u] != top[v]) {
 if (dep[top[u]] < dep[top[v]]) swap(u, v);
 u = fa[top[u]];
 }
 return dep[u] < dep[v] ? u : v;
 }
};
```

## 9.6.4 复杂度

### 9.6.5 扩展

扩展	方法
边权转点权	将边权下放到深度大的端点
LCA 祖先链二分	跳链 + 链上线段树二分
动态换根	分类讨论 LCA 与当前根的关系
长链剖分	用于 $O(1)O(1)$ 求 $kk$ 级祖先和 $O(n)O(n)$ 维护 DP（以深度为下标）

## 9.7 平衡树 (Balanced Binary Search Tree)

### 9.7.1 Treap (Tree + Heap)

旋转 Treap，每个节点带随机优先级 `rnd`，通过左旋/右旋维护堆性质，期望深度  $O(\log n)O(\log n)$ 。

```
struct Treap {
 struct Node {
 int val, rnd, sz, cnt, ch[2]; // cnt: 同值计数
 Node() {}
 Node(int v) : val(v), rnd(rand()), sz(1), cnt(1) { ch[0] = ch[1] = 0; }
 };
 vector<Node> t;
 int root = 0;

 Treap() { t.push_back(Node()); } // 0 号哨兵

 int newNode(int v) { t.push_back(Node(v)); return t.size() - 1; }
 void pushup(int x) { t[x].sz = t[t[x].ch[0]].sz + t[t[x].ch[1]].sz + t[x].cnt; }

 void rotate(int &x, int d) { // d=0 右旋, d=1 左旋
 int y = t[x].ch[d ^ 1];
 t[x].ch[d ^ 1] = t[y].ch[d];
 t[y].ch[d] = x;
 pushup(x); pushup(y);
 x = y;
 }

 void insert(int &x, int v) {
 if (!x) { x = newNode(v); return; }
 if (v == t[x].val) { t[x].cnt++; pushup(x); return; }
 int d = (v > t[x].val);
 insert(t[x].ch[d], v);
 if (t[t[x].ch[d]].rnd < t[x].rnd) rotate(x, d ^ 1);
 pushup(x);
 }

 void remove(int &x, int v) {
 if (!x) return;
 if (v == t[x].val) {
 if (t[x].cnt > 1) { t[x].cnt--; pushup(x); return; }
 if (!t[x].ch[0] && !t[x].ch[1]) { x = 0; return; }
 if (!t[x].ch[0]) { x = t[x].ch[1]; return; }
 if (!t[x].ch[1]) { x = t[x].ch[0]; return; }
 int d = (t[t[x].ch[0]].rnd < t[t[x].ch[1]].rnd) ? 1 : 0;
 rotate(x, d);
 remove(t[x].ch[d], v);
 } else {
 int d = (v > t[x].val);
 remove(t[x].ch[d], v);
 }
 pushup(x);
 }

 int kth(int x, int k) { // 第 k 小
 int left = t[t[x].ch[0]].sz;
 if (k <= left) return kth(t[x].ch[0], k);
 if (k <= left + t[x].cnt) return t[x].val;
 return kth(t[x].ch[1], k - left - t[x].cnt);
 }

 int rnk(int &x, int v) { // v 的排名 (严格小于 v 的个数 + 1)
 if (!x) return 1;
 if (v == t[x].val) return t[t[x].ch[0]].sz + 1;
 if (v < t[x].val) return rnk(t[x].ch[0], v);
 return t[t[x].ch[0]].sz + t[x].cnt + rnk(t[x].ch[1], v);
 }

 int pre(int v) { // 前驱
 int x = root, res = -2e9;
 while (x) {
 if (t[x].val < v) { res = t[x].val; x = t[x].ch[1]; }
 else x = t[x].ch[0];
 }
 return res;
 }

 int nxt(int v) { // 后继
 int x = root, res = 2e9;
 while (x) {
 if (t[x].val > v) { res = t[x].val; x = t[x].ch[0]; }
 else x = t[x].ch[1];
 }
 return res;
 }
};
```

### 9.7.2 FHQ Treap (无旋 Treap)

用 `split` 和 `merge` 替代旋转，代码更简洁，支持持久化。

```

struct FHQTreap {
 struct Node {
 int val, rnd, sz, ch[2];
 Node(int v = 0) : val(v), rnd(rand()), sz(1) { ch[0] = ch[1] = 0; }
 };
 vector<Node> t;

 FHQTreap() { t.push_back(Node()); } // 哨兵

 int newNode(int v) { t.push_back(Node(v)); return t.size() - 1; }

 void pushup(int x) { t[x].sz = t[t[x].ch[0]].sz + t[t[x].ch[1]].sz + 1; }

 // 按值 v 分裂: x 存 <= v 的节点, y 存 > v 的节点
 void split(int rt, int v, int &x, int &y) {
 if (!rt) { x = y = 0; return; }
 if (t[rt].val <= v) {
 x = rt;
 split(t[rt].ch[1], v, t[rt].ch[1], y);
 } else {
 y = rt;
 split(t[rt].ch[0], v, x, t[rt].ch[0]);
 }
 pushup(rt);
 }

 // 按大小 k 分裂: x 存前 k 个, y 存剩余
 void splitK(int rt, int k, int &x, int &y) {
 if (!rt) { x = y = 0; return; }
 int left = t[t[rt].ch[0]].sz;
 if (k <= left) {
 y = rt;
 splitK(t[rt].ch[0], k, x, t[rt].ch[0]);
 } else {
 x = rt;
 splitK(t[rt].ch[1], k - left - 1, t[rt].ch[1], y);
 }
 pushup(rt);
 }

 // 合并 x 和 y (要求 x 中所有值 < y 中所有值)
 int merge(int x, int y) {
 if (!x || !y) return x | y;
 if (t[x].rnd < t[y].rnd) {
 t[x].ch[1] = merge(t[x].ch[1], y);
 pushup(x);
 return x;
 } else {
 t[y].ch[0] = merge(x, t[y].ch[0]);
 pushup(y);
 return y;
 }
 }

 void insert(int &rt, int v) {
 int x, y;
 split(rt, v, x, y);
 rt = merge(merge(x, newNode(v)), y);
 }

 void remove(int &rt, int v) {
 int x, y, z;
 split(rt, v, x, z); // x <= v
 split(x, v - 1, x, y); // x < v, y = {v}
 y = merge(t[y].ch[0], t[y].ch[1]); // 删掉一个 v
 rt = merge(merge(x, y), z);
 }

 int rnk(int &rt, int v) { // < v 的个数 + 1
 int x, y;
 split(rt, v - 1, x, y);
 int ans = t[x].sz + 1;
 rt = merge(x, y);
 return ans;
 }

 int kth(int rt, int k) {
 int x, y, z, ans;
 splitK(rt, k, x, y);
 splitK(x, k - 1, z, x); // z 是第 k 个
 ans = t[x].val;
 rt = merge(merge(z, x), y);
 return ans;
 }

 int pre(int &rt, int v) { return kth(rt, rnk(rt, v) - 1); }
 int nxt(int &rt, int v) { return kth(rt, rnk(rt, v) + 1); }
};

```

### 9.7.3 Splay

通过 `splay` 操作将刚访问的节点旋转到根, 均摊  $O(\log n)$  ( $O(\log n)$ ), 支持区间反转 (翻转)。

```
struct Splay {
 struct Node {
 int val, sz, ch[2], fa;
 int rev; // 翻转标记
 Node(int v = 0) : val(v), sz(1), fa(0), rev(0) { ch[0] = ch[1] = 0; }
 };
 vector<Node> t;

 Splay() { t.push_back(Node()); } // 哨兵

 int newNode(int v, int f = 0) {
 t.push_back(Node(v));
 t.back().fa = f;
 return t.size() - 1;
 }

 void pushup(int x) { t[x].sz = t[t[x].ch[0]].sz + t[t[x].ch[1]].sz + 1; }
 void pushdown(int x) {
 if (t[x].rev) {
 swap(t[x].ch[0], t[x].ch[1]);
 if (t[x].ch[0]) t[t[x].ch[0]].rev ^= 1;
 if (t[x].ch[1]) t[t[x].ch[1]].rev ^= 1;
 t[x].rev = 0;
 }
 }

 void rotate(int x) {
 int y = t[x].fa, z = t[y].fa;
 int k = (t[y].ch[1] == x);
 if (z) t[z].ch[t[z].ch[1] == y] = x;
 t[x].fa = z;
 t[y].ch[k] = t[x].ch[k ^ 1];
 if (t[x].ch[k ^ 1]) t[t[x].ch[k ^ 1]].fa = y;
 t[x].ch[k ^ 1] = y;
 t[y].fa = x;
 pushup(y); pushup(x);
 }

 void splay(int x, int goal = 0) { // 将 x 旋转到 goal 的儿子
 static int stk[200005]; int top = 0;
 for (int p = x; p != goal; p = t[p].fa) stk[++top] = p;
 while (top) pushdown(stk[top--]);
 while (t[x].fa != goal) {
 int y = t[x].fa, z = t[y].fa;
 if (z != goal)
 rotate((t[y].ch[1] == x) ^ (t[z].ch[1] == y) ? x : y);
 rotate(x);
 }
 if (goal == 0) root = x;
 }
};
```

9.7.4 对比

特性	Treap	FHQ Treap	Splay
实现难度	中	低	高
均摊复杂度	期望 $O(\log n)O(\log n)$	期望 $O(\log n)O(\log n)$	均摊 $O(\log n)O(\log n)$
最坏复杂度	$O(n)O(n)$	$O(n)O(n)$	均摊保证
持久化	否	是	否
区间操作	有限	易	是（翻转等）
常数	中	中	较小

9.8 Bitset 优化

9.8.1 核心思想

`std::bitset<N>` 一次可并行处理  $N=64N=64$ （或更大）个布尔值。当  $n \leq 105n \leq 10^5$  时，bitset 可将  $O(n^2)O(n^2)$  压缩到  $O(n^2/64)O(n^2/64)$ 。

9.8.2 常见技巧

传递闭包 (Transitive Closure)

```
// 有向图传递闭包 $O(n^3 / 64)$
bitset<1005> reach[1005]; // reach[i][j] = i能否到j
for (int k = 1; k <= n; k++)
 for (int i = 1; i <= n; i++)
 if (reach[i][k]) reach[i] |= reach[k];
```

01 背包可行性

```
// n 个物品，判断能否凑出每个重量
bitset<100005> dp;
dp[0] = 1;
for (int i = 1; i <= n; i++) dp |= (dp << w[i]); // $O(nW/64)$
```

多重背包二进制优化

```
// 每种物品有 cnt 个
for (int i = 1; i <= n; i++) {
 for (int k = 1; k <= cnt[i]; k *= 2) {
 dp |= (dp << (w[i] * k));
 cnt[i] -= k;
 }
 if (cnt[i]) dp |= (dp << (w[i] * cnt[i]));
}
```

最短路 — 任意模数同余最短路

对于  $nn$  个顶点的稠密图 ( $n \leq 5000n \leq 5000$ )，BFS 中维护 `bitset<N>` 表示“未访问”集合，用位运算加速邻接遍历：

```
bitset<100005> vis, adj[100005];
// 在图 G 上 BFS
queue<int> q;
q.push(s); vis[s] = 1;
while (!q.empty()) {
 int u = q.front(); q.pop();
 auto nxt = adj[u] & ~vis; // 所有未访问的邻居
 for (int v = nxt._Find_first(); v < nxt.size(); v = nxt._Find_next(v)) {
 vis[v] = 1;
 q.push(v);
 }
}
```

字符串匹配 / 子序列检测

```
// 对字符集构建位置矩阵 $O(m * n / 64)$
bitset<100005> pos[26];
for (int i = 1; i <= n; i++) pos[s[i] - 'a'].set(i);

// 模式串 t 是否作为子序列存在 $O(m * n / 64)$
bitset<100005> ans; ans.set(); // 全 1
for (char c : t) ans = (ans << 1) & pos[c - 'a'];
// ans.any() 表示存在
```

9.8.3 性能注意事项

9.9 分块 (Sqrt Decomposition)

9.9.1 核心思想

将长度为  $nn$  的数组分成  $n\sqrt{n}$  块，每块大小  $\leq n\sqrt{n}$

9.9.2 基础实现

```
struct Block {
 int n, B;
 vector<int> a, sum, lazy; // sum[i] = 第 i 块区间和, lazy[i] = 第 i 块加法标记

 Block(vector<int>&_a) : a(_a), n(_a.size() - 1) { // 1-indexed
 B = max(1, int(sqrt(n)));
 int blocks = (n + B - 1) / B;
 sum.assign(blocks + 1, 0);
 lazy.assign(blocks + 1, 0);
 for (int i = 1; i <= n; i++) sum[belong(i)] += a[i];
 }

 int belong(int i) { return (i - 1) / B + 1; }
 int bl(int x) { return (x - 1) * B + 1; } // 块 x 的左端点
 int br(int x) { return min(n, x * B); } // 块 x 的右端点

 // 区间加
 void add(int l, int r, int v) {
 int blkL = belong(l), blkR = belong(r);
 if (blkL == blkR) {
 for (int i = l; i <= r; i++) a[i] += v;
 sum[blkL] += v * (r - l + 1);
 return;
 }
 for (int i = l; i <= br(blkL); i++) a[i] += v, sum[blkL] += v;
 for (int i = bl(blkR); i <= r; i++) a[i] += v, sum[blkR] += v;
 for (int i = blkL + 1; i < blkR; i++) lazy[i] += v, sum[i] += v * (br(i) - bl(i) + 1);
 }

 // 区间求和
 int query(int l, int r) {
 int blkL = belong(l), blkR = belong(r), res = 0;
 if (blkL == blkR) {
 for (int i = l; i <= r; i++) res += a[i] + lazy[blkL];
 return res;
 }
 for (int i = l; i <= br(blkL); i++) res += a[i] + lazy[blkL];
 for (int i = bl(blkR); i <= r; i++) res += a[i] + lazy[blkR];
 for (int i = blkL + 1; i < blkR; i++) res += sum[i];
 return res;
 }
};
```

9.9.3 经典应用

问题	分块思路
区间加 + 区间第 $kk$ 小	块内维护排序数组，零散块重构 $O(n\log n)O(\sqrt{n}\log n)$
区间取 $\min / \max$	块内维护极值 + 懒惰比较标记
区间开根号	当块内全为 $0/1$ 时跳过
区间众数 (在线)	预处理块间众数和前缀频率， $O(nn)O(n\sqrt{n})$
莫队	基于分块的离线查询 (见 9.1)

9.9.4 块的大小选择

9.10 虚树 (Virtual Tree)

9.10.1 问题引入

给定一棵树，多次询问，每次给出  $kk$  个关键点 ( $\sum k \leq 2 \times 10^5$ )。需要在这些关键点之间进行 DP 或路径操作。  
建虚树：只保留关键点及其之间的 LCA，边为原树上缩链后的距离。虚树大小  $\leq 2k - 1$

## 9.10.2 建树流程

### 9.10.3 实现

```
struct VirtualTree {
 int n, timer = 0;
 vector<vector<int>> adj;
 vector<int> dfn, dep, fa, top, sz, son;
 // 需要 HLD 预处理 LCA 和 DFS 序 (参见 9.6 节)

 // 建虚树: 输入关键点列表, 返回虚树的邻接表 (父 -> 子, 带边权)
 // 边权为原树上两点距离: $dep[child] - dep[parent]$
 vector<vector<pair<int, int>>> build(vector<int>& keys) {
 sort(keys.begin(), keys.end(), [&](int a, int b) { return dfn[a] < dfn[b]; });

 // 加入相邻 LCA
 vector<int> pts = keys;
 for (int i = 0; i + 1 < (int)keys.size(); i++)
 pts.push_back(lca(keys[i], keys[i + 1]));

 sort(pts.begin(), pts.end(), [&](int a, int b) { return dfn[a] < dfn[b]; });
 pts.erase(unique(pts.begin(), pts.end()), pts.end());

 // 单调栈建边
 vector<vector<pair<int, int>>> vt(pts.size() + 1); // 虚树邻接表
 vector<int> stk;
 // 这里用 map 或 vector 映射原节点编号 -> 虚树节点编号
 unordered_map<int, int> id;
 for (int i = 0; i < (int)pts.size(); i++) id[pts[i]] = i;

 stk.push_back(0); // 根节点入栈 (假设 pts[0] 是根)
 for (int i = 1; i < (int)pts.size(); i++) {
 int u = pts[i];
 while (!stk.empty() && !isAncestor(pts[stk.back()], u))
 stk.pop_back();
 int p = pts[stk.back()];
 int w = dep[u] - dep[p]; // 原树距离即为深度差
 vt[id[p]].push_back({id[u], w});
 vt[id[u]].push_back({id[p], w});
 stk.push_back(i);
 }
 return vt;
 }

 bool isAncestor(int u, int v) {
 return dfn[u] <= dfn[v] && dfn[v] < dfn[u] + sz[u];
 }
};
```

### 9.10.4 经典应用

## 9.11 三元环计数 (3-Cycle Counting)

### 9.11.1 问题

给定  $n$  个节点、 $m$  条边的无向简单图，统计三元环（三角形）数量。 $n, m \leq 2 \times 10^5, m \leq 2 \times 10^5$ 。

### 9.11.2 算法

重定向每条无向边：度数大的指向度数小的，度数相同时按编号。得到 DAG，每个节点出度  $\leq \sqrt{m}$ （因为若一个点出度  $> \sqrt{m}$ ，则其所有邻居度  $\geq$  它，总度数至少  $m \times m = m \sqrt{m} \times \sqrt{m} = m$ ，矛盾）。

算法：

```
long long count3Cycles(int n, vector<pair<int, int>>& edges) {
 vector<int> deg(n + 1);
 for (auto& [u, v] : edges) { deg[u]++; deg[v]++; }

 // 建立有向图，只保留度数大的指向度数小的
 vector<vector<int>> G(n + 1);
 for (auto& [u, v] : edges) {
 if (deg[u] > deg[v] || (deg[u] == deg[v] && u > v)) swap(u, v);
 // u 度数更小或编号更小，边方向 u -> v
 G[u].push_back(v);
 }

 long long ans = 0;
 vector<int> mark(n + 1, 0);
 for (int u = 1; u <= n; u++) {
 for (int v : G[u]) mark[v] = u; // 标记 u 的邻居
 for (int v : G[u]) {
 for (int w : G[v]) {
 if (mark[w] == u) ans++;
 }
 }
 }
 return ans;
}
```

### 9.11.3 复杂度分析

每条边在重定向后成为一条有向边。对于节点  $u$ ，内循环遍历其出边邻居  $v$  的邻居  $w$ ，总 work 为  $\sum_u \sum_{v \in \text{out}(u)} \text{out}(v) = \sum_u (\text{out}(u) \times \sum_{v \in \text{out}(u)} \text{out}(v))$ 。

每条有向边  $(u, v)$  贡献  $\text{out}(v) \times \text{out}(u)$ ，其中  $\text{out}(v) \leq \sqrt{m}$ （若  $\text{out}(v) > \sqrt{m}$ ，则  $v$  的度数更大，不会存在从度数大的点指向度数小的边被反向）。

总复杂度  $O(m \sqrt{m})$ 。

### 9.11.4 扩展：四元环计数

四元环（4-cycle 无弦）也可在  $O(m \sqrt{m})$  内计数。思想类似：按度数排序后标记。

9.12 Meet-in-the-Middle (折半搜索)

9.12.1 核心理念

当搜索空间指数级但可分割时，将问题拆成两半分别搜索，再在中间合并。  
典型复杂度：从  $O(2^n)O(2^n)$  降至  $O(2n/2\log 2n/2)=O(2n/2\cdot n)O(2^{n/2}\log 2^{n/2})=O(2^{n/2}\cdot \log 2^{n/2})$ 。

9.12.2 经典问题

01 背包 (n 小 W 大)

$n\leq 40n\leq 40$ ，容量  $W\leq 109W\leq 10^9$ ，问最大价值。

```
// n <= 40, w[i], v[i], maxW 给定
int meetInMiddleKnapsack(int n, vector<int>& w, vector<int>& v, int maxW) {
 int m = n / 2;
 // 前半部分
 vector<pair<int,int>> left; // (weight, value)
 for (int mask = 0; mask < (1 << m); mask++) {
 int wt = 0, val = 0;
 for (int i = 0; i < m; i++)
 if (mask >> i & 1) { wt += w[i]; val += v[i]; }
 if (wt <= maxW) left.push_back({wt, val});
 }
 sort(left.begin(), left.end());
 // 前缀优化：重量更大的方案至少价值也要更大（去劣）
 vector<pair<int,int>> opt;
 for (auto& [wt, val] : left) {
 if (!opt.empty() && val <= opt.back().second) continue;
 opt.push_back({wt, val});
 }

 // 后半 + 合并
 int ans = 0;
 int rest = n - m;
 for (int mask = 0; mask < (1 << rest); mask++) {
 int wt = 0, val = 0;
 for (int i = 0; i < rest; i++)
 if (mask >> i & 1) { wt += w[m + i]; val += v[m + i]; }
 if (wt > maxW) continue;
 // 二分找最重的 <= maxW - wt
 auto it = upper_bound(opt.begin(), opt.end(), make_pair(maxW - wt, INT_MAX));
 if (it != opt.begin()) {
 --it;
 ans = max(ans, val + it->second);
 }
 }
 return ans;
}
```

方程解计数

如  $a_1x_1+a_2x_2+...+a_nx_n=S$  的解数（ $x_i$  有限制），拆分前后两半枚举。

子集异或和

求  $n$  个数中异或和为  $K$  的子集个数。 $n\leq 40n\leq 40$ 。同样拆半，哈希表合并。

9.12.3 适用条件

9.13 随机化技巧

9.13.1 核心理念

用随机数将确定性困难转化为高概率正确。常用于：哈希判断相等、随机打乱消除最坏情况、随机采样逼近答案。

9.13.2 随机哈希 (XorHash / Zobrist Hashing)

给每种可能的值分配一个随机 64 位整数，集合的哈希值为所有元素的异或和。判断两集合是否相等：比较哈希值，错误概率  $O(2^{-64})O(2^{-64})$ 。

```
// ---- XorHash 基础模板 ----
// 区间可重集哈希：对每种值分配随机哈希，前缀 XOR 维护
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
unordered_map<int, ull> h;
auto get_hash = [&](int val) -> ull {
 if (!h.count(val)) h[val] = rng();
 return h[val];
};

vector<ull> pre(n + 1);
for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] ^ get_hash(a[i]);
// 区间 [l, r] 可重集哈希: pre[r] ^ pre[l-1]

// ---- 树上路径集合哈希 (LCA + 树上前缀 XorHash) ----
// 点权: pre[x] = 根到 x 路径上所有节点哈希的 XOR
// 边权: pre[x] = 根到 x 路径上所有边哈希的 XOR
// 路径 u-v 哈希 = pre[u] ^ pre[v] ^ pre[lca] ^ pre[fa[lca]]

// ---- 排列判等：判断区间是否恰好包含 1..n 各一次 ----
ull target = 0;
for (int i = 1; i <= n; i++) target ^= get_hash(i);
// 区间 [l, r] 为排列 ⇔ r-l+1==n ∧ pre[r]^pre[l-1]==target

// ---- 异或差分 + 随机哈希：边 → 连续区间映射 ----
// 边 (u,v) 影响编号区间 [u, v-1]，用异或差分维护区间内边集
vector<ull> diff(n + 2), cur_hash(n + 1);
for (auto [u, v, eid] : edges) {
 ull w = get_hash(eid);
 diff[u] ^= w, diff[v] ^= w; // 区间 [u, v-1], 右端点 (v-1)+1=v
}
ull cur = 0;
for (int i = 1; i <= n; i++) { cur ^= diff[i]; cur_hash[i] = cur; }
// cur_hash[i] != 0 ⇔ 位置 i 被奇数条边覆盖 (偶数条自动抵消)
```



双重哈希 (Double Hashing)

使用两组独立随机种子，碰撞概率从  $2^{-642}$  降至  $2^{-1282}$ 。

```
struct DoubleHash {
 ull h1, h2;
 bool operator==(const DoubleHash& o) const { return h1 == o.h1 && h2 == o.h2; }
 DoubleHash operator^(const DoubleHash& o) const { return {h1 ^ o.h1, h2 ^ o.h2}; }
};

mt19937_64 rng1(42), rng2(137);
auto get_double_hash = [&](int val) -> DoubleHash {
 static unordered_map<int, DoubleHash> cache;
 if (!cache.count(val)) cache[val] = {rng1(), rng2()};
 return cache[val];
};
```

XorHash 封装模板

```
template <typename T = int>
struct XorHash {
 mt19937_64 rng;
 unordered_map<T, ull> h;
 XorHash() : rng(chrono::steady_clock::now().time_since_epoch().count()) {}

 ull get(T val) {
 if (!h.count(val)) h[val] = rng();
 return h[val];
 }
 vector<ull> build_prefix(const auto& a) { // C++20
 vector<ull> p(sz(a) + 1);
 for (int i = 0; i < sz(a); i++) p[i + 1] = p[i] ^ get(a[i]);
 return p;
 }
 ull query(const vector<ull>& p, int l, int r) { return p[r + 1] ^ p[l]; }
};
```

常见用法：区间可重集判等、树上路径点/边集判等、排列检测、边区间覆盖（异或差分 + 随机哈希）、字符串多重集比较、图同构概率筛（WL 哈希）。

9.13.3 随机打乱 (Random Shuffle)

消除输入顺序带来的最坏情况（快速排序的 pivot、Treap 的随机优先级等）。

```
// 随机打乱数组
shuffle(a + 1, a + n + 1, rng);

// 随机化贪心：多次随机顺序求解，取最优
int best = INF;
for (int T = 0; T < 100; T++) {
 shuffle(order.begin(), order.end(), rng);
 int cur = greedy(order);
 best = min(best, cur);
}
```

9.13.4 随机采样 / 随机 pivot

最小圆覆盖（随机增量）

```
struct Point { double x, y; };
struct Circle { Point o; double r; };

Circle minCircle(vector<Point>& pts) {
 shuffle(pts.begin(), pts.end(), rng);
 Circle c = {pts[0], 0};
 for (int i = 1; i < (int)pts.size(); i++) {
 if (dist(c.o, pts[i]) <= c.r + EPS) continue;
 c = {pts[i], 0};
 for (int j = 0; j < i; j++) {
 if (dist(c.o, pts[j]) <= c.r + EPS) continue;
 c.o = {(pts[i].x + pts[j].x) / 2, (pts[i].y + pts[j].y) / 2};
 c.r = dist(c.o, pts[i]);
 for (int k = 0; k < j; k++) {
 if (dist(c.o, pts[k]) <= c.r + EPS) continue;
 c = circumcircle(pts[i], pts[j], pts[k]);
 }
 }
 }
 return c;
}
```

9.13.5 随机化算法总结

技巧	典型问题	复杂度
XorHash	可重集判等、区间 shuffle 检测	$O(n)O(n)$ 预处理, $O(1)O(1)$ 查询
随机打乱	消除输入序最坏影响、随机化贪心	$O(n)O(n)$ 或 $T \times f(n)$
随机 pivot (QuickSelect)	第 $k$ 小元素	期望 $O(n)O(n)$
随机增量	最小圆覆盖、线性规划 (2D)	期望 $O(n)O(n)$
模拟退火	组合优化、TSP 近似	依迭代次数
随机分治	平面最近点对	$O(n)O(n)$ 期望
Pollard's Rho	大整数分解	$O(n^{1/4})O(n^{1/4})$ 期望
Miller-Rabin	素数判定	$O(k \log 3n)O(k \log^3 n)$

9.14 综合例题

例 1：树上路径第  $k$  小（强制在线）

题意：nn 节点树，点有离散权值。qq 次询问  $u,v,k$  求路径上第  $k$  小的权值。强制在线， $n,q \leq 10^5, q \leq 10^5$ 。

解法：主席树 + LCA。在父节点基础上构建可持久化权值线段树。路径  $(u,v)$  对应的信息为：

$tree(u)+tree(v)-tree(lca)-tree(fa[lca])$   
 $tree(u) + tree(v) - tree(lca) - tree(fa[lca])$

查询时四个版本一起二分。

例 2：动态图连通性（离线）

题意：nn 个点的动态图，mm 次加/删边与询问连通性。

解法：线段树分治 + 可撤销并查集。将每条边的生效时间区间插入线段树，DFS 线段树并维护可撤销并查集。

例 3：区间 Mex（多种离线做法）

做法	复杂度
莫队 + BIT	$O(n\log n)O(n\sqrt{n}\log n)$
莫队 + 分块	$O(nn)O(n\sqrt{n})$
主席树	$O((n+q)\log n)O((n+q)\log n)$ 在线
整体二分	$O((n+q)\log n)O((n+q)\log n)$ 离线
线段树上二分	$O((n+q)\log n)O((n+q)\log n)$

本讲涵盖了竞赛中最常用的 13 种高级技巧，建议按照专题分别针对性刷题，掌握每种技巧的模板和变形。

附录

附录 A：常见错误与注意事项

算法竞赛和刷题过程中，同类型的问题层出不穷。掌握以下常见陷阱，以在考场上避免从零调试的时间浪费。

A.1 整数溢出 (Integer Overflow)

问题描述 在 C++ 中，两个 int 类型相乘时，中间结果也会以 int 类型存储，即使最终赋值给 long long，溢出已经发生。

错误示例

```
int a = 100000, b = 100000;
long long prod = a * b; // 溢出! a*b 先以 int 计算
long long cnt = n * (n - 1) / 2; // 若 n 为 int, 中间结果溢出
```

正确做法

```
long long prod = 1LL * a * b; // 强制提升到 long long
long long cnt = 1LL * n * (n - 1) / 2; // 所有乘法之前先转
#define int long long // 部分选手的做法 (需谨慎使用)
```

原则：只要乘法结果可能超过  $2\times10^9$ （约  $2.1\times10^9$  为 int 上限），就在第一个乘数前乘以 1LL。

A.2 取模负数的处理 (Negative Modulo)

问题描述 C++ 中取模运算符 % 对于负数返回负余数，而非数学意义上的非负余数。这在涉及减法取模时极易出错。

错误示例

```
int a = -5;
int res = a % 3; // C++ 返回 -2, 而非期望的 1
res = (a - b) % MOD; // 若 a-b 为负数, 结果也为负
```

正确做法

```
// 万能取模公式
int safe_mod(int x, int MOD) {
 return (x % MOD + MOD) % MOD;
}

// 常见写法
res = ((a - b) % MOD + MOD) % MOD;

// 多次取模时, 可提前加偏移
res = (a - b + MOD) % MOD; // 仅当 |a-b| < MOD 时安全
```

原则：凡减法后取模，一律用  $(x \% MOD + MOD) \% MOD$ ，不要心存侥幸。

A.3 STL 常见陷阱

STL 虽然方便，但某些接口暗藏性能坑或语义坑。

A.3.1 set::upper\_bound() (成员函数) vs std::upper\_bound() (自由函数)

```
set<int> st = {1, 3, 5, 7, 9};

// 正确: O(log n) - 调用 set 的成员函数
auto it = st.upper_bound(5);

// 错误: O(n) - 对 set 使用通用算法, 退化为线性扫描
auto it2 = std::upper_bound(st.begin(), st.end(), 5);
```

原因：set 底层是红黑树，其迭代器并非随机访问迭代器。std::upper\_bound 在非随机访问迭代器上每次前进都是  $O(1)$  但整体因无法二分而退化为  $O(n)$ 。成员函数利用树的内部结构实现真正  $O(\log n)$ 。

适用范围：set、map、multiset、multimap 均适用此规则。

A.3.2 multiset::count() 的复杂度

```
multiset<int> ms = {1, 1, 1, 2, 2, 3};

// 危险: count(k) 在 C++ 中是 O(log n + count(k)), 可能退化为 O(n)
int c = ms.count(1); // 如果 1 出现了 1e5 次, 这就是 O(1e5)
```

替代方案：若只想知道某个值是否存在，用 ms.find(k) != ms.end(); 若要删除所有等于 k 的元素，用 ms.erase(k) 而非反复 ms.erase(ms.find(k))。

### A.3.3 endl vs \n 导致的 TLE

```
// 极其危险：endl 不仅换行，还会 flush 缓冲区，巨额 I/O 时直接 TLE
for (int i = 0; i < 1e6; i++)
 cout << arr[i] << endl; // 每行都 flush，极慢

// 正确：显式换行，最后统一 flush
for (int i = 0; i < 1e6; i++)
 cout << arr[i] << '\n';
```

加上：

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

可以进一步提升 I/O 速度。注意在此设置之后不要混用 scanf/printf 与 cin/cout。

## A.4 二分查找边界 (Binary Search Bounds)

**问题描述** 二分的三种经典写法边界不同，混淆会陷入死循环或漏解。

**模板对比**

```
// 模板一：寻找第一个 >= target 的位置 (lower_bound)
int l = 0, r = n - 1, ans = n;
while (l <= r) {
 int mid = l + (r - l) / 2; // 防止溢出
 if (a[mid] >= target) {
 ans = mid;
 r = mid - 1; // 收缩右界
 } else {
 l = mid + 1;
 }
}
// 循环结束后 ans 为答案（可能为 n 表示不存在）

// 模板二：寻找最后一个 <= target 的位置 (upper_bound 前驱)
int l = 0, r = n - 1, ans = -1;
while (l <= r) {
 int mid = l + (r - l) / 2;
 if (a[mid] <= target) {
 ans = mid;
 l = mid + 1;
 } else {
 r = mid - 1;
 }
}
```

关于  $mid = (l + r) / 2$  的溢出

```
// 错误：l + r 可能溢出 int / long long
int mid = (l + r) / 2;

// 正确写法之一
int mid = l + (r - l) / 2;
// 或 C++20: int mid = midpoint(l, r);
```

对于 while (l < r) 写法：适用于浮点二分或特定整数场景，需明确区间开闭性。建议初学者统一使用 while (l <= r) + ans 变量的模式，边界最清晰。

## A.5 多测不清空 (Multi-test Clearing)

**问题描述** 同一程序处理多个测试用例时，若未完全清空全局数据结构（vector、数组、邻接表、计数器），上一个用例的残留数据会污染当前用例，产生难以调试的错误。

**错误示例**

```
vector<int> adj[N];
int vis[N];

void solve() {
 int n, m; cin >> n >> m;
 for (int i = 0; i < m; i++) {
 int u, v; cin >> u >> v;
 adj[u].push_back(v);
 }
 // ... 处理
}
// 下一组测试用例时，adj 中仍保留上一组的数据！
```

**正确示例**

```
void solve() {
 int n, m; cin >> n >> m;
 // 只清空会用到的范围，避免每次 O(N) 全清
 for (int i = 1; i <= n; i++) {
 adj[i].clear();
 vis[i] = 0;
 }
 // ... 处理
}
```

常见需要清空的结构：

结构	清空方式
vector<int> adj[N]	for i in 1..n: adj[i].clear()
int vis[N], int cnt[N]	for i in 1..n: vis[i] = 0
set/map 全局变量	.clear()
queue/stack 全局变量	while (!q.empty()) q.pop() 或 = queue<int>()
并查集 fa[N]	for i in 1..n: fa[i] = i
全局计数器变量	重置为 0

**技巧:** 在 Codeforces 等多测题目中, 将 `solve()` 设计为接收参数 `n, m, ...` 的函数, 并在函数开头局部定义数据结构, 利用 RAII 自动析构。若全局数组不可避免, 在 `while (t--)` 循环内手动清空。

A.6 浮点精度 (Floating-Point Precision)

**问题描述** 浮点数在二进制表示中存在舍入误差, 直接比较相等几乎永远失败。

**错误示例**

```
double a = 0.1 + 0.2;
if (a == 0.3) { ... } // 错误! 实际 a ≈ 0.30000000000000004

double lo = 0, hi = 1e9;
while (lo < hi) { ... } // 可能因精度问题死循环
```

**正确做法**

```
const double EPS = 1e-9;

// 比较相等
if (fabs(a - b) < EPS) { ... }

// 比较大小
if (a < b - EPS) { ... } // a 确实小于 b
if (a <= b + EPS) { ... } // a 小于等于 b

// 浮点二分: 固定迭代次数
for (int iter = 0; iter < 100; iter++) {
 double mid = (lo + hi) / 2;
 if (check(mid)) hi = mid;
 else lo = mid;
}

// 能避免浮点就避免: 将浮点运算转为整数
// 如 a <= b/c 改为 a * c <= b (注意溢出)
```

**经验法则:**

误差要求	EPS 取值
1e-6 绝对/相对误差	EPS = 1e-9
1e-9 绝对/相对误差	EPS = 1e-12
一般几何题	EPS = 1e-9 或 1e-10
long double	EPS = 1e-12 或更小

A.7 线段树空间 (Segment Tree Memory)

**问题描述** 线段树所需数组大小经常被低估, 导致 RE (数组越界) 或 WA (数据被覆盖)。

**公式**

```
// 递归线段树 (最常用)
int tree[4 * MAXN]; // 4 倍安全空间

// 迭代线段树 (zkw 风格)
int tree[2 * MAXN]; // 2 倍即可, 需保证 N 为 2 的幂或补全

// 带懒标记的递归线段树
int tree[4 * MAXN], lazy[4 * MAXN];
```

为什么是 4 倍 递归线段树将区间  $[1, n]$  递归分为两半。满二叉树叶子数为不小于  $n$  的最小 2 的幂, 设为  $2^{\lceil \log_2(n) \rceil} \leq 2n$ 。满二叉树总结点数  $= 2 \times \text{叶子数} - 1 \leq 4n$ , 故 4 倍足够覆盖所有情况。在实践中,  $4 * \text{MAXN} + 10$  更为稳妥。

**常见错误**

```
int tree[MAXN * 2]; // 对于递归线段树不够!
int tree[MAXN * 4]; // 正确
```

A.8 图论中的 INF 取值

**问题描述** 最短路径算法 (Dijkstra、Floyd 等) 中若 INF 设置得太小, 松弛时会溢出或被错误跳过; 若太大, INF+边权可能溢出为负数。

**推荐取值**

```
// 无权图 / 边权 ≤ 1e9, 节点数 ≤ 1e5
const long long INF = 1e18;

// 若最坏路径可能超过 1e18 (极少见, 如 Floyd 里三层循环加和)
const long long INF = 1e18; // 仍安全, 因为 1e18 + 1e9 < 9e18 << LLONG_MAX

// 判断不可达
if (dist[v] > INF / 2) { ... } // 而非 dist[v] == INF
```

**错误示例**

```
// 危险：若边长 1e9，路径长可达 1e14，INF=1e9 根本不够
const int INF = 1e9;

// 危险：INF + w 可能溢出为负数
const int INF = 0x3f3f3f3f; // 约 1.06e9，常见但不适用于大图

// 推荐：长整型 INF
const long long INF = 1e18;
```

常用 INF 值一览：

类型	值	适用场景
int INF	0x3f3f3f3f (≈1.06e9)	小图、不超 int 范围的 DP
long long INF	1e18	Dijkstra、Floyd、带权图一般情况
int INF	1e9	某些 DP 和极小图
memset	0x3f (逐字节)	memset(dist, 0x3f, sizeof dist) → 每个 int 约为 0x3f3f3f3f

### A.9 并查集常见陷阱

**问题描述** 错误地直接访问 parent 数组而非通过 find() 函数，导致路径压缩失效或得到过期父节点。

**错误示例**

```
int find(int x) { return fa[x] == x ? x : find(fa[x]); } // 无路径压缩（可用但低效）

// 直接访问父数组
int root = fa[u]; // 错误！可能不是真正的根
if (fa[u] == fa[v]) { ... } // 错误！路径压缩尚未执行
```

**正确示例**

```
int find(int x) {
 return fa[x] == x ? x : fa[x] = find(fa[x]); // 路径压缩
}

// 永远通过 find 获取根
int root_u = find(u);
int root_v = find(v);
if (root_u == root_v) { ... } // 正确

// 合并也通过 find
void unite(int u, int v) {
 u = find(u), v = find(v);
 if (u != v) {
 if (sz[u] < sz[v]) swap(u, v); // 按秩合并
 fa[v] = u;
 sz[u] += sz[v];
 }
}
```

原则：永远不要直接访问 fa[u] 来判断根；使用 find(u) 一步到位。

### A.10 0/1 背包与完全背包的循环方向

**问题描述** 二者代码几乎完全相同，唯一区别在于内层循环的方向，写反即错且极易被忽略。

**0/1 背包（每件物品最多一次）—— 内层递减**

```
for (int i = 1; i <= n; i++) {
 for (int j = W; j >= w[i]; j--) { // 递减，保证只用一次
 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
 }
}
```

**完全背包（每件物品无限次）—— 内层递增**

```
for (int i = 1; i <= n; i++) {
 for (int j = w[i]; j <= W; j++) { // 递增，允许同层多次使用
 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
 }
}
```

**直观理解：**

多重背包：二进制拆分后转化为 0/1 背包处理，内层循环递减。

### A.11 Floyd-Warshall 中 k 循环的顺序

**问题描述** Floyd 算法的三重循环中，k 必须是最外层循环。一旦将 k 放到中层或内层，算法将错误地使用不完全的中间节点信息。

**正确代码**

```
for (int k = 1; k <= n; k++) { // k 必须最外层！
 for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= n; j++) {
 if (dist[i][k] < INF && dist[k][j] < INF)
 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
 }
 }
}
```

**错误代码**

```
for (int i = 1; i <= n; i++)
 for (int j = 1; j <= n; j++)
 for (int k = 1; k <= n; k++) // k 在最内层 — 错误！
 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
```

原理：Floyd 是动态规划，dist[i][j] 在第 k 轮表示“只经过节点 1~k 作为中间节点的最短路径”。k 在外层保证每轮迭代使用的中间节点集逐步扩大。

## A.12 线段树懒标记 (Lazy Propagation)

**问题描述** 在使用带有懒标记的线段树时，任何访问子节点（或对子节点操作）之前，必须先将当前节点的懒标记下推（push down）。否则子节点存储的值是过期的。

**核心规则**

```
void push_down(int p, int l, int r) {
 if (lazy[p]) {
 int mid = (l + r) >> 1;

 // 将懒标记传递给左右子节点
 lazy[p << 1] += lazy[p];
 lazy[p << 1 | 1] += lazy[p];

 // 更新左右子节点的值
 tree[p << 1] += lazy[p] * (mid - l + 1);
 tree[p << 1 | 1] += lazy[p] * (r - mid);

 // 清除当前节点的懒标记
 lazy[p] = 0;
 }
}

void update(int p, int l, int r, int ql, int qr, int val) {
 if (ql <= l && r <= qr) {
 tree[p] += val * (r - l + 1);
 lazy[p] += val;
 return;
 }
 push_down(p, l, r); // 访问子节点前，必须先 push !
 int mid = (l + r) >> 1;
 if (ql <= mid) update(p << 1, l, mid, ql, qr, val);
 if (qr > mid) update(p << 1 | 1, mid + 1, r, ql, qr, val);
 tree[p] = tree[p << 1] + tree[p << 1 | 1];
}

int query(int p, int l, int r, int ql, int qr) {
 if (ql <= l && r <= qr) return tree[p];
 push_down(p, l, r); // 同理，查询时访问子节点也必须 push !
 int mid = (l + r) >> 1;
 int res = 0;
 if (ql <= mid) res += query(p << 1, l, mid, ql, qr);
 if (qr > mid) res += query(p << 1 | 1, mid + 1, r, ql, qr);
 return res;
}
```

**原则：**只要进入子节点（update 递归下行、query 递归下行），就必须先 `push_down`。凡是修改了区间值后，必须向上 `push_up`（即 `tree[p] = tree[p<<1] + tree[p<<1|1]`）。

## A.13 map[] VS map::count() / map::find()

**问题描述** `map[key]` 有一个隐蔽的副作用：如果 `key` 不存在，它会插入一个默认值并与该 `key` 关联。这不仅改变了 `map` 的大小，还可能在遍历时引入垃圾数据。

**错误示例**

```
map<int, int> mp;
mp[1] = 10;

if (mp[2] == 0) { ... } // 错误！这一行在 mp 中插入了 {2, 0}

for (auto &[k, v] : mp) // 现在会遍历到 key=2 的意外条目
 cout << k << " " << v << '\n';
```

**正确示例**

```
// 只查找，不插入
if (mp.count(2)) { ... } // 存在性检查
if (mp.find(2) != mp.end()) { ... } // 等价，可同时获取值
auto it = mp.find(key);
if (it != mp.end()) {
 int val = it->second; // 安全访问
}

// 仅在确定存在后访问
if (mp.count(key)) {
 int val = mp[key]; // 此时安全
}
```

在 C++20 中可以更简洁：

```
if (mp.contains(key)) { ... } // C++20，语义更清晰
```

`unordered_map` 同样适用以上所有规则。

## A.14 静态变量陷阱 (Static Variables in Recursion)

**问题描述** 函数内声明的 `static` 局部变量在多次调用中共享同一实例。递归函数中使用 `static` 变量会导致第二次调用时保留了第一次的状态，产生难以追踪的错误。

**错误示例**

```
void dfs(int u) {
 static int depth = 0; // 危险！所有 dfs 调用共享同一个 depth
 depth++;
 // ... 递归处理
 depth--;
}

// 第二次调用 dfs 时，depth 不是从 0 开始的！
```

**正确示例**

```
void dfs(int u, int depth) { // 通过参数传递状态
 depth++;
 // ... 递归处理
 depth--;
 // 或使用局部变量
}

// 或者将状态放在全局数组并通过参数索引
int depth[N];
void dfs(int u, int d) {
 depth[u] = d;
 for (int v : adj[u])
 if (depth[v] == -1)
 dfs(v, d + 1);
}
```

常见出错场景：

原则：竞赛代码中尽量避免函数内 `static` 局部变量；用参数传递或全局数组代替。

### A.15 运算符优先级陷阱

问题描述 C++ 中部分运算符的优先级与直觉不符，省略括号会导致表达式被错误解析，且编译不报错。

高危运算符（优先级低于直觉）

```
// 1. 位与 & 的优先级低于 ==
if (x & 1 == 0) { ... } // 被解析为 x & (1 == 0) → x & false → 0
if ((x & 1) == 0) { ... } // 正确写法

// 2. 异或 ^ 的优先级低于 <
if (a ^ b < c) { ... } // 解析为 a ^ (b < c)，很可能不是你的意图
if ((a ^ b) < c) { ... } // 正确写法

// 3. 左移/右移 的优先级低于 +
int mask = 1 << k + 1; // 解析为 1 << (k + 1)
int mask = (1 << k) + 1; // 或 1 << (k+1) 视语义而定

// 4. 三目运算符嵌套
int res = a ? b : c ? d : e; // 解析为 a ? b : (c ? d : e)
int res = (a ? b : c) ? d : e; // 按需加括号
```

黄金法则：

当涉及位运算（`&`，`|`，`^`，`<<`，`>>`）与比较/算术运算符混合时，**一律加括号**。多写一对括号的代价远低于赛后 WA 后逐行查错。

经验口诀：

## 附录 B：来源与致谢

本模板库在整理过程中参考了大量社区开源资源与个人选手的模板仓库。感谢每一位贡献者的无私分享。

主要参考项目

项目	链接	协议	说明
KACTL	github.com/kth-competitive-programming/kactl	CC0	KTH 皇家理工学院 ICPC 团队的竞赛模板库，质量极高，涵盖几乎所有算法竞赛核心主题
AtCoder Library (ACL)	github.com/atcoder/ac-library	CC0	AtCoder 官方算法库，包含经过工业级测试的 SegTree、LazySegTree、Fenwick Tree、MCFGraph 等实现
OI-wiki	oi-wiki.org / github.com/OI-wiki/OI-wiki	CC BY-SA 4.0	中文算法竞赛知识库，涵盖从入门到 ICPC 级别的系统知识，本站的理论讲解主要参考此处
cp-algorithms	cp-algorithms.com / github.com/cp-algorithms/cp-algorithms	CC BY-SA 4.0	英文算法竞赛百科，e-maxx 的英文翻译与扩充版本，数学和字符串部分的推导尤为详尽
hh2048/XCPC (jiangly 模板)	github.com/hh2048/XCPC	GPL-3.0	集中收录 jiangly 等顶尖选手在 XCPC 竞赛中使用的模板，代码风格简洁现代
ShahjalalShohag/code-library	github.com/ShahjalalShohag/code-library	MIT	主题全面、注释丰富的算法代码库，数论和组合数学部分尤为出色
hourai (HIT hourai)	github.com/DWaveletT/hourai	CC BY-SA 4.0	哈尔滨工业大学 hourai 队伍的竞赛模板，数据结构与图论实现质量高
old-yan/CP-template	github.com/old-yan/CP-template	—	结构清晰的中文竞赛模板，适合初学者参考
f_zyj ACM 模板	github.com/snake-lvyonghao/ACM	—	经典 ACM 模板库，广泛流传于中文算法竞赛圈

其他辅助资源

以下平台和工具在本模板的整理、测试与排版过程中提供了重要支持：

特别致谢

感谢以下个人和组织对算法竞赛社区的持续贡献（排名不分先后）：

<b>协议说明：</b> 本模板库中参考或直接引用的外部代码均标注其来源与原始协议。若您发现任何遗漏标注或协议冲突，欢迎通过 issue 或 PR 联系我们进行修正。 本模板库的原创部分遵循与项目整体一致的协议发布。
-----------------------------------------------------------------------------------------------------------------

最后更新于 2026 年 8 月